

Timeline Compiler

Technical Specification & Architecture

Timeline Compiler Project

June 2026

Contents

0.1	Problem Definition	10
0.1.1	The Communication Gap	10
0.1.2	Why Existing Tools Are Insufficient	10
0.1.2.1	Mermaid and Documentation-Grade Diagramming . . .	10
0.1.2.2	Project Management Tools	11
0.1.2.3	Design and Presentation Tools	11
0.1.2.4	AI Agent Blindspot	11
0.1.3	Who This Is For	12
0.1.3.1	Human Authors	12
0.1.3.2	AI Agents	12
0.1.4	What Success Looks Like	12
0.1.4.1	Scenario: Product Roadmap	12
0.1.4.2	Scenario: Transformation Plan	13
0.1.4.3	Success Criteria	14
0.1.5	What This Is Not	14
0.2	Design Principles	15
0.2.1	Presentation-First	15
0.2.2	Deterministic Output	15
0.2.3	Human-Readable Source	15
0.2.4	Agent-Friendly Authoring	16
0.2.5	Theme-Driven Rendering	16
0.2.6	Separation of Planning and Rendering	17
0.2.7	Minimal Visual Clutter	17
0.2.8	Source-Control Friendly	17
0.2.9	Composability	18
0.2.10	Graceful Degradation	18

0.2.11	Extensibility Without Complexity	19
0.2.12	Principle Summary	19
0.3	Scope Boundaries	19
0.3.1	Governing Principle	19
0.3.2	In Scope	20
0.3.2.1	Visual Artifact Types	20
0.3.2.2	IR Elements	20
0.3.2.3	Presentation Features	21
0.3.2.4	Output Formats	21
0.3.2.5	Tooling	21
0.3.3	Out of Scope	22
0.3.3.1	Project Management Semantics	22
0.3.3.2	Data Management	23
0.3.3.3	Interactive Features	23
0.3.3.4	Data Ingestion	23
0.3.4	Boundary Cases	23
0.3.4.1	Dependency Arrows (Visual Only)	23
0.3.4.2	Today Line	24
0.3.4.3	Auto-Layout	24
0.3.4.4	External Image Embedding	24
0.3.5	Scope Summary	24
0.4	Timeline IR	24
0.4.1	Design Principles	24
0.4.2	Type Vocabulary	25
0.4.3	Document Structure	25
0.4.3.1	Root Fields	26
0.4.4	Metadata	26
0.4.4.1	TimeRange	27
0.4.4.2	AxisUnit	27
0.4.4.3	Date Anchor and Fiscal Calendar	27

0.4.5	Date Model	28
0.4.5.1	Date Representations	28
0.4.5.2	Uncertain and Open Dates	29
0.4.5.3	DateRange	29
0.4.6	Tracks (Swimlanes)	30
0.4.7	Groups	30
0.4.8	Activities	31
0.4.8.1	Status Enum	32
0.4.8.2	Progress	33
0.4.9	Milestones	33
0.4.10	Annotations	34
0.4.10.1	Annotation Types	34
0.4.11	Sections	34
0.4.12	Legend	35
0.4.13	ID and Reference System	35
0.4.13.1	Identifier Format	35
0.4.13.2	References	36
0.4.13.3	Reference Namespacing	37
0.4.14	Well-Formedness Rules	37
0.4.14.1	Structural Invariants	37
0.4.14.2	Referential Invariants	37
0.4.14.3	Temporal Invariants	37
0.4.14.4	Value Invariants	38
0.4.14.5	Soft Warnings (Valid but Suspicious)	38
0.4.15	Complete Examples	38
0.4.15.1	Example 1: Product Roadmap	38
0.4.15.2	Example 2: Executive Program Timeline	40
0.4.15.3	Example 3: Architecture Evolution Timeline	42
0.4.16	Extensibility	45
0.4.16.1	Metadata Extension	45

0.4.16.2	Custom Categories	45
0.4.16.3	Version Compatibility	45
0.4.17	Serialization and Syntax	46
0.4.17.1	Canonical Format	46
0.4.17.2	Alternative Formats	46
0.4.17.3	Git Friendliness	46
0.4.18	Validation	47
0.4.18.1	Error Reporting	47
0.4.18.2	Strict vs. Lenient Mode	47
0.4.19	Relationship to Other Components	47
0.4.20	Build vs. Adopt: A Survey of Existing Representations	48
0.4.20.1	Timeline Text Languages	48
0.4.20.2	Visualization Grammars	49
0.4.20.3	Timeline Data Models	49
0.4.20.4	Temporal and Calendar Standards	50
0.4.20.5	Scheduling and PM Formats	50
0.4.20.6	Analogous Document IRs	51
0.4.20.7	Comparison Table	51
0.4.20.8	Recommendation: BUILD with Borrowed Vocabulary	51
0.4.20.9	Alignment of Mark's IR with Prior Art	52
0.5	Rendering Model	53
0.5.1	Determinism Contract	53
0.5.2	Coordinate System	54
0.5.3	Timeline Axis	55
0.5.3.1	Axis Unit and Inference	55
0.5.3.2	Date-to-Coordinate Function	55
0.5.3.3	Date Coercion for Bar Edges	55
0.5.3.4	Tick and Gridline Placement	55
0.5.4	The Six-Phase Layout Algorithm	56
0.5.4.1	Phase 1: Axis Computation	57

0.5.4.2	Phase 2: Track Placement	57
0.5.4.3	Phase 3: Activity Geometry	57
0.5.4.4	Phase 4: Milestone Geometry	59
0.5.4.5	Phase 5: Sections and Annotations	59
0.5.4.6	Phase 6: Label Collision Resolution	60
0.5.5	Scene / Render IR — Output of the Pipeline	60
0.5.6	Edge Cases	61
0.5.7	Known IR Gaps	63
0.6	Theme Architecture	64
0.6.1	Theme Design Philosophy	64
0.6.2	Theme Schema	64
0.6.2.1	Canvas and Margins	65
0.6.2.2	Typography	65
0.6.2.3	Axis Styling	65
0.6.2.4	Track Styling	66
0.6.2.5	Activity Styling	66
0.6.2.6	Milestone Styling	67
0.6.2.7	Annotation and Legend Styling	67
0.6.2.8	Status-to-Style Map	67
0.6.2.9	Category-to-Style Map	68
0.6.2.10	Fidelity Tier and Effect Knobs	68
0.6.3	Built-in Themes	69
0.6.3.1	Consulting Theme	69
0.6.3.2	Executive Theme	70
0.6.3.3	Product Theme	70
0.6.3.4	Release Theme	71
0.6.3.5	Minimal Theme	72
0.6.3.6	Showcase / Keynote Theme	72
0.6.4	Cross-Theme Visual Comparison	73
0.6.5	Fidelity Tiers and Backend Effect Profiles	73

0.6.6	Theme Inheritance and Extension	74
0.6.7	Theme-Engine Contract (Formal)	75
0.7	Output Targets	75
0.7.1	Scene / Render IR	76
0.7.1.1	Scene Model	76
0.7.1.2	Scene Determinism	77
0.7.2	SVG Backend	78
0.7.3	Raster Backend — High-Fidelity Art Effects	80
0.7.4	PPTX — Native-Shape Backend	81
0.7.5	HTML — Interactive Backend	82
0.7.6	Backend Capability and Fidelity-Tier Table	83
0.7.7	Output Priority Recommendation	83
0.7.8	Format Comparison Summary	84
0.8	Distribution Strategy	84
0.8.1	Distribution Requirements	84
0.8.2	Packaging Options	85
0.8.2.1	Core Library	85
0.8.2.2	Command-Line Interface	86
0.8.2.3	VS Code Extension	86
0.8.2.4	MCP Server (Model Context Protocol)	87
0.8.2.5	NPM Package for Embedding	87
0.8.2.6	Standalone Go Binary	88
0.8.3	Implementation Language Trade-offs	88
0.8.4	Recommended Architecture	89
0.8.5	Versioning and Compatibility	90
0.8.6	Distribution Priorities	90
0.9	Agent Integration	90
0.9.1	The Ingestion Contract	91
0.9.1.1	The Prompt as First-Class Input	91
0.9.1.2	What Ingestion May Not Do	91

0.9.2	Ingestion Workflows	91
0.9.2.1	Azure DevOps Work Items	91
0.9.2.2	Natural Language Prose	93
0.9.2.3	GitHub Projects	94
0.9.2.4	Mermaid Timeline Syntax	95
0.9.3	Reliable IR Generation by Agents	96
0.9.3.1	JSON Schema Publication	96
0.9.3.2	Structured Output Constraints	98
0.9.3.3	IR Design Choices That Help Agents	99
0.9.4	Validation Pipeline	99
0.9.4.1	Errors vs. Warnings	100
0.9.5	Error Handling and Agent Repair Cycle	100
0.9.5.1	Error Message Format	100
0.9.5.2	Agent Repair Cycle	101
0.9.6	MCP Server	102
0.9.6.1	Tool Contracts	102
0.9.6.2	MCP Server Deployment	104
0.9.7	Round-Trip and Iterative Editing	105
0.9.7.1	Source Provenance via Metadata	105
0.9.7.2	Re-Sync Strategy	106
0.9.7.3	Stable IDs and Git-Friendly YAML	106
0.9.7.4	Human Edits and Incremental Refinement	107
0.9.8	IR Gaps and Open Questions	107
0.10	Comparison Analysis	108
0.10.1	Framing the Comparison	108
0.10.2	Tool-by-Tool Analysis	108
0.10.2.1	Mermaid (timeline and gantt)	108
0.10.2.2	PlantUML	109
0.10.2.3	vis-timeline	109
0.10.2.4	Frappe Gantt	110

0.10.2.5	Microsoft Project	110
0.10.2.6	think-cell	110
0.10.2.7	PowerPoint Timeline (native SmartArt)	110
0.10.3	Comparison Table	111
0.10.4	Where Timeline Compiler Intentionally Differs	111
0.10.5	Real-Data Crawl: Practical Patterns Observed	112
0.11	MVP Design	112
0.11.1	MVP Philosophy	112
0.11.2	Feature Prioritization	113
0.11.2.1	Must Have (P0)	113
0.11.2.2	Should Have (P1)	113
0.11.2.3	Nice to Have (P2)	114
0.11.2.4	Future (P3)	114
0.11.3	MVP Scope Summary	115
0.11.4	Phased Roadmap	115
0.11.4.1	Phase 1: Core Engine (MVP Launch)	115
0.11.4.2	Phase 2: Polish and Usability	115
0.11.4.3	Phase 3: Ecosystem	116
0.11.4.4	Phase 4: Scale and Community	116
0.11.5	Risks and Mitigations	116
0.11.5.1	Technical Risks	116
0.11.5.2	Adoption Risks	117
0.11.5.3	Complexity Hotspots	117
0.11.6	Smallest Viable Version	117
0.11.7	Success Metrics	118
0.12	Open-Source Strategy	118
0.12.1	Is This a Viable Open-Source Project?	118
0.12.2	Closest Existing Projects and the Gap They Leave	118
0.12.3	Potential Adopters	119
0.12.4	Ecosystem Fit	120

0.12.4.1	Mermaid / Markdown / CI Ecosystem	120
0.12.4.2	MCP / Agent Ecosystem	120
0.12.4.3	PPTX Ecosystem	120
0.12.5	Extension Opportunities	120
0.12.6	Strongest Differentiators	121
0.12.7	Community Contribution Model	121
0.12.8	Adoption Risk Assessment	122
0.13	Timeline Grammar vs. Task Scheduling Grammar	123
0.13.1	The Thesis	123
0.13.2	Two Grammars Compared	123
0.13.2.1	Task Scheduling Grammar	123
0.13.2.2	Timeline Grammar	124
0.13.3	Why Gantt-Thinking Is Wrong Here	124
0.13.3.1	Computational Complexity	124
0.13.3.2	Semantic Overload	125
0.13.3.3	Schema Bloat	125
0.13.3.4	Agent Hostility	125
0.13.3.5	The Gantt Trap	126
0.13.4	Timeline Grammar: Design Implications	126
0.13.4.1	IR Simplicity	126
0.13.4.2	No Computation	126
0.13.4.3	Visual-First Extensions	127
0.13.4.4	Determinism by Default	127
0.13.5	Addressing the Counter-Arguments	127
0.13.5.1	“But I need dependency arrows”	127
0.13.5.2	“What if my dates are inconsistent?”	128
0.13.5.3	“Project management tools already have timeline views”	128
0.13.5.4	“Won’t users expect scheduling features?”	128
0.13.6	Optimization Goals Alignment	128
0.13.7	Conclusion	129

0.1 Problem Definition

0.1.1 The Communication Gap

Executives, product managers, consultants, and program leads communicate plans through visual timelines. These artifacts appear in board presentations, investor decks, quarterly reviews, transformation roadmaps, and architecture evolution documents. The quality of these visuals directly affects stakeholder comprehension and decision-making.

Yet producing presentation-quality timelines remains surprisingly difficult. The gap exists because available tools fall into two categories, each failing half the requirement:

Documentation Tools

(Mermaid, PlantUML, text-based DSLs) — machine-readable, version-controllable, agent-friendly, but produce diagrams suited for technical documentation rather than executive communication.

Design Tools

(PowerPoint, Visio, Lucidchart, think-cell, Figma) — produce polished visuals, but require significant manual effort, resist automation, and cannot be reliably generated by AI agents.

Timeline Compiler bridges this gap: a compiler target for structured plan data that produces presentation-quality visual output.

0.1.2 Why Existing Tools Are Insufficient

0.1.2.1 Mermaid and Documentation-Grade Diagramming

Mermaid [20] has become the de facto standard for diagrams-as-code in technical documentation. Its Gantt syntax illustrates the limitation:

```
1 gantt
2   title Product Roadmap Q3
3   dateFormat YYYY-MM-DD
4   section Platform
5     Core API :a1, 2026-07-01, 30d
6     Mobile SDK :a2, after a1, 45d
7   section Integrations
8     Partner A :b1, 2026-08-01, 60d
```

Listing 1: Mermaid Gantt — functional but not presentation-grade

Mermaid excels at documentation — syntax is simple, output is deterministic, rendering integrates with GitHub, GitLab, and Notion. But the output is not presentation-grade:

- Fixed visual style optimized for legibility in documentation contexts
- No theming beyond basic color overrides
- Limited typographic control

- Dependency-centric (task scheduling) rather than communication-centric (timeline narrative)
- No built-in support for swimlanes as first-class visual groupings
- No annotations, callouts, or emphasis mechanisms for executive communication

The result: Mermaid diagrams are useful in READMEs and wikis but inappropriate for board decks and investor presentations.

0.1.2.2 Project Management Tools

Tools like Microsoft Project, Jira, Asana, Monday.com, and Linear are optimized for *managing work*, not *communicating plans*. Their timeline views:

- Export poorly or not at all to static visual formats
- Embed project-management semantics (resources, dependencies, critical paths) that clutter executive communication
- Assume ongoing interaction rather than snapshot communication
- Produce visuals tied to the tool’s aesthetic, not the organization’s brand

A quarterly roadmap exported from Jira looks like a Jira export, not like a McKinsey slide.

0.1.2.3 Design and Presentation Tools

PowerPoint, Keynote, Visio, Figma, and Lucidchart produce beautiful results — but at the cost of manual effort. Each timeline is a bespoke artifact:

- No structured data input; changes require manual redrawing
- Not version-controllable in meaningful ways (binary or JSON blobs)
- Not agent-accessible; an AI cannot reliably produce a Visio file
- Theming requires manual application
- Determinism is impossible; two designers will produce different artifacts from the same brief

This creates a maintenance burden. When scope shifts, the timeline must be manually updated. When quarterly plans roll forward, someone rebuilds the slide.

0.1.2.4 AI Agent Blindspot

Modern AI agents (LLMs with tool use, planning agents, copilots) can generate structured plans — sprints, phases, milestones, dependencies. But they have no widely-adopted open-source target for rendering those plans into visual artifacts.

An agent can output JSON or YAML describing a roadmap. But then what? Without a compiler that accepts structured input and produces presentation-grade output, the agent’s plan stops at structured text.

0.1.3 Who This Is For

Timeline Compiler serves two complementary audiences:

0.1.3.1 Human Authors

Executives and Chiefs of Staff

— Need polished quarterly plans, transformation roadmaps, and board-ready timelines without designer dependency.

Product Managers

— Need roadmaps that communicate priority and sequencing to stakeholders, not dependency graphs for engineers.

Consultants

— Need branded deliverables (BCG-style roadmaps, McKinsey program timelines) that are data-driven yet visually refined.

DevRel and Technical Writers

— Need version-controlled timelines that embed in documentation but also export to slides.

Program Managers

— Need milestone views and phase summaries that roll up operational detail into executive narrative.

0.1.3.2 AI Agents

AI agents require:

- A well-defined intermediate representation (IR) they can generate
- Deterministic rendering — the same IR always produces the same visual
- Theming separation — agents produce structure, not style
- Validation — agents can verify their output conforms to the schema
- Embeddability — agents can invoke rendering as a tool

Timeline Compiler’s IR is designed as a *compiler target*: a stable, documented, validatable representation that agents can emit and humans can inspect, version, and override.

0.1.4 What Success Looks Like

0.1.4.1 Scenario: Product Roadmap

A product manager writes (or an agent generates):

```
1 timeline:
2   title: "Platform Roadmap 2026"
3   range: { start: 2026-Q1, end: 2026-Q4 }
4   tracks:
5     - id: platform
```

```

6     label: "Core Platform"
7     activities:
8       - label: "API v2"
9         range: { start: 2026-01, end: 2026-03 }
10        status: complete
11       - label: "SDK Release"
12         range: { start: 2026-04, end: 2026-06 }
13         status: in-progress
14         progress: 0.6
15     - id: mobile
16       label: "Mobile"
17       activities:
18         - label: "iOS App"
19           range: { start: 2026-05, end: 2026-08 }
20           status: planned
21     milestones:
22       - label: "Public Beta"
23         date: 2026-06-15
24         tracks: [platform, mobile]

```

Listing 2: Product Roadmap IR Example

The compiler, given a theme (e.g., `corporate-blue`), produces a PDF suitable for a board presentation: clean typography, proper spacing, consistent color palette, no visual noise.

0.1.4.2 Scenario: Transformation Plan

A management consultant defines a multi-year digital transformation:

```

1  timeline:
2    title: "Digital Transformation – 3 Year Plan"
3    range: { start: 2026-01, end: 2028-12 }
4    sections:
5      - id: foundation
6        label: "Foundation"
7        range: { start: 2026-01, end: 2026-12 }
8      - id: scaling
9        label: "Scaling"
10       range: { start: 2027-01, end: 2027-12 }
11      - id: optimization
12        label: "Optimization"
13        range: { start: 2028-01, end: 2028-12 }
14    tracks:
15      - id: technology
16        label: "Technology"
17        activities:
18          - label: "Cloud Migration"
19            range: { start: 2026-03, end: 2026-09 }
20            section: foundation
21          - label: "Platform Modernization"
22            range: { start: 2027-01, end: 2027-09 }
23            section: scaling
24      - id: organization
25        label: "Organization"
26        activities:
27          - label: "Team Restructuring"
28            range: { start: 2026-06, end: 2026-12 }

```

```
29         section: foundation
30     annotations:
31     - label: "Board Checkpoint"
32       date: 2026-12-15
33       style: callout
```

Listing 3: Transformation Timeline IR Example

The output: a three-year view with clear phase demarcation, swimlanes for workstreams, and a visual hierarchy that communicates strategic sequencing rather than tactical dependencies.

0.1.4.3 Success Criteria

Timeline Compiler succeeds when:

1. A human can write the IR by hand in under 10 minutes for a typical roadmap
2. An AI agent can generate valid IR without special prompting beyond the schema
3. The same IR, rendered with the same theme, produces byte-identical output across runs (determinism)
4. A non-designer can produce board-ready visuals by selecting a theme
5. The IR is diff-friendly — changes to the plan produce readable diffs
6. The output is accepted in contexts where PowerPoint slides are currently required

0.1.5 What This Is Not

To prevent scope creep and misunderstanding:

- **Not a project management tool** — does not track work, assign resources, or compute schedules
- **Not a Gantt chart replacement** — though it can render timeline views, it is not optimized for dependency visualization
- **Not a Jira/Asana competitor** — does not replace work-tracking systems; may consume data from them
- **Not a design tool** — does not provide a canvas, drag-and-drop, or WYSIWYG editing
- **Not interactive** — output is static (SVG, PNG, PDF, PPTX); interactivity is out of scope

Timeline Compiler is a *compiler*: structured input in, presentation-quality visual output out. The value is in the deterministic, themeable, agent-accessible transformation — not in the tooling around authoring or managing plans.

0.2 Design Principles

The following principles govern Timeline Compiler’s architecture. They are listed in priority order; when principles conflict, earlier principles take precedence.

0.2.1 Presentation-First

Principle: Every design decision optimizes for presentation-quality visual output.

Rationale: The distinguishing feature of Timeline Compiler is producing visuals suitable for board presentations, investor decks, and executive communication. This is not a documentation tool (Mermaid serves that purpose) nor a project management tool (many exist). If a feature does not improve presentation quality, it should not exist.

Implications:

- Typography, spacing, and color receive first-class treatment in the theme engine
- The IR includes presentation hints (emphasis, annotations, callouts) that have no project-management semantics
- Output formats prioritize fidelity (PDF, SVG) over convenience

0.2.2 Deterministic Output

Principle: The same IR plus the same theme produces byte-identical output across all runs.

Rationale: Determinism enables:

- Version control — changes to the IR produce predictable visual diffs
- CI/CD integration — automated rendering in pipelines
- Agent reliability — agents can trust their output
- Reproducibility — artifacts can be regenerated exactly

Implications:

- No random layout algorithms; all positioning is computed deterministically
- Timestamps, UUIDs, and other entropy sources are excluded from output
- Font metrics must be stable (system fonts are problematic; embedded or specified fonts are required)
- Floating-point operations must be reproducible (or avoided)

0.2.3 Human-Readable Source

Principle: The IR must be readable and writable by humans without tooling.

Rationale: The IR serves dual audiences (humans and agents). Human readability ensures:

- Onboarding friction is low — a PM can write a roadmap without learning a complex DSL
- Debugging is possible — when output is wrong, the IR is inspectable
- Manual overrides are feasible — humans can adjust agent-generated IR
- Version control diffs are meaningful

Implications:

- YAML or YAML-like syntax preferred over JSON (less visual noise)
- Field names are descriptive English, not abbreviations
- Nesting depth is minimized
- Dates use ISO 8601; durations use human-readable formats where possible

0.2.4 Agent-Friendly Authoring

Principle: AI agents must be able to generate valid IR without special prompting or examples.

Rationale: A primary use case is agent-generated timelines. If agents struggle to produce valid IR, the system fails its purpose.

Implications:

- The IR has a formal schema (JSON Schema or equivalent) that agents can reference
- Field names and structure align with how LLMs naturally express plans
- The IR avoids footguns — common mistakes should be invalid or auto-corrected
- Validation provides clear, actionable error messages
- The schema is small enough to fit in a context window

0.2.5 Theme-Driven Rendering

Principle: Visual styling is entirely separated from content; all presentation decisions flow from the theme.

Rationale: Separation of content and presentation enables:

- Rebranding without changing content — swap themes, regenerate
- Consistent organizational aesthetics across timelines
- Agents authoring content without making design decisions
- Themes as a distribution mechanism (theme marketplaces, branded themes)

Implications:

- The IR contains no colors, fonts, or sizes — only semantic hints
- Themes define all visual properties
- The rendering model maps semantic IR elements to themed visual primitives
- Inline style overrides, if permitted, are explicit escapes from theme control

0.2.6 Separation of Planning and Rendering

Principle: The IR represents *what to communicate*, not *how to compute it*.

Rationale: Timeline Compiler is a rendering compiler, not a scheduling engine. The IR is a communication artifact, not a project plan. This separation:

- Keeps the IR minimal and focused
- Avoids reimplementing project-management semantics
- Allows integration with any upstream planning tool
- Clarifies scope boundaries

Implications:

- No dependency resolution, critical path calculation, or resource leveling
- Dates are provided, not computed
- Progress is asserted, not measured
- The IR describes the *output* of planning, not the *inputs* to planning

0.2.7 Minimal Visual Clutter

Principle: Default output favors clarity over information density.

Rationale: Presentation-quality visuals communicate one message clearly rather than many messages densely. Executive audiences prefer clean visuals over comprehensive ones.

Implications:

- Default themes use generous whitespace
- Labels are concise; detail lives in supporting materials
- Gridlines, axes, and chrome are minimized by default
- Information hierarchy is clear — not all elements receive equal visual weight

0.2.8 Source-Control Friendly

Principle: The IR is designed for version control workflows.

Rationale: Modern teams version everything. If the IR produces noisy diffs or merge conflicts, adoption suffers.

Implications:

- Text-based format (YAML preferred)
- Stable field ordering (or sorted keys)
- No generated IDs unless explicitly authored
- Changes to one element do not cascade spuriously to others
- Line-oriented structure where possible (each item on its own line)

0.2.9 Composability

Principle: Timeline definitions can reference, include, and compose other definitions.

Rationale: Real-world use involves:

- Shared track definitions across timelines
- Theme inheritance and overrides
- Milestone libraries
- Multi-file organization for large timelines

Implications:

- Support for file includes or references
- Namespacing to avoid collisions
- Clear resolution rules for overrides
- Themes can extend other themes

0.2.10 Graceful Degradation

Principle: Invalid or incomplete IR produces useful output or clear errors, not silent failures.

Rationale: Authors (especially agents) make mistakes. The system should:

- Validate early and report clearly
- Render partial output when possible (with warnings)
- Never produce corrupt or misleading visuals silently

Implications:

- Schema validation runs before rendering
- Semantic validation catches contradictions (e.g., end before start)
- Warnings are visible but do not block output when the issue is cosmetic
- Errors specify exactly what is wrong and where

0.2.11 Extensibility Without Complexity

Principle: The IR and theme systems support extension without requiring core changes.

Rationale: Long-term success requires community contribution — new themes, new semantic elements, integration with new upstream tools. But extension points must not compromise core simplicity.

Implications:

- Unknown IR fields are ignored (forward compatibility)
- Themes support custom properties via namespacing
- Plugin or extension architecture for renderers (future)
- Versioned schemas with clear migration paths

0.2.12 Principle Summary

Table 1: Design Principles Summary

Principle	Core Commitment
Presentation-First	Optimize for board-ready visuals
Deterministic Output	Same input, same output, always
Human-Readable Source	Writable without tooling
Agent-Friendly Authoring	Agents generate valid IR naturally
Theme-Driven Rendering	Content and style fully separated
Separation of Planning and Rendering	Describe output, not compute schedule
Minimal Visual Clutter	Clarity over density
Source-Control Friendly	Clean diffs, easy merges
Composability	Reference, include, extend
Graceful Degradation	Clear errors, useful partial output
Extensibility Without Complexity	Extend without core changes

0.3 Scope Boundaries

This section defines explicit boundaries for Timeline Compiler. Clarity here prevents scope creep and ensures the system remains focused on its core value proposition: transforming structured plan descriptions into presentation-quality visual timelines.

0.3.1 Governing Principle

The scope boundary is defined by the *rendering abstraction*: Timeline Compiler transforms a communication-oriented intermediate representation into visual output. It does not generate, compute, or manage the plan data itself.

If a feature requires understanding the semantics of work — dependencies, resources, costs, velocity — it is out of scope. If a feature improves the visual communication of a plan already decided, it is potentially in scope.

0.3.2 In Scope

The following capabilities are within Timeline Compiler's scope:

0.3.2.1 Visual Artifact Types

Timelines

Horizontal time-based visualizations showing activities, milestones, and phases across a date range.

Roadmaps

Product or program roadmaps with tracks (swimlanes), grouped activities, and strategic milestones.

Milestone Views

Focused views highlighting key dates and deliverables without detailed activity bars.

Phase Summaries

High-level views showing phases or stages of a program without granular activities.

Swimlane Diagrams

Multi-track views where parallel workstreams are visually separated.

0.3.2.2 IR Elements

The IR supports the following semantic elements (detailed in Section 0.4):

tracks

Named swimlanes or rows that group related activities. Tracks provide visual and semantic organization.

groups

Logical groupings within or across tracks for nested organization (e.g., workstreams within a track).

activities

Time-bounded items (bars) representing work, initiatives, or phases. Activities have start/end dates, labels, and optional metadata.

milestones

Point-in-time markers representing key dates, deliverables, or decisions.

sections

Temporal divisions of the timeline (e.g., quarters, phases) that span all tracks.

range

The overall time window for the visualization.

date ranges

Start and end dates for activities and sections. Supports dates, months, quarters, and years.

progress

Numeric (0.0–1.0) indication of completion for activities. Purely visual — not computed.

status

Semantic status indicators (e.g., planned, in-progress, complete, at-risk, blocked).

labels

Text labels for all elements. Brief, presentation-appropriate text.

annotations

Callouts, notes, or emphasis markers attached to dates or elements.

legends

Explanatory legends for status colors, track semantics, or custom indicators.

0.3.2.3 Presentation Features**Theming**

Complete visual customization via theme files: colors, typography, spacing, shapes.

Emphasis

Visual mechanisms to highlight specific activities or milestones (e.g., bold, glow, callout).

Today Markers

Visual indicator for the current date (provided as input, not computed).

Progress Visualization

Visual representation of progress within activity bars.

Status Indicators

Color or icon mapping for semantic statuses.

Annotations

Callouts, arrows, or labels that highlight specific points.

0.3.2.4 Output Formats**SVG**

Primary vector output for web embedding and further processing.

PNG

Rasterized output for contexts requiring bitmap images.

PDF

Print-quality vector output for documents and decks.

PPTX

PowerPoint-compatible output for direct use in presentations.

0.3.2.5 Tooling**CLI**

Command-line interface for rendering (`timeline render input.yaml -o output.pdf`).

Library API

Programmatic access for embedding in other tools.

Schema Validation

Validation of IR against formal schema.

Theme Validation

Validation of theme files.

0.3.3 Out of Scope

The following capabilities are explicitly excluded:

0.3.3.1 Project Management Semantics

Dependency Scheduling

Timeline Compiler does not compute dates from dependencies. If Activity B depends on Activity A, the author must provide both dates explicitly. The IR may optionally *record* dependencies for visual rendering (e.g., arrows), but it does not *resolve* them.

Rationale: Dependency resolution is core project-management logic. It requires understanding task semantics, durations, constraints, and often iterative solving. This is the domain of MS Project, Primavera, and similar tools. Timeline Compiler consumes the *output* of scheduling, not the inputs.

Resource Management

No support for assigning people, teams, or resource pools to activities. No resource leveling or capacity planning.

Rationale: Resource management requires organizational context (who is available, at what capacity, with what skills) that is outside the scope of a rendering compiler.

Critical Path Analysis

No calculation of critical paths or slack time.

Rationale: Critical path analysis is a scheduling concern, not a rendering concern. Upstream tools compute this; Timeline Compiler may visualize the result if provided.

Sprint/Iteration Tracking

No support for agile sprints, velocity tracking, burndowns, or iteration planning.

Rationale: Sprint tracking is operational work management. Timeline Compiler visualizes strategic communication, not operational tracking.

Cost Management

No budgets, cost tracking, earned value, or financial projections.

Rationale: Financial data requires integration with accounting and project-control systems. This is far outside the rendering scope.

Portfolio Optimization

No multi-project prioritization, resource allocation across projects, or strategic portfolio balancing.

Rationale: Portfolio management is a strategic planning function, not a visualization function.

Baseline Comparison

No tracking of baseline vs. actual schedules over time.

Rationale: Baseline tracking requires temporal versioning and comparison logic. This is project-control functionality.

Risk Registers

No formal risk tracking, probability/impact matrices, or mitigation planning.

Rationale: Risk management is a project-management discipline with its own tooling requirements.

0.3.3.2 Data Management

Persistent Storage

Timeline Compiler does not store timelines. It transforms input files to output files. Storage is the user's responsibility.

Collaboration Features

No real-time collaboration, comments, change tracking, or multi-user editing.

Version History

No built-in versioning. Users should use Git or similar tools.

0.3.3.3 Interactive Features

Clickable Outputs

Output formats (SVG, PNG, PDF, PPTX) are static. Hyperlinking or interactive drill-down is out of scope for MVP, though SVG hyperlinks may be a future extension.

WYSIWYG Editing

No drag-and-drop authoring. Timeline Compiler is a compiler, not an editor.

Live Preview During Authoring

Out of scope for core; a VS Code extension may provide this separately.

0.3.3.4 Data Ingestion

Native Integrations

Timeline Compiler does not natively read from Jira, Azure DevOps, Asana, GitHub Projects, or other systems. It consumes only its IR format.

Note: Separate adapter tools or scripts may transform external data to the IR, but these are outside Timeline Compiler's core scope.

0.3.4 Boundary Cases

Some features lie at the boundary and require explicit decisions:

0.3.4.1 Dependency Arrows (Visual Only)

Decision: IN SCOPE — visual only.

The IR may include dependency declarations for the purpose of rendering arrows or lines between activities. However, dependencies are purely visual annotations; they do not affect date computation or validation.


```

1 activities:
2   - id: design
3     label: "Design Phase"
4     range: { start: 2026-01, end: 2026-03 }
5   - id: build
6     label: "Build Phase"
7     range: { start: 2026-04, end: 2026-08 }
8     depends_on: [design] # Visual arrow only

```

Listing 4: Dependencies as visual annotations

0.3.4.2 Today Line

Decision: IN SCOPE — provided as input.

The “today” date may be provided in the IR (e.g., `today: 2026-06-09`) and rendered as a vertical marker. The compiler does not infer today’s date from the system clock; determinism requires all inputs to be explicit.

0.3.4.3 Auto-Layout

Decision: IN SCOPE — deterministic algorithms only.

The compiler automatically lays out activities within tracks (e.g., stacking overlapping items). Layout algorithms must be deterministic. No randomized or heuristic placement that could vary between runs.

0.3.4.4 External Image Embedding

Decision: DEFERRED to future.

Embedding external images (logos, icons) in output is not in MVP scope. Theme-provided icons (e.g., milestone diamonds) are in scope.

0.3.5 Scope Summary

0.4 Timeline IR

The Timeline Intermediate Representation (IR) is the central contract of this system. It defines *what* a timeline communicates—entities, relationships, temporal structure, and semantic status—while remaining agnostic to *how* that communication is rendered. This separation enables deterministic rendering, source-control-friendly editing, and reliable generation by both humans and AI agents.

0.4.1 Design Principles

1. **Make illegal states unrepresentable.** The IR’s type system should prevent malformed timelines at the structural level, not merely validate them after the fact.

Table 2: Scope Boundaries Summary

Category	Scope Status
Timelines, roadmaps, milestone views	In Scope
Tracks, activities, milestones, sections, annotations	In Scope
Progress and status visualization	In Scope
Theming and visual customization	In Scope
SVG, PNG, PDF, PPTX output	In Scope
CLI and library API	In Scope
Dependency arrows (visual only)	In Scope
Today marker (explicit input)	In Scope
Dependency scheduling/resolution	Out of Scope
Resource management	Out of Scope
Critical path analysis	Out of Scope
Sprint/iteration tracking	Out of Scope
Cost management	Out of Scope
Portfolio optimization	Out of Scope
Baseline comparison	Out of Scope
Risk registers	Out of Scope
Data storage and collaboration	Out of Scope
Interactive/clickable output	Out of Scope
WYSIWYG editing	Out of Scope
Native data source integrations	Out of Scope

2. **Rendering-agnostic.** The IR describes meaning, not pixels. Visual decisions—colors, fonts, positioning—belong to the theme engine.
3. **Git-friendly.** Stable field ordering, line-oriented YAML syntax, deterministic serialization, minimal diff churn.
4. **Agent-generatable.** Clear required-vs-optional distinctions, explicit defaults, forgiving structure that remains validatable.
5. **Orthogonal core.** A small set of primitives; convenience features are optional layers, not core complexity.

0.4.2 Type Vocabulary

The IR uses the following type vocabulary consistently throughout this specification:

0.4.3 Document Structure

A Timeline IR document is a single YAML file with the following root structure:

```

1 version: "1.0"
2 metadata: { ... }
3 tracks: [ ... ]
4 groups: [ ... ]      # optional
5 activities: [ ... ]
6 milestones: [ ... ]  # optional
7 annotations: [ ... ] # optional

```

Type	Description
string	UTF-8 text, non-empty unless explicitly noted
string?	Optional string (may be omitted or null)
int	Integer
number	Decimal number
number[0..1]	Decimal in closed interval [0, 1]
bool	Boolean (true/false)
date	A temporal point (see §0.4.5)
daterange	A temporal interval with start and optional end
id	Document-unique identifier (kebab-case slug, e.g., alpha-release)
ref<T>	Reference to an entity of type T by its id
list<T>	Ordered list of elements of type T
optional<T>	Value of type T that may be omitted
enum{...}	One of the listed literal values
map<K,V>	Key-value mapping

Table 3: IR type vocabulary

```

8 sections: [ ... ]      # optional
9 legend: { ... }       # optional

```

Listing 5: Root document structure

0.4.3.1 Root Fields

Field	Type	Req	Default	Description
version	string	Yes	—	Specification version (semver, e.g., "1.0")
metadata	Metadata	Yes	—	Document-level metadata
tracks	list<Track>	Yes	—	Swimlanes/rows; at least one required
groups	list<Group>	No	[]	Logical groupings of activities
activities	list<Activity>	Yes	—	Time-spanning items; may be empty
milestones	list<Milestone>	No	[]	Point-in-time markers
annotations	list<Annotation>	No	[]	Callouts, notes, highlights
sections	list<Section>	No	[]	Visual/logical document sections
legend	Legend	No	auto	Legend mapping; auto-generated if omitted

Table 4: Root document fields

Invariant: The document must contain at least one track. Activities and milestones may both be empty, but a timeline with neither is semantically vacuous (valid but degenerate).

0.4.4 Metadata

The metadata object contains document-level information and temporal framing.

Field	Type	Req	Default	Description
title	string	Yes	—	Primary document title
subtitle	string?	No	null	Secondary title or tagline
author	string?	No	null	Author or team name
created	date	No	now	Document creation date
updated	date	No	now	Last modification date
time_range	TimeRange	Yes	—	Temporal bounds of the timeline
axis_unit	AxisUnit	No	inferred	Granularity of the time axis
theme	string?	No	null	Theme identifier (resolved by renderer)
locale	string?	No	"en"	BCP-47 locale for date formatting
today	date?	No	eval time	Reference date for now, relative dates, and today-marker; see §0.4.4.3
fiscal_year_start	int [1..12]	No	1	Calendar month on which the fiscal year begins (1 = January); see §0.4.4.3
description	string?	No	null	Extended description or notes

Table 5: Metadata fields

0.4.4.1 TimeRange

The `time_range` defines the temporal viewport of the timeline—the span that will be rendered. Activities or milestones outside this range are valid but may be clipped.

```

1 time_range:
2   start: 2026-Q1
3   end: 2026-Q4

```

Field	Type	Req	Default	Description
start	date	Yes	—	Inclusive start of the viewport
end	date	No	open	Inclusive end; omit for open-ended

Table 6: TimeRange fields

0.4.4.2 AxisUnit

```

1 enum AxisUnit { day, week, month, quarter, half, year }

```

If omitted, the renderer infers axis unit from the time range span. The axis unit affects visual tick marks and labels, not the precision of individual dates.

0.4.4.3 Date Anchor and Fiscal Calendar

Date anchor (`today`). The symbolic date `now` and all relative date offsets (`+3m`, `-2w`, etc.) resolve against the *date anchor*, evaluated in this order:

1. `metadata.today` — if present, this value is used exclusively.
2. `metadata.created` — fallback when `today` is absent.
3. **Hard error** — if neither is present and any `now` or relative date appears in the document, the document is not deterministically evaluable and validation must reject it.

Renderers **must not** consult the system clock to resolve `now` or relative dates. The `today-marker` annotation (`type: today-marker`) also uses this anchor.

Determinism caveat: When `today` is omitted and `created` serves as the anchor, output is deterministic (the `created` date is fixed in the document). However, the document’s visual “current position” does not advance with real time. Authors and agents **should** set `today` explicitly whenever `now` or relative dates appear, so that the document’s meaning is unambiguous and byte-stable across environments and rendering runs.

Fiscal calendar (`fiscal_year_start`). `fiscal_year_start` specifies the calendar month (1–12) on which the fiscal year begins. The default is 1 (January), which makes fiscal and calendar years identical.

Fiscal quarter dates (`FYnn-Qk`) map to calendar dates as follows. `FYnn` denotes the fiscal year beginning on month `fiscal_year_start` of calendar year `20nn`. Fiscal quarter k ($k \in \{1, 2, 3, 4\}$) spans three consecutive calendar months starting at calendar month m_k , where:

$$m_k = ((\text{fiscal_year_start} - 1) + (k - 1) \times 3) \bmod 12 + 1$$

and the calendar year is `20nn` if $m_k \geq \text{fiscal_year_start}$, otherwise `20nn + 1`.

Examples with `fiscal_year_start: 1` (default): `FY26-Q1` = Jan–Mar 2026; `FY26-Q2` = Apr–Jun 2026 (identical to calendar quarters).

Examples with `fiscal_year_start: 4` (April fiscal year): `FY26-Q1` = Apr–Jun 2026; `FY26-Q2` = Jul–Sep 2026; `FY26-Q3` = Oct–Dec 2026; `FY26-Q4` = Jan–Mar 2027.

If any `FY...` date appears in the document and `fiscal_year_start` $\neq 1$, authors **should** set `fiscal_year_start` explicitly to ensure all renderers produce consistent x-coordinates for fiscal dates.

0.4.5 Date Model

Timelines present temporal information at varying granularities. A product roadmap may show quarters; a conference schedule shows hours. The IR must represent this heterogeneity without forcing false precision or losing meaningful vagueness.

0.4.5.1 Date Representations

A date in the IR is one of the following forms:

Design rationale: We deliberately support coarse granularity (quarters, halves, years) as first-class concepts rather than forcing conversion to day-level dates. This preserves authorial intent: “Q3 2026” means the entire quarter, not “2026-07-01”.

Form	Example	Semantics
ISO date	2026-06-09	Specific calendar day
ISO datetime	2026-06-09T14:00	Specific moment (minute precision)
Year-month	2026-06	Entire month (June 2026)
Year only	2026	Entire year
Quarter	2026-Q2	Calendar quarter (Apr–Jun)
Half	2026-H1	Calendar half (Jan–Jun)
Fiscal quarter	FY26-Q2	Fiscal quarter (calendar mapping per <code>fiscal_year_start</code> ; see §0.4.4.3)
Relative	+3m	3 months from the date anchor (<code>metadata.today</code> → <code>metadata.created</code> ; see §0.4.4.3)
Symbolic	now	Resolves to the date anchor (<code>metadata.today</code> → <code>metadata.created</code> ; see §0.4.4.3)

Table 7: Date representation forms

0.4.5.2 Uncertain and Open Dates

For dates that are genuinely unknown, tentative, or open-ended, the IR provides explicit encodings rather than using null-as-magic:

Value	Semantics
tbd	Date is not yet determined; renders as “TBD”
ongoing	No planned end; activity continues indefinitely
unknown	Date exists but is not known to the author
~2026-Q3	Approximate/tentative (prefix tilde)

Table 8: Uncertain and open date markers

Invariant: `tbd`, `ongoing`, and `unknown` are valid only in positions where the schema allows uncertainty (e.g., end of a daterange, not `start` of the document’s `time_range`).

0.4.5.3 DateRange

A daterange represents a temporal interval:

```

1 # Explicit start and end
2 start: 2026-04-01
3 end: 2026-06-30
4
5 # Open-ended (no planned end)
6 start: 2026-04-01
7 end: ongoing
8
9 # Shorthand for single-granularity span
10 span: 2026-Q2 # equivalent to start: 2026-04-01, end: 2026-06-30

```

Invariant: When both `start` and `end` are concrete dates, `end` ≥ `start`. The span shorthand expands to the natural bounds of its granularity (e.g., `span: 2026-Q2` expands to Q2’s first and last days).

Field	Type	Req	Default	Description
start	date	Yes*	—	Interval start (inclusive)
end	date ongoing tbd	No	ongoing	Interval end (inclusive); omitting is equivalent to end: ongoing
span	date	Yes*	—	Single-unit shorthand; mutually exclusive with start/end

Table 9: DateRange fields (*exactly one of `span` or `start` (+ optional `end`) is required; co-presence of `span` with `start` or `end` is a well-formedness error)

Invariant (exclusivity): `span` is *mutually exclusive* with `start` and `end`. A daterange must use exactly one of:

- `span` alone (shorthand form), or
- `start` alone or `start` + `end` (explicit form).

Co-presence of `span` with `start` or `end` on the same entity is a well-formedness error (SPAN_START_CONFLICT).

Open/ongoing semantics: An Activity or daterange with `start` specified but `end` omitted is an *open/ongoing interval*. This is semantically identical to writing `end: ongoing` and is an explicit, valid state (not an authoring error). Renderers must extend the bar to the canvas right edge and apply an open-end indicator (e.g., right-pointing chevron).

0.4.6 Tracks (Swimlanes)

Tracks are horizontal lanes that organize activities visually. Every activity belongs to exactly one track. Tracks appear in document order (first track at top).

```

1 tracks:
2   - id: platform
3     label: Platform Team
4     description: Core infrastructure work
5
6   - id: mobile
7     label: Mobile Apps
8     color: blue           # optional hint

```

Listing 6: Track examples

Invariant: Track `id` values must be unique within the document.

0.4.7 Groups

Groups provide logical clustering of activities, tracks, or other groups. They enable hierarchical organization (e.g., “Engineering” containing “Platform” and “Mobile” tracks) without conflating logical grouping with visual swimlanes.

Field	Type	Req	Default	Description
id	id	Yes	—	Unique track identifier
label	string	Yes	—	Display label
description	string?	No	null	Extended description
color	string?	No	theme	Color hint (theme may override)
group	ref<Group>?	No	null	Parent group (for nested tracks)
index	int?	No	position	Explicit sort index; unique non-negative integer; tracks render top-to-bottom in ascending index order
collapsed	bool	No	false	Initial collapsed state (if renderer supports)

Table 10: Track fields

```

1 groups:
2   - id: engineering
3     label: Engineering
4     children:
5       - platform      # ref to track
6       - mobile        # ref to track
7
8   - id: releases
9     label: Release Milestones
10    members:
11      - alpha-release  # ref to milestone
12      - beta-release

```

Listing 7: Group example

Field	Type	Req	Default	Description
id	id	Yes	—	Unique group identifier
label	string	Yes	—	Display label
description	string?	No	null	Extended description
parent	ref<Group>?	No	null	Parent group (for nesting)
children	list<ref<Track Group>>	No	[]	Ordered child tracks/groups
members	list<ref<Activity Milestone>>	No	[]	Member entities
color	string?	No	theme	Color hint

Table 11: Group fields

Invariant: Group hierarchies must be acyclic. A group cannot be its own ancestor.

0.4.8 Activities

Activities are time-spanning items—phases, projects, tasks, or efforts. They are the primary content of most timelines.

```

1 activities:
2   - id: api-redesign
3     label: API Redesign

```



```

4   track: platform
5   start: 2026-Q1
6   end: 2026-Q2
7   status: in-progress
8   progress: 0.4
9
10  - id: mobile-launch
11    label: Mobile App Launch
12    track: mobile
13    span: 2026-Q3
14    status: planned
15    category: launch

```

Listing 8: Activity examples

Field	Type	Req	Default	Description
id	id	Yes	—	Unique activity identifier
label	string	Yes	—	Display label (shown on bar)
track	ref<Track>	Yes	—	Containing track
start	date	Yes*	—	Start date
end	date ongoing tbd	No	ongoing	End date; omitting is equivalent to end: ongoing (open interval)
span	date	Yes*	—	Single-unit shorthand; mutually exclusive with start/end
status	Status	No	planned	Visual communication status
progress	number[0..1]?	No	null	Completion fraction
category	string?	No	null	Semantic category (for styling)
description	string?	No	null	Extended description
group	ref<Group>?	No	null	Logical group membership
url	string?	No	null	External link
tags	list<string>	No	[]	Freeform tags
metadata	map<string,any>	No	{}	Extension metadata

Table 12: Activity fields (*exactly one of `span` or `start` (+ optional `end`) is required; co-presence of `span` with `start` or `end` is a well-formedness error)

0.4.8.1 Status Enum

The `status` field communicates *visual* state for presentation purposes. This is explicitly **not** a project-management workflow state; it describes what the audience should understand about the item’s current condition.

```

1  enum Status {
2    planned,      # Scheduled but not yet started
3    in-progress,  # Currently active
4    done,         # Completed
5    at-risk,      # May miss target (visual warning)
6    blocked,      # Cannot proceed (visual alert)
7    cancelled,    # No longer planned
8    tentative     # Not yet confirmed
9  }

```

Design rationale: We include `at-risk` and `blocked` because executive timelines frequently need to communicate risk visually. We omit workflow states like “in review” or “approved” which belong to project-management tools, not visual communication artifacts.

0.4.8.2 Progress

The `progress` field is a number in $[0, 1]$ representing completion fraction. It has no inherent visual meaning—the theme decides whether to render it as a progress bar, percentage label, or ignore it entirely.

Invariant: If `progress` is provided, it must be in $[0, 1]$. `progress: 1.0` does not imply `status: done`; these are orthogonal.

0.4.9 Milestones

Milestones are point-in-time markers—releases, deadlines, decision points, or events. Unlike activities, they have no duration.

```

1 milestones:
2   - id: alpha-release
3     label: Alpha Release
4     date: 2026-03-15
5     track: platform
6     status: done
7     icon: flag
8
9   - id: board-review
10    label: Board Review
11    date: 2026-Q2      # first day of Q2
12    category: governance

```

Listing 9: Milestone examples

Field	Type	Req	Default	Description
<code>id</code>	<code>id</code>	Yes	—	Unique milestone identifier
<code>label</code>	<code>string</code>	Yes	—	Display label
<code>date</code>	<code>date</code>	Yes	—	Point in time
<code>track</code>	<code>ref<Track>?</code>	No	<code>global</code>	Associated track (or global)
<code>status</code>	<code>Status</code>	No	<code>planned</code>	Visual status
<code>category</code>	<code>string?</code>	No	<code>null</code>	Semantic category
<code>icon</code>	<code>string?</code>	No	<code>theme</code>	Icon hint (diamond, flag, star, etc.)
<code>description</code>	<code>string?</code>	No	<code>null</code>	Extended description
<code>group</code>	<code>ref<Group>?</code>	No	<code>null</code>	Logical group membership
<code>url</code>	<code>string?</code>	No	<code>null</code>	External link
<code>tags</code>	<code>list<string></code>	No	<code>[]</code>	Freeform tags

Table 13: Milestone fields

Note: When `track` is omitted, the milestone is “global” and may be rendered spanning all tracks (e.g., a vertical line with label at top).

0.4.10 Annotations

Annotations add contextual information—callouts, notes, highlights, or today-markers—without being first-class timeline entities.

```

1 annotations:
2   - type: today-marker
3     date: now
4
5   - type: callout
6     target: api-redesign
7     text: "Critical path item"
8     position: above
9
10  - type: bracket
11    targets: [alpha-release, beta-release]
12    label: "Testing Phase"
13
14  - type: period
15    start: 2026-06-01
16    end: 2026-06-15
17    label: "Code Freeze"
18    style: highlight

```

Listing 10: Annotation examples

Field	Type	Req	Default	Description
type	AnnotationType	Yes	—	Annotation kind
id	id?	No	auto	Optional identifier
target	ref<Activity Milestone>?	Cond	—	Single target entity
targets	list<ref<...>?>	Cond	—	Multiple targets
date	date?	Cond	—	Point annotation
start/end	date?	Cond	—	Range annotation
text/label	string?	No	null	Display text
position	enum{above,below,left,right}?	No	auto	Hint only
style	string?	No	theme	Style hint

Table 14: Annotation fields (conditional fields depend on type)

0.4.10.1 Annotation Types

```

1 enum AnnotationType {
2   today-marker, # Vertical line at current date
3   callout,      # Note attached to an entity
4   bracket,      # Visual bracket spanning entities
5   period,       # Highlighted time period
6   note,         # Freestanding note
7   connector     # Line connecting two entities
8 }

```

0.4.11 Sections

Sections divide the timeline into logical or visual chapters. They enable multi-page timelines or distinct phases that should be visually separated.

```

1 sections:
2   - id: current-state
3     label: "Current State (Q1-Q2)"
4     time_range:
5       start: 2026-Q1
6       end: 2026-Q2
7
8   - id: future-state
9     label: "Future Roadmap (Q3-Q4)"
10    time_range:
11      start: 2026-Q3
12      end: 2026-Q4

```

Listing 11: Section example

Field	Type	Req	Default	Description
id	id	Yes	—	Unique section identifier
label	string	Yes	—	Section title
time_range	TimeRange?	No	inherit	Override document time range
tracks	list<ref<Track>?	No	all	Subset of tracks to show
description	string?	No	null	Section description
page_break	bool	No	false	Hint for paginated output

Table 15: Section fields

0.4.12 Legend

The legend documents the meaning of statuses, categories, and visual conventions used in the timeline. If omitted, the renderer may auto-generate a legend from used values.

```

1 legend:
2   show: true
3   position: bottom-right
4   entries:
5     - key: in-progress
6       label: In Progress
7       description: Currently being worked on
8     - key: at-risk
9       label: At Risk
10      description: May miss planned date
11     - key: launch
12       label: Launch Event
13       category: true

```

Listing 12: Legend example

0.4.13 ID and Reference System

0.4.13.1 Identifier Format

All id fields must be:

Field	Type	Req	Default	Description
show	bool	No	true	Whether to render legend
position	string?	No	theme	Position hint
title	string?	No	"Legend"	Legend title
entries	list<LegendEntry>	No	auto	Legend entries

Table 16: Legend fields

Field	Type	Req	Default	Description
key	string	Yes	—	Status or category key
label	string	Yes	—	Human-readable label
description	string?	No	null	Extended description
category	bool	No	false	True if this is a category (not status)

Table 17: LegendEntry fields

- Non-empty strings
- Composed of lowercase letters, digits, and hyphens
- Starting with a letter
- Unique within their entity type AND globally within the document

Regex: `^[a-z][a-z0-9-]*$`

Rationale for global uniqueness: References may appear in contexts where the entity type is ambiguous (e.g., annotation targets). Global uniqueness eliminates the need for type prefixes in references.

```

1 # Valid IDs
2 id: api-v2
3 id: q3-release
4 id: mobile-team-ios
5
6 # Invalid IDs
7 id: API-v2           # uppercase
8 id: 2026-release    # starts with digit
9 id: api_v2          # underscore

```

Listing 13: ID examples

0.4.13.2 References

A reference is a string containing the id of another entity. The IR uses bare strings for references (not structured objects) for YAML terseness:

```

1 activities:
2   - id: feature-a
3     track: platform      # ref<Track>
4     group: engineering   # ref<Group>
5
6 annotations:
7   - type: callout
8     target: feature-a    # ref<Activity>

```

Invariant: Every reference must resolve to exactly one entity in the document. A validator must reject documents with dangling references.

0.4.13.3 Reference Namespacing

Because IDs are globally unique, references are unambiguous without type prefixes. The schema context determines what types are valid:

- `activity.track`: must reference a `Track`
- `annotation.target`: may reference `Activity` or `Milestone`
- `group.children`: may reference `Track` or `Group`

A validator checks that the referenced entity has the correct type for its context.

0.4.14 Well-Formedness Rules

A Timeline IR document is **well-formed** if and only if all the following invariants hold:

0.4.14.1 Structural Invariants

1. **Version present:** `version` field exists and matches a known specification version.
2. **Required fields:** All fields marked “Req: Yes” are present and non-null.
3. **At least one track:** The `tracks` list is non-empty.
4. **Unique IDs:** All id values are globally unique within the document.
5. **Valid ID format:** All id values match `^[a-z][a-z0-9-]*$`.

0.4.14.2 Referential Invariants

6. **References resolve:** Every reference points to an existing entity.
7. **Reference types match:** References point to entities of the expected type (e.g., `activity.track` references a `Track`, not a `Group`).
8. **No circular groups:** The group parent-child graph is acyclic.

0.4.14.3 Temporal Invariants

9. **Valid time range:** If `metadata.time_range` has both `start` and `end`, then `end >= start`.
10. **Activity duration non-negative:** For activities with concrete `start` and `end`, `end >= start`.
11. **Date format valid:** All date values conform to the date model (§0.4.5).
12. **Activity date-source exclusive:** Every activity must satisfy exactly one of:
 - (a) `span` is present and neither `start` nor `end` is present; or
 - (b) `start` is present and `span` is absent (`end` optional). Co-presence of `span` with `start` or `end` is a hard well-formedness error: `SPAN_START_CONFLICT`.

13. **Date anchor present when required:** If any date field in the document contains now or a relative offset (e.g., +3m, -2w), then at least one of `metadata.today` or `metadata.created` must be present. Absence of both is a hard error: `DATE_ANCHOR_MISSING`.
14. **Track index values unique:** When `track.index` is present, all supplied values must be unique non-negative integers within the document.

0.4.14.4 Value Invariants

15. **Progress in range:** If `progress` is present, $0 \leq \text{progress} \leq 1$.
16. **Status valid:** All `status` values are members of the `Status` enum.
17. **Axis unit valid:** If present, `axis_unit` is a member of `AxisUnit` enum.

0.4.14.5 Soft Warnings (Valid but Suspicious)

The following are not errors but may indicate authorial mistakes:

- Activity or milestone outside `metadata.time_range` (will be clipped)
- `progress: 1.0` with `status: in-progress` (likely stale)
- Empty `activities` and `milestones` lists (vacuous timeline)
- Unused tracks (no activities reference them)

0.4.15 Complete Examples

The following examples demonstrate realistic Timeline IR documents. Each is a complete, valid document showcasing different timeline use cases.

0.4.15.1 Example 1: Product Roadmap

A quarterly product roadmap with multiple teams, showing how the IR represents a typical software planning artifact.

```
1 version: "1.0"
2
3 metadata:
4   title: "Product Roadmap 2026"
5   subtitle: "Engineering & Design"
6   author: "Product Team"
7   created: 2026-06-09
8   time_range:
9     start: 2026-Q1
10    end: 2026-Q4
11   axis_unit: quarter
12   theme: corporate-blue
13
14 tracks:
15   - id: platform
16     label: Platform
17     description: Core infrastructure and APIs
18
```

```
19 - id: mobile
20   label: Mobile Apps
21   description: iOS and Android applications
22
23 - id: design
24   label: Design System
25   description: Component library and guidelines
26
27 groups:
28   - id: foundation
29     label: Foundation Work
30     members: [api-v2, component-lib]
31
32 activities:
33   - id: api-v2
34     label: API v2 Migration
35     track: platform
36     start: 2026-Q1
37     end: 2026-Q2
38     status: in-progress
39     progress: 0.6
40     category: infrastructure
41
42   - id: mobile-redesign
43     label: Mobile App Redesign
44     track: mobile
45     start: 2026-Q2
46     end: 2026-Q3
47     status: planned
48     description: Complete visual refresh
49
50   - id: component-lib
51     label: Component Library
52     track: design
53     start: 2026-Q1
54     end: 2026-Q4
55     status: in-progress
56     progress: 0.3
57
58   - id: analytics
59     label: Analytics Integration
60     track: platform
61     span: 2026-Q3
62     status: planned
63
64 milestones:
65   - id: api-ga
66     label: API v2 GA
67     date: 2026-06-30
68     track: platform
69     status: planned
70     icon: flag
71
72   - id: app-launch
73     label: App Store Launch
74     date: 2026-Q4
75     track: mobile
76     status: planned
```



```

77     category: launch
78
79 annotations:
80   - type: today-marker
81     date: now
82
83 legend:
84   show: true
85   entries:
86     - key: infrastructure
87       label: Infrastructure
88       category: true
89     - key: launch
90       label: Launch Event
91       category: true

```

Listing 14: Product roadmap example

0.4.15.2 Example 2: Executive Program Timeline

A high-level transformation program timeline suitable for board presentations, demonstrating status communication and risk visibility.

```

1 version: "1.0"
2
3 metadata:
4   title: "Digital Transformation Program"
5   subtitle: "FY26 Executive View"
6   author: "PMO"
7   created: 2026-06-09
8   time_range:
9     start: 2026-01-01
10    end: 2026-12-31
11   axis_unit: month
12   theme: executive
13
14 tracks:
15   - id: strategy
16     label: Strategy & Governance
17
18   - id: technology
19     label: Technology Modernization
20
21   - id: people
22     label: People & Change
23
24 activities:
25   - id: strategy-define
26     label: Strategy Definition
27     track: strategy
28     start: 2026-01-01
29     end: 2026-03-31
30     status: done
31     progress: 1.0
32
33   - id: vendor-selection
34     label: Vendor Selection

```

```
35     track: technology
36     start: 2026-02-01
37     end: 2026-04-30
38     status: done
39
40   - id: platform-build
41     label: Platform Build
42     track: technology
43     start: 2026-04-01
44     end: 2026-09-30
45     status: in-progress
46     progress: 0.35
47
48   - id: data-migration
49     label: Data Migration
50     track: technology
51     start: 2026-07-01
52     end: 2026-11-30
53     status: at-risk
54     description: Dependency on legacy system documentation
55
56   - id: training
57     label: Training Program
58     track: people
59     start: 2026-06-01
60     end: 2026-12-31
61     status: planned
62
63   - id: change-mgmt
64     label: Change Management
65     track: people
66     start: 2026-03-01
67     end: ongoing
68     status: in-progress
69
70 milestones:
71   - id: board-approval
72     label: Board Approval
73     date: 2026-03-15
74     status: done
75     icon: star
76
77   - id: go-live
78     label: Go Live
79     date: 2026-10-01
80     track: technology
81     status: planned
82     icon: flag
83
84   - id: program-close
85     label: Program Close
86     date: 2026-12-15
87     status: planned
88
89 annotations:
90   - type: today-marker
91     date: now
92
```

```

93   - type: callout
94     target: data-migration
95     text: "Risk: Legacy documentation gaps"
96     position: above
97
98   - type: bracket
99     targets: [platform-build, data-migration]
100    label: "Critical Path"
101
102  sections:
103    - id: h1
104      label: "H1 2026: Foundation"
105      time_range:
106        start: 2026-01-01
107        end: 2026-06-30
108
109    - id: h2
110      label: "H2 2026: Execution"
111      time_range:
112        start: 2026-07-01
113        end: 2026-12-31

```

Listing 15: Executive program timeline

0.4.15.3 Example 3: Architecture Evolution Timeline

A technical architecture timeline showing system evolution over multiple years, demonstrating coarse granularity and technical categorization.

```

1  version: "1.0"
2
3  metadata:
4    title: "Platform Architecture Evolution"
5    subtitle: "2026-2028 Technical Roadmap"
6    author: "Architecture Team"
7    created: 2026-06-09
8    time_range:
9      start: 2026
10     end: 2028
11    axis_unit: half
12    theme: technical
13
14  tracks:
15    - id: frontend
16      label: Frontend
17
18    - id: backend
19      label: Backend Services
20
21    - id: data
22      label: Data Platform
23
24    - id: infra
25      label: Infrastructure
26
27  activities:
28    - id: react-migration

```

```
29   label: React 19 Migration
30   track: frontend
31   start: 2026-H1
32   end: 2026-H2
33   status: in-progress
34   category: modernization
35
36 - id: micro-frontend
37   label: Micro-Frontend Architecture
38   track: frontend
39   start: 2027-H1
40   end: 2027-H2
41   status: planned
42   category: architecture
43
44 - id: api-gateway
45   label: API Gateway
46   track: backend
47   span: 2026-H1
48   status: done
49   category: infrastructure
50
51 - id: service-mesh
52   label: Service Mesh Adoption
53   track: backend
54   start: 2026-H2
55   end: 2027-H1
56   status: planned
57   category: infrastructure
58
59 - id: event-driven
60   label: Event-Driven Architecture
61   track: backend
62   start: 2027
63   end: 2028
64   status: tentative
65   category: architecture
66
67 - id: lakehouse
68   label: Data Lakehouse
69   track: data
70   start: 2026-H1
71   end: 2027-H1
72   status: in-progress
73   progress: 0.4
74   category: data
75
76 - id: ml-platform
77   label: ML Platform
78   track: data
79   start: 2027
80   end: ongoing
81   status: tentative
82   category: ai
83
84 - id: k8s-migration
85   label: Kubernetes Migration
86   track: infra
```

```
87     span: 2026
88     status: in-progress
89     progress: 0.7
90     category: infrastructure
91
92     - id: multi-cloud
93       label: Multi-Cloud Strategy
94       track: infra
95       start: 2027
96       end: 2028
97       status: planned
98       category: infrastructure
99
100 milestones:
101   - id: legacy-sunset
102     label: Legacy System Sunset
103     date: 2027-H1
104     track: backend
105     status: planned
106     icon: flag
107
108   - id: cloud-native
109     label: Cloud Native Milestone
110     date: 2026
111     track: infra
112     status: planned
113
114 annotations:
115   - type: period
116     start: 2026-H2
117     end: 2027-H1
118     label: "Transition Period"
119     style: highlight
120
121 legend:
122   entries:
123     - key: modernization
124       label: Modernization
125       category: true
126     - key: architecture
127       label: Architecture Change
128       category: true
129     - key: infrastructure
130       label: Infrastructure
131       category: true
132     - key: data
133       label: Data Platform
134       category: true
135     - key: ai
136       label: AI/ML
137       category: true
```

Listing 16: Architecture evolution timeline

0.4.16 Extensibility

The IR is designed to be extended without breaking existing consumers.

0.4.16.1 Metadata Extension

Every entity type includes a metadata field (type `map<string,any>`) for application-specific data:

```
1 activities:
2   - id: feature-x
3     label: Feature X
4     track: platform
5     span: 2026-Q2
6     metadata:
7       jira_key: PROJ-1234
8       owner: alice@example.com
9       priority: high
```

Renderers should ignore unrecognized metadata keys. Source integrations may use metadata to preserve round-trip fidelity with external systems.

0.4.16.2 Custom Categories

The `category` field accepts arbitrary strings. The theme engine maps categories to visual styles. New categories can be introduced without IR changes:

```
1 activities:
2   - id: security-audit
3     category: security    # custom category
4     # ...
5
6 legend:
7   entries:
8     - key: security
9       label: Security Work
10      category: true
```

0.4.16.3 Version Compatibility

The `version` field enables forward compatibility:

- Consumers should accept documents with higher minor versions (1.1, 1.2, etc.) and ignore unknown fields.
- Major version changes (2.0) may introduce breaking changes; consumers should reject documents with unsupported major versions.

0.4.17 Serialization and Syntax

0.4.17.1 Canonical Format

YAML 1.2 is the canonical surface syntax for Timeline IR documents. Design decisions prioritizing YAML:

- **Minimal punctuation:** No braces for simple mappings, no quotes for most strings.
- **Line-oriented:** Each field on its own line for clean diffs.
- **Stable ordering:** Fields should appear in specification order for consistency; validators may enforce this.
- **Comments preserved:** YAML allows comments; tools should preserve them.

0.4.17.2 Alternative Formats

Tooling may support additional formats with lossless conversion:

- **JSON:** Direct mapping; verbose but universally parseable.
- **TOML:** Alternative for users who prefer it.

The IR is defined abstractly; the YAML syntax is one serialization, not the definition.

0.4.17.3 Git Friendliness

For minimal diff churn:

1. Lists use block style (one item per line), not flow style.
2. Fields appear in consistent order.
3. Auto-generated fields (like `updated` timestamps) are optional and may be omitted in version-controlled documents.
4. IDs are stable slugs, not generated UUIDs.

```
1 # Good: block style, stable order
2 activities:
3   - id: feature-a
4     label: Feature A
5     track: platform
6     span: 2026-Q2
7
8 # Avoid: flow style, hard to diff
9 activities: [{id: feature-a, label: Feature A, track: platform,
10              span: 2026-Q2}]
```

Listing 17: Git-friendly style

0.4.18 Validation

A conforming validator must check all invariants in §0.4.14. Validation is a function:

$$\text{validate} : \text{Document} \rightarrow \text{Valid} \mid \text{Errors}$$

0.4.18.1 Error Reporting

Validation errors should include:

- Path to the offending field (e.g., `activities[2].track`)
- The invariant violated
- The actual value
- Suggested fix (when determinable)

0.4.18.2 Strict vs. Lenient Mode

- **Strict mode:** All invariants enforced; required for rendering.
- **Lenient mode:** Warnings instead of errors for soft issues; useful for work-in-progress documents.

0.4.19 Relationship to Other Components

The IR is the stable contract between pipeline stages:

Ingestion (upstream)

Source adapters (ADO, GitHub, Markdown, etc.) produce IR documents. The IR defines the target shape; adapters handle mapping.

Theme Engine (downstream)

Themes consume IR and produce styled visual specifications. Themes interpret `status`, `category`, and hint fields (`color`, `icon`) but cannot require additional IR fields.

Renderer (downstream)

The renderer consumes themed IR (or IR + theme) and produces output (SVG, PNG, PDF, PPTX). The IR remains rendering-agnostic; pixel-level decisions are not IR concerns.

Agent Integration

AI agents generate IR directly. The IR's explicit typing, clear required/optional fields, and forgiving structure enable reliable generation. Agents need not understand rendering.

Contract stability: Downstream components may not require IR fields beyond this specification. Extension fields (`metadata`) are pass-through.

0.4.20 Build vs. Adopt: A Survey of Existing Representations

Before committing to a fully greenfield IR, this section surveys existing formats, grammars, and standards as potential adoption or extension candidates. Every candidate is assessed against the five criteria derived from the design principles (§0.4.3):

1. **Visual-communication focus:** roadmap/milestone/swimlane semantics, *not* task-scheduling (no dependencies, resource levelling, or critical path);
2. **Git-friendly:** line-oriented text, stable field order, minimal diff churn;
3. **LLM-generatable:** clear required/optional schema, simple references, forgiving structure;
4. **Render/theme separation:** clean boundary between *what* is communicated and *how* it looks; and
5. **Open licence with community or specification body.**

0.4.20.1 Timeline Text Languages

Markwhen. Markwhen [16] (MIT licence) is the closest existing tool to our surface-syntax goals. Events are written as `date: label` or `start/end: label` lines; `group:` blocks provide swimlane-like structure; `#tag` tokens allow lightweight categorisation. The format is text-native, line-oriented, and git-diffable.

Critical gaps exist, however. There is no first-class status concept (no equivalent of `status: at-risk`); granularity is limited to ISO dates and informal text strings (no `2026-Q3` or fiscal-quarter shorthand); render/theme separation is absent—the view container interprets colour from tags, not a separate theme layer; static SVG, PDF, and PPTX output are unsupported; and Markwhen does not publish a versioned, machine-verifiable schema for its internal JSON parse tree, which makes reliable LLM generation fragile. The editor (markwhen.com) is proprietary; only the parser and view container are open source.

Mermaid timeline and Gantt. The Mermaid timeline type [18] groups events into sections on a free-text time axis. It has no swimlane concept, no status enum, no PPTX output, and no render/theme separation (styles are hard-coded in the renderer). The Mermaid gantt type [19] adds dependency keywords (`crit`, `done`, `active`)—precisely the scheduling semantics this specification rejects. Neither type exposes a stable IR that downstream tooling can consume.

PlantUML Gantt. PlantUML’s `@startgantt` [30] is experimental and scheduling-oriented (dependency declarations, work-remaining percentages). Its verbose, UML-rooted syntax is not LLM-friendly for presentation timelines.

D2 and Structurizr. D2 [39] is a general-purpose diagram DSL with no native timeline type. Structurizr [36] is domain-specific to software architecture (C4 model). Neither is a relevant adoption candidate.

0.4.20.2 Visualization Grammars

Vega-Lite. Vega-Lite [33] is the most significant analogy in this category. It demonstrates that a high-level declarative JSON grammar can cleanly separate *what to visualize* (data, encodings, marks) from *how to render it* (scales, axes, guides, legends), with actual rendering delegated to the Vega runtime. This is precisely the WHAT/HOW philosophy that this IR embodies (see §0.4.19).

Vega-Lite is, however, a statistical-graphics grammar, not a roadmap grammar. Encoding a swimlane timeline in Vega-Lite requires manual mark composition: a `rect` mark with `y: track`, `x: start`, `x2: end`, plus separate layers for milestones and annotations. There is no first-class concept of `tracks`, `milestones`, `status`, `span`, or `annotations`. The resulting spec is verbose (100+ JSON lines for a five-activity roadmap), fragile for LLM generation, and not human-editable in the YAML sense intended here. Vega-Lite is the strongest *architectural analogy*—its declarative WHAT/HOW separation is the direct design precedent for this IR—but cannot be adopted wholesale.

The Grammar of Graphics and ggplot2. Wilkinson’s grammar [45] and Wickham’s layered refinement [43] establish the theoretical foundation for declarative visual grammars: decompose a graphic into data, aesthetics, geometry, statistics, and scales. The key insight—separating semantic content from visual treatment—directly motivates this IR’s separation of tracks/activities/status from theme/colours/layout. These are *conceptual donors*, not adoptable formats.

Observable Plot and Apache ECharts. Observable Plot (ISC licence [26]) is a concise JavaScript charting API. It has no native swimlane roadmap type and requires imperative programming, not declarative text authoring. Apache ECharts has a `timeline` component, but it is an animated-playback control that cycles through multiple chart states over time—not a swimlane roadmap grammar. Neither is suitable for adoption.

0.4.20.3 Timeline Data Models

Knight Lab TimelineJS. TimelineJS [15] accepts a flat JSON array of events with `start_date`, `end_date`, `text.headline`, and `text.text`. It is storytelling-oriented (narrative slides, embedded media) with no swimlanes, no status enum, no PPTX output, and no render/theme separation.

vis-timeline. vis-timeline [41] is a browser-only interactive widget requiring JavaScript item objects (`{id, content, start, end, group}`). There is no declarative text format, no static SVG/PDF/PPTX export, and no theme-layer concept. Not adoptable.

react-chrono and Timeline Storyteller. react-chrono [32] (MIT) accepts a JavaScript items array with `title`, `cardTitle`, `cardDetailedText`; date fields are plain strings with no semantic granularity and no swimlanes. Microsoft Research’s Timeline Storyteller [25] (open-sourced 2019) accepts a simple `[{start, end, label}]` JSON array—no swimlanes, no status, no maintained community. Neither is adoptable.

0.4.20.4 Temporal and Calendar Standards

iCalendar (RFC 5545). RFC 5545 [5] defines `VEVENT` with properties `DTSTART`, `DTEND`, `SUMMARY`, `DESCRIPTION`, `CATEGORIES`, `STATUS` (`TENTATIVE`, `CONFIRMED`, `CANCELLED`), and `URL`. This vocabulary is the outstanding *donor*: Mark’s `start/end/label/description/category/status/url` all map directly to iCalendar terms, establishing prior-art legitimacy for these names (see Table 19).

The ICS wire format is wholly unsuitable: property-folded lines (`DTSTART:20260609T140000Z`), no YAML/JSON structure, and no swimlane, visual-status, or render-pipeline concept. The `STATUS` enum covers scheduling state (`CONFIRMED`) but not visual-communication urgency (`at-risk`, `blocked`). **Not adoptable; strongest single vocabulary donor.**

Schema.org/Event. The schema.org Event type [35] defines `name`, `startDate`, `endDate`, `description`, `url`, and `eventStatus`. These are the canonical web vocabulary for events and corroborate Mark’s field naming. Schema.org is a semantic-markup standard (JSON-LD/Microdata), not a visual representation format. **Not adoptable; useful vocabulary donor.**

W3C OWL-Time. OWL-Time [42] (W3C Recommendation, 2022) formalises temporal entities as `Instant` and `Interval` with object properties `hasBeginning`, `hasEnd`, and `hasTemporalDuration`. Crucially, omitting `hasEnd` explicitly represents an open/ongoing interval—the precise semantics of the IR’s `end`: *ongoing*. OWL-Time implements Allen’s thirteen interval relations [2] and is the authoritative formal reference for the IR’s open-ended activity semantics.

Allen’s Interval Algebra. Allen’s 1983 paper [2] establishes 13 binary relations between time intervals: *before*, *meets*, *overlaps*, *starts*, *during*, *finishes*, *equals*, and their converses. The algebra provides formal vocabulary for the uncertain/open date model: an activity with `end`: *ongoing* participates in the *started-by* relation with the document’s `time_range` without requiring a concrete end date. Not a format; the formal grounding for temporal semantics.

ISO 8601. ISO 8601 [12] is already the foundation of the date model (§0.4.5). The IR’s `2026-06-09`, `2026-06`, and `2026` forms are ISO 8601; the `2026-Q2` and `2026-H1` extensions follow the same `YYYY-unit` pattern as a natural extension. No further alignment is needed.

0.4.20.5 Scheduling and PM Formats

TaskJuggler [38] (GPL v2), MS Project XML [21], and Primavera P6 are scheduling grammars built around *computed* dates: dependency resolution, resource levelling, and critical-path analysis. All are explicitly out of scope per the Timeline Grammar decision. MS Project XML additionally fails the git-friendly criterion: the format is non-deterministic between saves (GUIDs and timestamps change on every write). No vocabulary from these formats merits borrowing beyond what iCalendar and schema.org already provide.

0.4.20.6 Analogous Document IRs

The Pandoc AST [13] is the strongest structural analogy: a typed, versioned, language-agnostic JSON document IR with a `pandoc-api-version` key, a `meta` block, and a sequence of typed block elements. This pattern—`version` + `metadata` + typed entity lists—maps directly onto the root structure of §0.4.3. The Dragon Book [1] provides theoretical backing for a typed, pipeline-stable IR as the contract between compilation stages. Both are *structural analogies*, not adoptable formats.

0.4.20.7 Comparison Table

Table 18 scores the seven strongest candidates against the five criteria. **Y** = good fit; **P** = partial; **N** = poor fit or not applicable.

Candidate	Vis.	Git	LLM	Sep.	Licence	Tooling
Markwhen [16]	Y	Y	P	N	MIT	P
Mermaid time-line [18]	P	Y	P	N	MIT	Y
iCalendar	N	N	N	N	IETF	Y
VEVENT [5]						
TimelineJS	Y	P	Y	N	GPL	P
JSON [15]						
vis-timeline [41]	Y	N	N	N	MIT	Y
Vega-Lite [33]	P	P	P	Y	BSD-3	Y
TaskJuggler [38]	N	Y	N	N	GPLv2	P

Table 18: Build-vs-Adopt scoring. Vis. = visual-communication focus (not scheduling); Git = git-friendly text format; LLM = LLM-generatable schema; Sep. = render/theme separation; Tooling = active community. Y = good; P = partial; N = poor.

No candidate scores **Y** on all five dimensions. Markwhen—the closest—fails on render/theme separation, lacks a stable machine-verifiable schema, and has no PPTX output path.

0.4.20.8 Recommendation: Build with Borrowed Vocabulary

Verdict: Build a bespoke IR. No existing format is adoptable wholesale and no viable extension profile exists.

Two orthogonal gaps eliminate every candidate:

1. **Semantic gap.** All formats with real communities (Mermaid, iCalendar, Vega-Lite) are scheduling-oriented, statistical-graphics-oriented, or calendar-oriented. None is a visual-communication roadmap grammar. Defining a “profile” of iCalendar or a “mark recipe” in Vega-Lite would require adding swimlanes, visual status, coarse-grain dates, PPTX output, and a theme engine as first-class concepts—at which point we have built a new layer regardless.
2. **Pipeline gap.** The requirement of a static, multi-backend render pipeline (SVG / PDF / PPTX) with a separate theme engine has no counterpart in any existing open-source tool. Mermaid, Markwhen, and vis-timeline all bake rendering into

the format. Vega-Lite delegates to a JavaScript runtime, making it unsuitable for PPTX or print-PDF output without adaptation that would negate any adoption benefit.

A BUILD decision is therefore justified. It must not be a wheel-reinvention: the following **vocabulary and semantic donors** are explicitly leveraged so that the IR speaks standard language wherever standards exist.

ISO 8601 [12]

Date string syntax (2026-06-09, 2026-Q2). Already used in the date model (§0.4.5); no change needed.

iCalendar RFC 5545 [5]

Field naming: DTSTART → start, DTEND → end, SUMMARY → label, DESCRIPTION → description, CATEGORIES → tags/category, URL → url. Status vocabulary: TENTATIVE appears verbatim in the Status enum; CANCELLED maps to cancelled.

Schema.org/Event [35]

Confirms name/startDate/endDate/ description/url as the canonical web vocabulary for events. Mark's label/start/end are direct equivalents (shorter names preferred for human authoring).

W3C OWL-Time [42] and Allen's Algebra [2]

Open-interval semantics: omitting hasEnd denotes an ongoing interval, validating end: ongoing. The Instant/Interval duality formalises the Milestone (instant) vs. Activity (interval) distinction.

Vega-Lite [33]

Architectural precedent: strict WHAT/HOW separation between a declarative specification (this IR) and the rendering runtime (Theme Engine + Renderer). The pattern of a versioned, typed YAML/JSON document as the sole contract between source and output is adopted from Vega-Lite's design.

Markwhen [16] and Mermaid [18]

Surface-syntax precedents: readable event lines, section/group structure, tag-based categorisation. These inform future higher-level authoring syntaxes that compile down to this IR.

Pandoc AST [13]

Structural analogy: versioned, typed, language-agnostic document IR with version + meta + typed entity lists. Validates the root structure (§0.4.3) and the principle of a stable, machine-verifiable schema.

0.4.20.9 Alignment of Mark's IR with Prior Art

Table 19 maps Mark's field names to their closest standard equivalents. Fields marked *Original* have no adequate standard analog and are justified by the visual-communication use case.

Flags for Mark and Leslie (no schema changes required).

- **No schema changes are recommended.** Mark's field choices are well-aligned with iCalendar RFC 5545, schema.org/Event, and OWL-Time. The original con-

tributions (`at-risk`, `blocked`, `span`, `progress`, `track`) are justified by the visual-communication use case and have no adequate standard equivalent.

- **Document vocabulary donors in the spec preamble.** This section serves that purpose. Future contributors should not propose renames (e.g., `name` for `label`, `startDate` for `start`) without consulting this survey, since shorter names are deliberate and standards-corroborated.
- **Optional surface alias `type: event`:** `schema.org` uses `Event` and `OWL-Time` uses `Instant` for point-in-time items. Exposing `type: event` as a surface-syntax alias for milestones could ease LLM generation without changing the IR schema. Flagged for future surface-language design.

0.5 Rendering Model

The rendering model defines a *deterministic mapping* from a Timeline IR document and a theme to a concrete visual geometry: positions, dimensions, label placements, and visual treatments. “Deterministic” is not aspirational; it is a hard contract. Two conforming renderers, given identical IR and theme, must produce geometrically identical output. If they diverge, a defect exists in one renderer—not an interpretation ambiguity in this specification [33].

The model is organized as an ordered six-phase pipeline. Each phase consumes only outputs of preceding phases and produces geometry for later phases. The pipeline is *pure*: no phase reads from the IR after its designated turn, and no phase performs I/O, reads a system clock, or invokes a random number generator.

0.5.1 Determinism Contract

Two conforming renderers R_1 and R_2 , given the same IR document D and theme θ , must produce identical coordinate values, label positions, and visual-treatment assignments. This contract requires:

1. **Pure function.** Output is a pure function of (D, θ) . No external state—timestamps, random values, environment variables, system locale, or screen resolution—influences any computed value.
2. **Stable sort keys.** Every ordered collection is sorted by an explicit, unambiguous key sequence. Collections without semantic ordering sort by `id` alphabetically:
 - Tracks: by `index` ascending (integer, unique by well-formedness invariant).
 - Activities within a track: by `(start_ordinal, id)` ascending.
 - Milestones: by `(date_ordinal, id)` ascending.
 - Annotations, sections, groups: by `id` ascending.

The ID values break all ties; global uniqueness is guaranteed by the IR invariant.

3. **Fixed rounding.** All intermediate floating-point values are rounded using *round-half-up* ($\lfloor v + 0.5 \rfloor$). Scene geometry values are expressed to two decimal places using the same convention. No other rounding rule is used anywhere in the pipeline.

4. **Deterministic symbolic date resolution.** The symbolic date `now` and relative dates (`+3m`, `-2w`) resolve to a fixed anchor. The anchor is: `metadata.today` if present; else `metadata.created`; else a validation error (see Gap 1 in §0.5.7). The system clock is never consulted.
5. **Embedded font metrics.** Label-width and line-height computations use metrics from font files bundled with the theme. System fonts are not consulted for any layout-critical measurement. This guarantees that label placement is identical on macOS, Linux, and Windows.
6. **Specification-version governance.** The renderer reads `version` from the IR document and verifies it against a supported specification version before beginning layout. A version mismatch is a hard error, not a warning.
7. **Layered determinism.** The determinism contract applies at three levels. (i) *Scene geometry* is always byte-deterministic: the layout pipeline output is bitwise-reproducible across runs, platforms, and renderer implementations. (ii) *Per-backend output* is deterministic given a pinned backend version and fixed effect seeds; the backend version is an explicit parameter of the contract. (iii) *Cross-backend pixel identity* is explicitly *not* promised: the SVG backend and the Raster backend are permitted and expected to produce visually different outputs from the same Scene. This is correct behaviour at different fidelity tiers, not a defect. See Section 0.7.1.2 for the full layered contract.

Consequence. If R_1 and R_2 both satisfy this contract and their outputs diverge, the divergent renderer has a defect. The specification is the arbiter.

0.5.2 Coordinate System

The renderer works in a *logical-pixel* coordinate system. The top-left corner of the canvas is the origin (0, 0); x increases rightward, y downward. All layout computations use integer arithmetic in logical pixels; the Scene/Render IR records decimal-valued coordinates as its output (see Section 0.5.5).

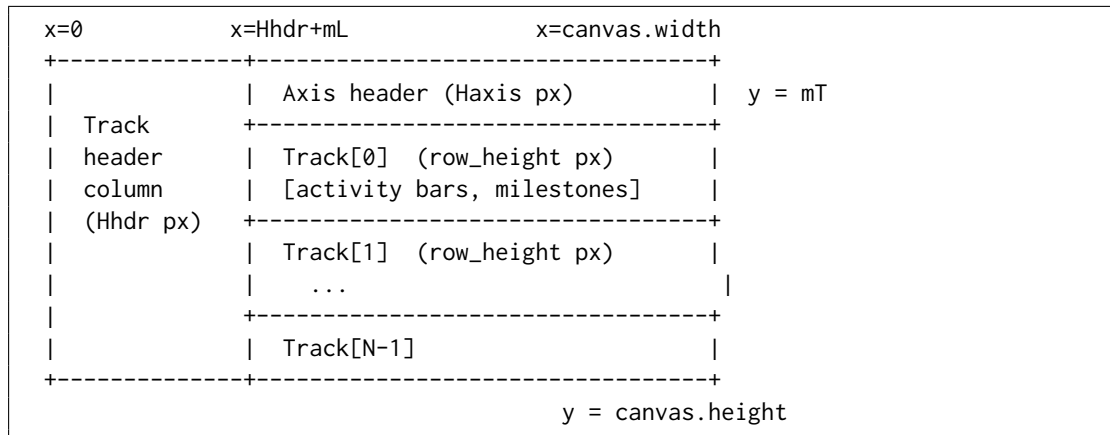


Figure 1: Canvas coordinate system and layout zones

The canvas *width* is fixed by the theme. The canvas *height* is always computed from track count and sub-lane expansion; it must never be truncated.

0.5.3 Timeline Axis

0.5.3.1 Axis Unit and Inference

`metadata.axis_unit` controls tick granularity. If absent, the renderer infers a suitable unit from the `time_range` span, applying the first matching threshold:

These thresholds yield 4–26 visible ticks at a 1200-pixel canvas width—the range that avoids crowding while remaining informative for the executive-roadmap use cases described in Section 0.2.

0.5.3.2 Date-to-Coordinate Function

All horizontal positions derive from a single deterministic function $x(T)$. Let T_{ord} denote the *day ordinal* of date T : an integer count of days since 2000-01-01 (epoch day 0). Coarser-precision dates are resolved to their period-start day before ordinal conversion (see §0.5.3.3).

$$x(T) = H_{\text{hdr}} + m_L + \left\lfloor \frac{(T_{\text{ord}} - T_{s,\text{ord}}) \cdot W_{\text{draw}}}{T_{e,\text{ord}} - T_{s,\text{ord}}} + 0.5 \right\rfloor$$

where $T_{s,\text{ord}}$ and $T_{e,\text{ord}}$ are the day ordinals of the axis domain start and end after coercion. The numerator product $(T_{\text{ord}} - T_{s,\text{ord}}) \cdot W_{\text{draw}}$ is computed before division to prevent precision loss. For a 1200-pixel canvas spanning ten years (3650 days), the intermediate product is at most $1200 \times 3650 = 4\,380\,000$, well within 32-bit integer range.

Boundary invariants. $x(T_s) = H_{\text{hdr}} + m_L$ exactly; $x(T_e) = H_{\text{hdr}} + m_L + W_{\text{draw}}$ exactly. Any date at or before T_s maps to the left canvas boundary; any date at or after T_e maps to the right canvas boundary, before clipping.

0.5.3.3 Date Coercion for Bar Edges

Dates coarser than day must be coerced to a specific calendar day before ordinal conversion. The rule depends on whether the date is a *left edge* (start of a bar) or a *right edge* (end of a bar):

Fiscal dates require a fiscal-year start month that is currently absent from the IR contract (see Gap 2 in §0.5.7).

The same left-edge/right-edge coercion applies to `time_range.start` (left) and `time_range.end` (right) when establishing the axis domain.

0.5.3.4 Tick and Gridline Placement

1. Enumerate all `axis_unit` period-start dates within $[T_s, T_e]$ inclusive.
2. For each period boundary T_k : compute $x_k = x(T_k)$ per §0.5.3.2.
3. Place a tick mark of height `theme.axis.tick_height` px at x_k , at the bottom of the axis header band.

4. Place a vertical gridline at x_k spanning from the top of `Track[0]` to the bottom of `Track[N - 1]`, styled per `theme.axis.gridline_*` properties.
5. Place a tick label centered at x_k , `theme.axis.tick_label_offset` px below the tick. Format using `metadata.locale` and the label rules in Table 23. If a label would overlap its neighbor (right edge > neighbor's left edge), suppress the label for this tick only.

Week ticks. For `axis_unit = week`, week boundaries are ISO 8601 Monday starts regardless of locale. The locale affects label string formatting only (language, order), never tick positions.

Section bands. If `sections[]` is non-empty, the theme may render alternating column shading between section boundaries. Band edges coincide with section start/end dates coerced per Table 22. Section bands are painted at the lowest z-order, below gridlines and all content elements.

0.5.4 The Six-Phase Layout Algorithm

The renderer executes geometry computation in exactly six phases in the order given below. Each phase's output is immutable input to subsequent phases. No phase may invoke logic from a later phase.

```

1 Phase 1 | AXIS COMPUTATION
2         |   Input:  IR metadata (time_range, axis_unit, today-anchor)
3         |   Output: axis domain [Ts, Te], axis_unit, tick_positions[]
4
5 Phase 2 | TRACK PLACEMENT
6         |   Input:  tracks[] sorted by index
7         |   Output: track.y_top[], track.height[] (provisional)
8
9 Phase 3 | ACTIVITY GEOMETRY
10        |   Input:  activities[], Phase-1 axis, Phase-2 track
11        |           positions
12        |   Output: activity.{x_left, x_right, y, lane, width}
13        |           track.height[] (final, expanded for sub-lanes)
14
15 Phase 4 | MILESTONE GEOMETRY
16        |   Input:  milestones[], Phase-1 axis, Phase-3 final track
17        |           heights
18        |   Output: milestone.{x_center, y_center, stack_index}
19
20 Phase 5 | SECTIONS AND ANNOTATIONS
21        |   Input:  sections[], annotations[], all Phase 1-4 geometry
22        |   Output: section.{x_left, x_right}, annotation.{x, y}
23
24 Phase 6 | LABEL COLLISION RESOLUTION
25        |   Input:  all geometry from Phases 3-5
26        |   Output: final label positions for milestones

```

Listing 18: Layout pipeline overview

0.5.4.1 Phase 1: Axis Computation

```

1  1. Resolve today-anchor:
2      if metadata.today is set : anchor = metadata.today
3      elif metadata.created is set : anchor = metadata.created
4      else : ERROR("Cannot resolve symbolic/relative dates: no
5              today/created")
6
7  2. Resolve time_range.start:
8      a. Expand 'now' -> anchor; expand '+3m' -> anchor + 3 months;
9      etc.
10     b. Coerce to period start (left-edge rule, Table 5.3). -> T_s
11
12  3. Resolve time_range.end (same expansion; coerce to period end ->
13     T_e).
14     ASSERT T_e >= T_s; else ERROR.
15
16  4. Infer axis_unit if absent:
17     span_days = T_e_ord - T_s_ord
18     Apply Table 5.2 thresholds in order; assign axis_unit.
19
20  5. Compute W_draw = canvas.width - mL - mR -
21     theme.track.header_width.
22
23  6. Enumerate tick positions:
24     T_k = all period-start dates of axis_unit in [T_s, T_e]
25     inclusive.
26     For each T_k: x_k = x(T_k) per Section 5.3.2.
27     Compute tick label strings using metadata.locale.

```

Listing 19: Phase 1: Axis computation

0.5.4.2 Phase 2: Track Placement

```

1  1. Sort tracks by index ascending.
2     ASSERT all index values are unique non-negative integers.
3
4  2. y_cursor = mT + Haxis
5
6  3. For i in 0 .. N_tracks-1:
7     track[i].y_top = y_cursor
8     track[i].height = theme.track.row_height      -- provisional
9     y_cursor      += theme.track.row_height + theme.track.row_gap
10
11  4. Record group bracket extents after Phase 3 finalises track
    heights.

```

Listing 20: Phase 2: Track placement

Track heights set here are provisional. Phase 3 may expand them when overlapping activities require sub-lanes. Downstream phases must use the Phase-3-finalised heights.

0.5.4.3 Phase 3: Activity Geometry

Sub-lane assignment.

```

1 For each track T (in index order):
2   1. A = activities where A.track == T.id.
3   2. Resolve each A.start_ord = ordinal(coerce_left(A.start)).
4     Resolve each A.end_x per special-end-date rules below.
5   3. Sort A by (start_ord, id) ascending (stable sort).
6   4. Greedy sub-lane assignment:
7     lane_end_x = [-infinity]
8     For each A_i in sort order:
9       k = smallest index where lane_end_x[k] <= x(A_i.start_ord)
10      if no such k exists:
11        if len(lane_end_x) < theme.track.max_sub_lanes:
12          append -infinity to lane_end_x; k = new last index
13        else:
14          k = theme.track.max_sub_lanes - 1  -- cap; emit WARNING
15      A_i.lane = k
16      lane_end_x[k] = A_i.end_x
17   5. n_lanes = max(A.lane) + 1
18   6. if n_lanes > 1:
19     track[T].height = theme.track.row_height
20                      + (n_lanes - 1) * theme.track.sub_lane_height
21   7. For each A_i:
22     A_i.y = track[T].y_top
23           + theme.activity.bar_top_margin
24           + A_i.lane * theme.track.sub_lane_height
25     A_i.x_left = x(A_i.start_ord)
26     A_i.x_right = A_i.end_x
27     logical_w = A_i.x_right - A_i.x_left
28     if logical_w < theme.activity.min_width:
29       mid = (A_i.x_left + A_i.x_right) / 2  --
30          round-half-up
31       A_i.x_left = mid - theme.activity.min_width / 2
32       A_i.x_right = A_i.x_left + theme.activity.min_width
33     A_i.width = A_i.x_right - A_i.x_left

```

Listing 21: Phase 3: Activity geometry and greedy sub-lane assignment

Special end-date resolution.

- end absent (omitted): treated identically to ongoing.
- ongoing: $\text{end}_x = H_{\text{hdr}} + m_L + W_{\text{draw}}$ (right canvas boundary); append right-chevron ($\rightarrow$) decoration.
- tbd: $\text{end}_x = x_{\text{left}} + \text{theme.activity.tbd_extension_px}$ (default: $2 \times \overline{\Delta x_{\text{tick}}}$, the mean tick spacing in pixels); render rightward extension with dashed border at 50% opacity; place “TBD” label inside extension zone.
- unknown: same geometry as tbd; use unknown visual treatment from theme status map.
- ~date: coerce date normally to ordinal; $\text{end}_x = x(\text{date_ord})$; flag right edge as approximate for gradient-fade rendering.

Label placement (deterministic three-way rule). The rule is applied exactly once per activity; no iteration:

1. **Inside:** if $A.width \geq theme.activity.label_inside_min_width$, render label horizontally centered in bar, vertically centered on bar height, in `theme.activity.label_color_inside`.
2. **Right:** render label starting at $A.x_right + 4$ px in `theme.activity.label_color_outside`. If label right edge $> H_{hdr} + m_L + W_{draw} - 4$, proceed to step 3.
3. **Left:** render label ending at $A.x_left - 4$ px. If label left edge $< H_{hdr} + m_L$, truncate label to `theme.activity.label_truncate_chars` characters + “...” (U+2026) and re-attempt step 2.

Progress indicator. If `activity.progress` is set: render a filled strip of height `theme.activity.progress_bar_height` px at the bottom of the bar, width = $\lfloor A.width \cdot A.progress + 0.5 \rfloor$ px, inset inside the bar border.

0.5.4.4 Phase 4: Milestone Geometry

Placement and stacking.

```

1 For each milestone M (sorted by date_ord, id):
2   1. x_center = x(M.date_ordinal).
3   2. Determine base_y:
4       if M.track is set:
5         base_y = track[M.track].y_top + track[M.track].height / 2
6       else (cross-track):
7         base_y = mT + Haxis + H_draw / 2
8
9   3. Stacking within (effective_track_id, x_center) group:
10      Sort group members by id ascending.
11      Assign stack_index k = 0, 1, 2, ...
12      M.y_center = base_y + k * theme.milestone.stack_offset_y
13
14   4. Diamond vertices (integer arithmetic):
15      d = theme.milestone.diamond_size
16      top    = (x_center,      M.y_center - d)
17      right  = (x_center + d, M.y_center   )
18      bottom = (x_center,      M.y_center + d)
19      left   = (x_center - d, M.y_center   )

```

Listing 22: Phase 4: Milestone placement and stacking

The diamond is a four-vertex polygon with vertices computed directly. No `transform="rotate(...)"` attribute is used; rotation-transform-dependent rendering differences across SVG viewers are avoided.

Cross-track milestones. When `track` is null, the milestone is drawn at the vertical centre of the full rendered area. Its diamond size is `theme.milestone.diamond_size` (not scaled); visual emphasis comes from the spanning vertical extent of the label, which may be increased by a theme multiplier `theme.milestone.cross_track_label_scale` (default: 1.2).

0.5.4.5 Phase 5: Sections and Annotations

Section bands. For each section S (sorted by start date, then id):

- $x_{\text{left}}(S) = x(\text{coerce_left}(S.\text{start}))$.
- $x_{\text{right}}(S) = x(\text{coerce_right}(S.\text{end}))$.
- Band spans vertically from $m_T + H_{\text{axis}}$ to $m_T + H_{\text{axis}} + H_{\text{draw}}$.
- Z-order: sections paint first (below gridlines, activities, milestones, labels).

Overlapping sections are valid; a later section in sort order paints over an earlier one.

Annotation placement.

1. Sort annotations by id ascending.
2. For each annotation: compute anchor as centre of target entity's bounding box.
3. Place annotation box in the first quadrant that keeps it entirely within the canvas.
Preference order: top-right, top-left, bottom-right, bottom-left. If no quadrant fits, use top-right and clip to canvas boundary.
4. Draw connector (line/elbow) from annotation box edge to anchor, styled per `theme.annotation.connector`.

0.5.4.6 Phase 6: Label Collision Resolution

Activity labels are placed definitively in Phase 3 (no iteration). Milestone labels require a collision-resolution pass because multiple milestones at similar x positions can produce overlapping labels.

```

1  1. Collect all milestone label bounding boxes (x_label, y_label,
   width, height, id).
2  2. Sort by (x_label, y_label, id) ascending (stable).
3  3. Scan pass (bounded by N iterations, N = label count):
4      For each consecutive pair (L_i, L_{i+1}):
5          if L_i.right_edge > L_{i+1}.left_edge
6              AND |L_i.y_label - L_{i+1}.y_label| < label_height:
7              L_{i+1}.y_label += theme.milestone.label_stack_offset
8  4. Repeat step 3 until no overlapping pairs remain OR N iterations
   reached.
9      If overlaps persist after N iterations: truncate labels to
10     theme.milestone.label_max_width px, accepting visual overlap.
11 5. Clamp all final y_label values to [mT, mT + Haxis + H_draw + mB].

```

Listing 23: Phase 6: Milestone label collision resolution

Because the input sort and every shift decision are deterministic, two renderers executing this algorithm on identical inputs produce identical label placements.

0.5.5 Scene / Render IR — Output of the Pipeline

The six-phase layout pipeline does not target any output format directly. Its output is the **Scene / Render IR**: a byte-deterministic, backend-agnostic record of every drawing primitive, resolved coordinate, visual treatment, and effect request. The Scene is described fully in Section 0.7.1; its role here is to name what the pipeline produces and to record what each phase contributes.

Every value computed in Phases 1–6 becomes a field of a Scene drawing primitive or a top-level Scene property. The Scene is a pure data record: it carries no rendering instructions specific to SVG, rasterisation, or PowerPoint. Rendering backends (SVG, Raster, PPTX, HTML) consume the Scene and emit their respective formats; they are described in Section 0.7.

What the Scene preserves from the pipeline.

- **Phase 1** outputs: axis domain $[T_s, T_e]$, axis unit, tick positions x_k , and tick label strings.
- **Phases 2–3** outputs: all activity bar coordinates $(x_{\text{left}}, x_{\text{right}}, y, \text{width})$, lane assignments, and final track heights.
- **Phase 4** outputs: all milestone vertex coordinates and stack indices.
- **Phase 5** outputs: section band bounds $(x_{\text{left}}, x_{\text{right}})$, annotation positions, and connector endpoints.
- **Phase 6** outputs: final label positions after collision resolution.
- **Theme-resolved visual treatments**: fill colours, stroke styles, opacity, corner radii, pattern references, and effect requests with their fallback policies (see Section 0.6.2.10).

The Scene does not carry the IR document or theme; it is the fully resolved, self-contained rendering specification. A backend needs only the Scene and its own capability profile to produce output.

0.5.6 Edge Cases

Every case below is normative. A renderer that produces different behaviour for any case has a defect. The summary table gives the complete catalogue; extended descriptions follow for the most structurally complex cases.

Table 24: Edge-case definitions (normative)

Case	Condition	Rendering Rule
Zero-duration activity	<code>start == end</code> or <code>span</code> that resolves to a single point	Render bar of <code>min_width</code> px centred at $x(\text{start})$. Never render 0-width. Label placed right (outside).
Overlapping activities (same track)	Two activities in same track whose date ranges overlap	Greedy sub-lane assignment (Phase 3). Track height expands. Cap at <code>max_sub_lanes</code> .
Open interval (ongoing)	<code>end: ongoing</code> or <code>end</code> absent	Bar extends to right canvas boundary. Right-chevron decoration. No overflow beyond canvas edge.
TBD end date	<code>end: tbd</code>	Bar extends <code>tbd_extension_px</code> beyond <code>x_left</code> . Dashed, 50% opacity. “TBD” label inside extension.

continued ...

Table 24 continued

Case	Condition	Rendering Rule
Both dates TBD	start: tbd or equivalent	Full-track-width hatched block at 40% opacity. Activity label + “(TBD)”.
Unknown start	start: unknown	Left edge placed at $x(T_s)$. Left-fade gradient over approx_fade_px.
Unknown end	end: unknown	Same geometry as tbd; rendered with unknown visual treatment from theme.
Approximate start	start: ~date	Geometry at nominal date; left edge rendered with gradient fade of approx_fade_px.
Approximate end	end: ~date	Geometry at nominal date; right edge rendered with gradient fade.
Activity fully before range	end < T_s	Not rendered. Renderer warning. Activity still appears in legend.
Activity fully after range	start > T_e	Not rendered. Renderer warning. Activity still appears in legend.
Activity partially before range	start < T_s , end $\geq T_s$	Bar clipped to $[x(T_s), x(\text{end})]$. Left-clip indicator on clipped edge.
Activity partially after range	start $\leq T_e$, end > T_e	Bar clipped to $[x(\text{start}), x(T_e)]$. Right-clip indicator on clipped edge.
Very short span	$x(\text{end}) - x(\text{start}) < \text{min_width}$ after rounding	Render at min_width; centre on logical midpoint (round-half-up). Label right (outside).
Very long span	Bar occupies > 80% of W_{draw}	Render normally. Prefer inside label placement if label fits within bar.
Simultaneous milestones (same track)	Two milestones with equal date on the same track	Stack vertically by stack_offset_y, sorted by id ascending (Phase 4).
Simultaneous milestones (cross-track)	Two milestones with track: null and equal date	Stack at full-timeline vertical centre, sorted by id ascending.
Milestone outside range	Milestone date < T_s or > T_e	Not rendered. Renderer warning. Appears in legend.
Empty track	Track has no activities and no milestones	Row rendered at row_height. Label visible. Body empty. Never collapsed.
Sub-lane cap exceeded	Overlapping activities require > max_sub_lanes lanes	Excess activities placed in the last lane (visible overlap). Renderer warning.

Overlapping activities: sub-lane assignment in detail. The greedy algorithm assigns each activity (in stable (start_ord, id) order) to the lowest-indexed lane where

it does not overlap any previously placed activity. Overlap is defined as $x_{\text{left}}(A_j) < x_{\text{right}}(A_i)$ for activities A_i, A_j in the same lane. Because sort order is deterministic and each assignment is a pure function of the current `lane_end_x` state, all renderers produce identical lane assignments for identical inputs.

Simultaneous milestones in detail. When milestones share an `(track, x_center)` pair, they are stacked downward from the nominal track centre. The first milestone in id-ascending order is at the base position (stack index 0). Subsequent milestones are offset by $k \times \text{theme.milestone.stack_offset_y}$. Stacking is permitted to overflow into the inter-track gap; milestones must *not* be silently suppressed if they cannot fit within track height.

Clip indicators. An activity bar clipped at the axis boundary receives a *clip indicator*: a small “//” mark (two angled lines, 4×8 px) drawn in the activity’s fill colour at full opacity, positioned at the clipped edge centre. The indicator signals to the reader that the activity continues beyond the visible axis range.

Minimum-width semantics. `theme.activity.min_width` (default: 4 px) prevents activities from being rendered invisibly narrow at coarse zoom levels. A zero-duration activity and a very-short-span activity both produce a bar of exactly `min_width` px. The two cases are *visually indistinguishable*; distinguishing them would require a separate icon or tooltip, which is a theme-level option outside the core rendering contract.

0.5.7 Known IR Gaps

The following gaps in Mark’s IR contract (§0.4) affect rendering determinism. They are flagged here for Mark and Leslie to resolve; this spec does *not* unilaterally extend the IR. Renderers encountering these gaps should apply the documented fallback and emit a validation warning.

Gap 1 — `metadata.today` field missing.

Leslie’s decision record mandates an explicit `today` field in the IR for deterministic resolution of the now symbolic date and the today-marker feature. Mark’s current `metadata` object does not include this field. **Proposed resolution:** add `today` (type `date`, optional) to `metadata`. Renderer fallback chain: `metadata.today` → `metadata.created` → validation error.

Gap 2 — `metadata.fiscal_year_start` missing.

The FY26-Q2 fiscal-date format requires knowing the fiscal-year start month. Two renderers assuming different organisations’ fiscal calendars (US federal: October; UK: April; common corporate: January) will produce different x -coordinates for identical fiscal dates. **Proposed resolution:** add `fiscal_year_start` (type `int`, range 1–12, default 1 = January) to `metadata`.

Gap 3 — Relative date anchor undefined.

Relative dates (+3m, -2w) are described in Mark’s contract as resolving from “context”, which is insufficient. A renderer must know the anchor date to compute an ordinal. **Proposed resolution:** define that relative dates use the same anchor as Gap 1 (`today` → `created` → error).

Gap 4 — Omitted end semantics ambiguous.

Mark's Activity contract marks end as optional but does not state whether an absent end is equivalent to ongoing or a validation error. This rendering model treats omitted end as ongoing (see the open-interval row in Table 24). The IR spec should make this explicit.

0.6 Theme Architecture

A theme is a *pure styling layer*: it accepts the geometry produced by the rendering model (Section 0.5) and assigns colors, typography, shapes, and decorations, producing a final visual specification. A theme operates exclusively on signals the renderer has already computed—coordinates, status values, categories, progress fractions. It may not alter geometry, and it may not require IR fields beyond those defined in Section 0.4 [33, 40].

Theme-engine contract (binding): A theme MUST accept all valid IR documents. A theme MUST NOT require IR fields beyond the specification. Two valid themes applied to the same IR may produce visually different outputs, but their underlying geometries—coordinates, stacking order, label positions—must be identical.

This section defines the complete theme schema, describes the five built-in themes, and illustrates how the same IR renders visually differently under each.

0.6.1 Theme Design Philosophy

Themes are built on four principles:

- **Data-independent.** A theme knows nothing about the content of the IR. It maps semantic signals (status, category) to visual treatments through fixed lookup tables. No IR field other than those defined in Section 0.4 is read.
- **Composable.** Themes support single inheritance. A child theme specifies only the properties that differ from its parent; all others inherit. This enables branded variants (acme-consulting extending consulting) with minimal duplication.
- **Separable.** Swapping themes requires zero IR changes. The same YAML document renders correctly under any theme. Content and style are fully orthogonal.
- **Deterministic.** Themes introduce no layout variability. Given the same IR, theme, and rendering algorithm, output is fully determined (see Section 0.5.1).

0.6.2 Theme Schema

The theme schema defines all visual properties a theme may configure. Properties not set by a theme are inherited from its extends parent, defaulting to the built-in default theme if no parent is specified. The schema is organized into blocks; all property names, types, and defaults apply uniformly across themes.

0.6.2.1 Canvas and Margins

```

1 canvas:
2   width:          1200 # logical pixels; height is always computed
3   background_color: "#FFFFFF"
4   margin:
5     top:         48      # px above the axis header
6     right:       40      # px right of the last tick
7     bottom:     48      # px below the last track row
8     left:        0       # px left of the track header column

```

Listing 24: Theme schema: canvas block

The canvas height is computed from track heights and inter-track gaps (§0.5.2) and is never specified in a theme.

0.6.2.2 Typography

```

1 typography:
2   font_family:      "Helvetica Neue, Arial, sans-serif"
3   # Embedded font files required for deterministic metric
4   computation:
5   font_files:
6     regular: "fonts/HelveticaNeue-Regular.woff2"
7     bold:    "fonts/HelveticaNeue-Bold.woff2"
8   font_size_base:   11 # pt; body text and activity labels
9   font_size_axis:   10 # pt; axis tick labels
10  font_size_title:   18 # pt; document title in canvas header
11  font_size_subtitle: 13 # pt
12  font_size_track:   11 # pt; track header labels
13  font_weight_label: 600
14  font_weight_axis:  400
15  font_weight_header: 700
16  locale_aware:      true # use metadata.locale for date label
17  formatting

```

Listing 25: Theme schema: typography block

Font embedding requirement. Themes must specify embedded font files (WOFF2 or equivalent) for all families used in layout-critical computations (label width, line height, character advance widths). System fonts are permitted only for decorative text—e.g., HTML tooltip overlays—where exact cross-platform metric agreement is not required. The renderer ships the embedded metrics tables at build time.

0.6.2.3 Axis Styling

```

1 axis:
2   height:          40 # px; height of the time-axis header
3   band
4   tick_height:     6 # px; length of tick marks
5   tick_label_offset: 4 # px; gap between tick mark base and
6   label top
7   gridline_color:   "#E0E0E0"
8   gridline_width:   0.5 # px

```

```

7  gridline_opacity:      0.8
8  gridline_style:       "solid" # solid | dashed | none
9  section_band_alpha:   0.04 # alternating column shading opacity (0
    = off)
10 today_marker:
11   enabled:             false
12   color:               "#CC2222"
13   width:               1.5 # px
14   style:               "solid" # solid | dashed
15   label:               "Today"
16   label_visible:      true

```

Listing 26: Theme schema: axis block

0.6.2.4 Track Styling

```

1  track:
2   header_width:         160 # px; fixed left column for track label
    text
3   row_height:           48 # px; single-lane track height (no
    sub-lanes)
4   sub_lane_height:      24 # px; height added per additional
    overlap lane
5   max_sub_lanes:        8 # hard cap; excess activities go to
    last lane
6   row_gap:              8 # px; vertical gap between consecutive
    tracks
7   header_font_size:     11 # pt
8   header_font_weight:   600
9   header_color:         "#111111"
10  header_background:    null # null = transparent
11  header_align:         "right" # right | left | center
12  separator_color:      "#CCCCCC"
13  separator_width:       0.5
14  group_bracket_width:   4 # px; left-edge bracket for grouped
    tracks
15  group_label_size:      10 # pt; group label above bracket

```

Listing 27: Theme schema: track block

0.6.2.5 Activity Styling

```

1  activity:
2   bar_height:           24 # px; height within a single lane
3   bar_radius:           2 # px; corner radius
4   bar_top_margin:       12 # px; from lane top to bar top (centres
    bar in lane)
5   min_width:            4 # px; minimum rendered bar width (see
    edge cases)
6   tbd_extension_px:     null # null = 2 * mean tick spacing in pixels
7   approx_fade_px:       8 # px; gradient fade width for
    approximate-date edges
8   label_inside_min_width: 40 # px; bar must be >= this to place
    label inside
9   label_font_size:      10 # pt

```

```

10  label_color_inside: "#FFFFFF"
11  label_color_outside: "#111111"
12  label_truncate_chars: 32 # max chars before ellipsis truncation
13  progress_bar_height: 4 # px; filled strip at bar bottom for
    progress value
14  progress_bar_alpha: 0.35
15  progress_label_visible: false

```

Listing 28: Theme schema: activity block

0.6.2.6 Milestone Styling

```

1  milestone:
2    diamond_size: 14 # px; half-diagonal of the diamond
    polygon
3    diamond_stroke_width: 1.5 # px
4    cross_track_label_scale: 1.2 # label font size multiplier for
    cross-track
5    label_offset_x: 8 # px; gap from diamond right vertex to
    label left
6    label_font_size: 9 # pt
7    label_max_width: 80 # px; truncate label if wider
8    label_stack_offset: 14 # px; y-shift per collision pass (Phase
    6)
9    stack_offset_y: 20 # px; y-offset per stacking slot (Phase
    4)

```

Listing 29: Theme schema: milestone block

0.6.2.7 Annotation and Legend Styling

```

1  annotation:
2    font_size: 9 # pt
3    background_color: "#FFFFFF0"
4    background_alpha: 0.9
5    border_color: "#CCCC88"
6    border_width: 0.5
7    padding: 6 # px; all sides
8    connector_style: "elbow" # elbow | straight | none
9
10 legend:
11   position: "bottom" # bottom | right | top | none
12   item_height: 14 # px
13   item_gap: 8 # px
14   font_size: 9 # pt
15   swatch_size: 10 # px square
16   column_spacing: 16 # px between columns

```

Listing 30: Theme schema: annotation and legend blocks

0.6.2.8 Status-to-Style Map

Every theme must define a complete `status_map` covering all seven IR status values. The map is a lookup table: `status` $\rightarrow$ { fill, stroke, opacity, pattern }.

Pattern vocabulary. Patterns are defined as named SVG `<pattern>` elements in the theme's `<defs>` block. All themes must use the same pattern geometry so that cross-theme pattern rendering is consistent:

- **solid** — no pattern; fill and stroke colours only.
- **diagonal-hatch** — 45° lines at 4 px spacing, 0.5 px stroke width, stroke colour = `status.stroke`, fill = `status.fill` at 30% opacity.
- **dashed-border** — solid fill at `status.fill`; border rendered as a dashed stroke (4 px on, 4 px off) at `status.stroke`.

Status-map completeness. A theme with a missing status entry is malformed. The renderer must detect and report this at theme load time, not at render time.

0.6.2.9 Category-to-Style Map

Categories are arbitrary strings defined by IR authors. Themes provide an optional `category_map` that overrides colour for activities carrying a matching `category` value:

```
1 category_map:
2   infrastructure: { fill: "#2D6A8F", stroke: "#1A4A6A" }
3   security:      { fill: "#8B0000", stroke: "#5A0000" }
4   ux:            { fill: "#4A7C59", stroke: "#2A5C39" }
```

Listing 31: Category map example

Category styling overrides *only* the fill and stroke of the status entry. Opacity and pattern are always drawn from the status map. This preserves the semantic visual signal of status (done = muted, cancelled = hatched) while allowing brand-aligned colour coding per initiative type.

If an activity's category is absent from `category_map`, status-only styling applies without error or warning.

0.6.2.10 Fidelity Tier and Effect Knobs

A theme declares its intended *fidelity tier* and the effect classes it requests. The rendering backend uses the tier to determine which capability profile to activate; themes degrade gracefully when targeting a backend with lower capability (Section 0.6.5).

```
1 fidelity:
2   tier: 1          # 0=Minimal | 1=Crisp | 2=Polished | 3=Showcase
3   effects:
4     drop_shadow:
5       enabled:      false
6       color:        "#000000"
7       opacity:      0.25
8       offset_x:     2          # px
9       offset_y:     2          # px
10      blur_radius:   4          # px
11      fallback_policy: "approximate" #
12      approximate|omit|embed-raster
13      glow:
```

```

13     enabled:          false
14     color:            "#FFFFFF"
15     radius_px:        6
16     intensity:         0.6
17     fallback_policy:  "omit"
18   cloud_layer:
19     enabled:          false
20     opacity:          0.12
21     scale:            1.0
22     color_stops:
23       - { offset: 0.0, color: "#FFFFFF" }
24       - { offset: 1.0, color: "#A0C8FF" }
25     fallback_policy:  "embed-raster"
26   noise_texture:
27     enabled:          false
28     opacity:          0.05
29     scale:            0.8
30     turbulence_type:  "fractalNoise"
31     fallback_policy:  "omit"

```

Listing 32: Theme schema: fidelity block

Effect blocks not declared in a theme default to `enabled: false`. A Tier-0 or Tier-1 theme SHOULD leave all effect knobs disabled; the renderer emits a warning if a low-tier theme requests an effect that requires Tier-2 or above.

0.6.3 Built-in Themes

Five built-in themes address the primary use cases identified in David’s research. Each is a complete, self-contained style specification; none requires IR changes or additional IR fields.

0.6.3.1 Consulting Theme

Fidelity tier: Tier 1 (Crisp).

Visual identity. McKinsey/BCG-style executive roadmap. Black, white, and a single accent colour (default navy #1F497D). Heavy typography, generous whitespace, no decorative chrome. Optimised for A4/Letter printing and A3 poster output. Visual language matches consulting deliverables that non-technical executives expect [40].

- **Font:** Helvetica Neue (Arial fallback); weights 400/600/700. Embedded for metric determinism.
- **Canvas:** 1200 px wide; 48 px top/bottom margins; track header width 180 px.
- **Axis:** No gridlines. Tick marks only. Quarter labels in bold uppercase (e.g., **Q1 2026**). Today marker disabled by default.
- **Tracks:** Wide headers (180 px). Thick separator (1 px, #333333). No row background colour alternation.
- **Activities:** Square corners (`bar_radius: 0`). Two status colours: navy for planned/in-progress, mid-grey for done. Cancelled activities are rendered at 0% opacity with

diagonal hatching at 20% opacity (structural presence without visual weight). No progress bar by default.

- **Milestones:** Solid black diamonds, 14 px half-diagonal. Label right, bold, 9 pt.
- **Legend:** Suppressed (`position: none`). Status is communicated by colour; a legend would clutter the clean aesthetic.

Use case: Programme timelines, transformation roadmaps, board presentations. The deliberate sparseness focuses attention on the strategic narrative rather than operational detail.

0.6.3.2 Executive Theme

Fidelity tier: Tier 2 (Polished).

Visual identity. Corporate boardroom aesthetic. Navy/slate palette on white, with subtle alternating quarter column shading and a gentle border radius on bars. Status is communicated by both colour and a small status icon at the left of each bar. Designed for 16:9 slide dimensions.

- **Font:** Georgia or Garamond (serif headings); Helvetica Neue (body). Both embedded.
- **Canvas:** 1600 px wide (16:9 target); 40 px margins; header width 160 px.
- **Axis:** Light grey gridlines (0.5 px). Month abbreviation labels. Alternating quarter shading (`section_band_alpha: 0.04`). Today marker enabled (red dashed line, “Today” label).
- **Tracks:** Medium headers (150 px). Alternating row background: white / #F8F8F8. Thin separator (0.5 px, #DDDDDD).
- **Activities:** Rounded corners (`bar_radius: 4`). Full status palette: navy (planned), royal blue (in-progress), silver (done), amber (at-risk), coral (blocked), grey hatched (cancelled), grey dashed-border (tentative). Status icon (8 pt glyph) at bar left edge; icon colour is white on dark fills.
- **Milestones:** Navy diamonds with a white centre dot (4 px radius inset fill). Label right, 9 pt.
- **Legend:** Right-aligned, full status legend.

Use case: Quarterly business reviews, investor roadmaps, product strategy presentations. Polished without being sparse; the status palette lets executives absorb delivery health at a glance.

0.6.3.3 Product Theme

Fidelity tier: Tier 2 (Polished).

Visual identity. Developer-friendly; information-dense. Per-track colour coding from the `color hint` field. Category colours take visual precedence over status colours; status is communicated by a small icon rather than fill. Progress bars always visible when set.

- **Font:** Inter or Roboto (system-friendly sans-serif), 10 pt base. Compact sizing for high information density.
- **Canvas:** 1400 px wide; tighter margins (32 px top/bottom); header 140 px.
- **Axis:** Dashed monthly gridlines. Today marker always visible and prominent. Week numbers as secondary axis labels when `axis_unit = week`.
- **Tracks:** Color-coded header backgrounds when `track.color` hint is set. Compact rows (36 px). Sub-lanes shown without track-height expansion cue — the density is expected.
- **Activities:** Slightly rounded bars (`bar_radius: 3`). Category colours override status fill; status glyph icon at bar left edge. Progress bar always rendered (even if thin) when `progress` is set.
- **Milestones:** Smaller diamonds (10 px half-diagonal). Label *below* diamond, 8 pt. Stacks horizontally when space permits.
- **Legend:** Bottom; full category + status legend in two rows.

Use case: Product roadmaps for engineering and product-management audiences. Dense information; appropriate for teams who need per-track status at a glance.

0.6.3.4 Release Theme

Fidelity tier: Tier 1 (Crisp).

Visual identity. Engineering release calendar. Traffic-light status palette (green/amber/red). Monochrome headers. Today marker is the most prominent element. Optimised for CI/CD dashboard display and dense sprint/release schedules.

- **Font:** JetBrains Mono or Courier New (monospace). Consistent character widths simplify label truncation computations.
- **Canvas:** 1200 px wide; tight margins (24 px top/bottom); header 120 px.
- **Axis:** Solid weekly gridlines (1 px, #CCCCCC). Today marker: bold red solid line (2 px), “TODAY” label in uppercase red above axis.
- **Tracks:** No separator lines; alternating row backgrounds (#FFFFFF / #F4F4F4) imply boundaries. Minimal header (dark text on light bg).
- **Activities:** No border radius (`bar_radius: 0`). Traffic-light fill: #2E7D32 (done), #1565C0 (in-progress), #757575 (planned), #F57F17 (at-risk), #C62828 (blocked); diagonal hatching for cancelled; dashed border for tentative. No status icon; colour is the sole status signal.
- **Milestones:** Upward-pointing triangle (`shape: triangle-up`), 12 px height, filled with status colour. This is a theme-level shape override; the IR still uses the Milestone entity type.
- **Legend:** Top-right; compact icon + one-word label per status.

Use case: Software release schedules, deployment windows, sprint calendars. Engineers over executives; clarity of status over aesthetic refinement.

0.6.3.5 Minimal Theme

Fidelity tier: Tier 0 (Minimal).

Visual identity. Maximum whitespace, minimum decoration. All bars are a single dark grey regardless of status; status is communicated by pattern alone (hatching for cancelled, dashed border for tentative). Print-optimised; renders correctly in greyscale. Designed to blend into reports and LaTeX documents without visual dominance.

- **Font:** Times New Roman or Latin Modern (LaTeX-compatible serif). 10 pt base.
- **Canvas:** 900 px wide; narrow margins (24 px top/bottom); header 140 px.
- **Axis:** No gridlines. No tick marks. Period labels only (year or quarter), in 9 pt regular weight. No today marker.
- **Tracks:** No header background. Single thin separator (0.5 px, #AAAAAA). No row alternation.
- **Activities:** No border radius. All bars #333333 fill, regardless of status. Pattern differentiates cancelled (diagonal-hatch) and tentative (dashed-border). No progress bar. No icons. Done activities rendered at 65% opacity.
- **Milestones:** Open diamonds (stroke only, no fill), 1 px stroke. Label right, 9 pt regular.
- **Legend:** None (position: none).

Use case: Academic papers, internal reports, LaTeX-embedded timelines, any context where the diagram must be subordinate to surrounding text.

0.6.3.6 Showcase / Keynote Theme

Fidelity tier: Tier 3 (Showcase).

Visual identity. Premium presentation aesthetic with rich art effects. Designed for high-visibility keynotes, external investor presentations, and conference material where the timeline is the centrepiece of the slide. Requires the Raster backend for full fidelity; degrades gracefully to Polished on PPTX (via native PowerPoint effects) and to Crisp on the SVG backend.

- **Font:** Inter or SF Pro (premium sans-serif), embedded. Bold headings, regular body. 12 pt base for comfortable large-screen reading.
- **Canvas:** 1920 px wide (full-HD target); 60 px margins; header 180 px. Deep navy (#0A1628) background variant or crisp white—two palette options, both at Tier 3.
- **Axis:** Subtle gridlines with soft luminous separation. Soft glow on the today-marker line. Large, premium-weighted quarter labels.
- **Tracks:** Frosted-glass-style header backgrounds (white at 8% opacity over the deep background). Thin separator with a faint glow ring.
- **Activities:** Generously rounded bars (bar_radius: 8). Gradient fill from accent-light to accent-dark per status. Drop shadow on each bar (offset_x/y: 2 px, blur_radius: 6 px, fallback_policy: approximate). The Raster backend renders per-pixel; the SVG backend emits feDropShadow with a determinism caveat; the PPTX backend applies native a:outerShdw.

- **Cloud layer:** A subtle atmospheric gradient overlay across the full canvas at 10% opacity via the `cloud_layer` effect (`fallback_policy: embed-raster`). Requires Raster backend; falls back to `omit` on SVG-only targets with no impact on geometry.
- **Milestones:** Large filled diamonds (18 px half-diagonal) with a soft glow ring (`radius_px: 6`, `intensity: 0.6`, white, `fallback_policy: approximate`). Glow maps to native `a:glow` on PPTX; to per-pixel compositing on Raster; to SVG `feGaussianBlur` with caveat on SVG backend.
- **Today marker:** Bold dashed line with a bloom halo effect (`radius_px: 8`, `threshold: 0.8`, `fallback_policy: omit`). Falls back to bold solid line on SVG/PPTX with no geometry change.
- **Legend:** Bottom; white text on a dark translucent background.

Use case: External investor roadmaps, conference keynote slides, and marketing material where the timeline must make an impact. The PPTX hybrid backend provides an editable version with native effects for Tier-2 elements (drop shadows, glow on milestones) and an embedded raster overlay for the cloud layer; the Raster backend produces the full-fidelity PNG or PDF for print and screen.

0.6.4 Cross-Theme Visual Comparison

To illustrate data-independence, consider a single IR with three tracks (Platform, Mobile, Data), six activities across statuses planned / in-progress / done / at-risk / cancelled / tentative, and two milestones (Beta, GA), spanning 2026-Q1 through 2026-Q4.

Geometric invariant. All six renderings share *identical* geometry: the same bar widths, milestone *x*-positions, track heights, and label coordinate values. Only style and effects differ. This invariant can be verified by asserting coordinate equality across all outputs before applying theme styling and effect layers.

0.6.5 Fidelity Tiers and Backend Effect Profiles

Fidelity tiers define what class of visual richness a theme requests and therefore which rendering backend is sufficient to honour it fully. A backend that cannot satisfy the requested tier applies the fallback policies declared in each effect's `fallback_policy` field and degrades gracefully; *the geometry is never affected*.

Same Scene, three backends. Consider a Tier-2 (Polished) theme with drop shadows and a subtle glow on milestone diamonds applied to an identical Scene:

- **SVG backend:** Emits `feDropShadow` and `feGaussianBlur+feComposite` filters. Includes a determinism caveat comment. Output is slightly renderer-dependent but fully inspectable as vector geometry with live text.
- **Raster backend:** Renders glow and shadows pixel-perfectly via Skia or Canvas compositing. Fully deterministic given a pinned backend version. PNG at requested DPI.

- **PPTX backend:** Maps glow to native PowerPoint `a:glow` property and shadow to `a:outerShdw`. No embedded raster layers needed for Tier-2. All bars, diamonds, and labels remain fully editable MSO shapes.

Degradation policy example. A Tier-3 (Showcase) cloud-layer effect targets the SVG backend (e.g., the raster plugin is not installed):

1. The SVG backend checks its capability against `CloudLayer`: “not available.”
2. It reads the effect’s `fallback_policy`: `embed-raster`.
3. It invokes the Raster backend for the cloud-layer group only, receives a PNG, and embeds it as a `<image>` element. The rest of the timeline remains clean vector.
4. If the Raster backend is also unavailable, the policy chain falls back to `omit`: the cloud layer is silently dropped; no geometry is changed.

Geometry invariant under degradation. Fallback decisions never alter coordinates, dimensions, or label positions. The Scene geometry is the invariant shared across all backends and all fidelity levels.

0.6.6 Theme Inheritance and Extension

Themes support single-level inheritance via the `extends` key:

```

1  theme_id:      "acme-corp"
2  display_name:  "ACME Corporate"
3  extends:      "consulting"
4
5  canvas:
6    background_color: "#F5F5F0"    # warm paper white
7
8  typography:
9    font_family: "Futura, Century Gothic, sans-serif"
10   font_files:
11     regular: "fonts/Futura-Book.woff2"
12     bold:    "fonts/Futura-Bold.woff2"
13
14  status_map:
15    planned:
16      fill:    "#005A8E"    # ACME primary brand blue
17      stroke:  "#003A5E"
18    in-progress:
19      fill:    "#003A5E"
20      stroke:  "#001A2E"

```

Listing 33: Custom theme extending Consulting

Inheritance rules:

1. Properties not specified in the child are taken from the parent exactly.
2. `status_map` and `category_map` are merged entry-by-entry: a child overrides only the entries it declares; unspecified entries inherit from parent.

3. Introducing a new font family requires supplying the corresponding embedded font files. A theme that names a font without providing files is malformed.
4. Inheritance depth is exactly one level. Chains (A extends B extends C) are not supported; each theme must be resolvable from its parent alone.
5. A theme may `extend`: `"default"` explicitly to inherit from the baseline default theme without inheriting any of the five named built-in themes.

0.6.7 Theme-Engine Contract (Formal)

A conforming theme engine must satisfy all of the following:

1. **Accept all valid IR.** A theme engine must not reject a well-formed IR document for content-related reasons. An unrecognised `status` value emits a warning and falls back to `planned` styling.
2. **Require no additional IR fields.** The theme engine reads only the fields defined in Section 0.4. It may read hint fields (`activity.color`, `activity.icon`) as advisory inputs but must not fail if they are absent.
3. **Full status-map coverage.** The theme's `status_map` must contain entries for all seven IR status values. A missing entry is a theme authoring error detectable at theme-load time.
4. **Geometry immutability.** The theme engine applies visual treatment only; it may not alter any coordinate computed by the rendering phases in Section 0.5.4. Style does not affect geometry.
5. **Hint fields are advisory.** The `activity.color` and `activity.icon` hint fields may be silently ignored by the theme engine. If honoured, they supplement status styling (colour override only); they do not replace the pattern or opacity components of the status-map entry.
6. **Unknown categories fall back silently.** If an activity's `category` is absent from the theme's `category_map`, status-only styling applies without error or warning. New categories never require theme changes; they are transparent to the theme engine until explicitly mapped.
7. **Effect-tier transparency.** The theme engine declares the fidelity tier and effect knobs; it does not select the rendering backend. Fallback decisions are backend-local: the theme engine is never invoked during fallback resolution; the backend applies the `fallback_policy` from the Scene's `EffectDefinition` directly. This separation ensures themes degrade without requiring theme changes.

0.7 Output Targets

The rendering pipeline produces a deterministic *Scene*—a backend-agnostic description of every drawing primitive, visual treatment, and effect request—from an IR document and theme. The Scene is the stable contract between the layout engine and any output format. Rendering backends consume the Scene and emit their respective outputs. This architecture replaces the prior design in which SVG was the primary format from which all other formats were derived.

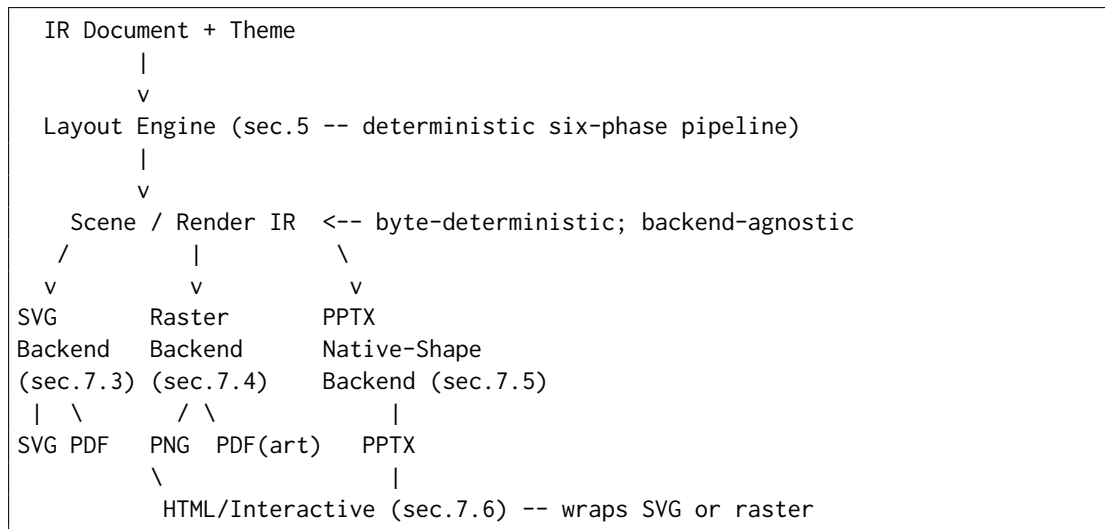


Figure 2: Revised output pipeline: Scene/Render IR as root; SVG, Raster, and PPTX as pluggable backends; PNG and PDF are produced per backend, not universally from SVG

Why SVG is one beloved backend, not the universal root. SVG is not incapable of visual effects. `feGaussianBlur`, `feDropShadow`, `feTurbulence`, and `feColorMatrix` filter primitives exist in SVG and can approximate glow, drop shadows, and noise textures. The architectural problem is different. First, SVG filter rendering is *not deterministic or consistent* across SVG renderers and browsers: identical filter markup produces visually different results in Inkscape, Chrome, Safari, Firefox, and resvg. This breaks the project’s byte-determinism principle for any filter-bearing output. Second, effects expressed as SVG filters do not survive faithfully when rasterising to PNG or converting to PDF, and they have no native equivalent in PPTX’s shape model. Third—and most consequentially—anchoring every output target to SVG’s semantics means the most expressive targets are constrained by the least expressive: clean, deterministic vector geometry is SVG’s strength, but it is not art effects’ domain. The Scene as root resolves this: it describes *what* to draw, including effect requests with explicit fallback policies, independently of *how* any backend draws it.

0.7.1 Scene / Render IR

The Scene is the immutable, byte-deterministic record produced by the six-phase layout pipeline (Section 0.5.4). It is the single handoff point between the layout engine and every rendering backend.

0.7.1.1 Scene Model

The Scene consists of three top-level structures: a **canvas descriptor**, an **ordered element list**, and an **effect registry**. The following is a data-level description, not code:

```

Scene {
  canvas:  { width, height, background_color }

```

```

elements: [ DrawingPrimitive ] -- ordered; painting order preserved
effects: { effect_id -> EffectDefinition }
meta:     { theme_id, fidelity_tier, scene_hash(SHA-256) }
}

DrawingPrimitive := one of:
  Rect    { id, x, y, width, height, fill, stroke, stroke_width,
            corner_radius, opacity, clip_id?, effect_refs[] }
  Polygon { id, vertices[(x,y)...], fill, stroke, stroke_width, opacity }
  Line    { id, x1, y1, x2, y2, stroke, stroke_width, stroke_style,
            opacity }
  Text    { id, x, y, content, font_family, font_size, font_weight,
            color, anchor, clip_id? }
  Path    { id, d_commands[(op,args)...], fill, stroke, stroke_width,
            opacity }
  Image   { id, x, y, width, height, data_uri } -- embedded raster
  Group   { id, children[DrawingPrimitive], clip_rect? }

EffectDefinition := one of:
  Glow      { color, radius_px, intensity, fallback_policy }
  DropShadow { offset_x, offset_y, blur_radius, color, opacity,
                fallback_policy }
  GradientFade { direction, color_stops[(offset,color)...], fade_px }
  NoiseTexture { seed, scale, turbulence_type, opacity, color,
                fallback_policy }
  CloudLayer  { seed, scale, octaves, opacity, color_stops,
                fallback_policy }
  Bloom       { radius_px, threshold, intensity, fallback_policy }

```

Effect requests and fallback policies. Each `EffectDefinition` carries a `fallback_policy` that instructs a backend what to do when it cannot render the effect natively:

Policy value	Meaning
approximate	Use the closest available native mechanism
omit	Silently omit the effect; geometry unchanged
embed-raster	Rasterise the affected layer; embed as <code>Image</code>
error	Abort with a descriptive diagnostic message

Themes declare effects and their fallback policies (Section 0.6.2.10 and 0.6.5). The layout engine does not choose backends; it records effect requests faithfully in the Scene. The backend resolves fallback when it cannot fulfil a request.

0.7.1.2 Scene Determinism

The Scene is byte-deterministic: two executions of the layout pipeline on identical IR and theme produce bitwise-identical Scene records. The `scene_hash` (SHA-256 of the serialised element list and effect registry) is included in Scene metadata and is reproducible across runs.

Layered determinism contract. The determinism guarantee applies at three layers, each with different strength:

1. **Scene geometry—always byte-deterministic.** All coordinates, dimensions, label positions, and element ordering are bitwise-reproducible. This is guaranteed unconditionally by the pure pipeline contract in Section 0.5.1.
2. **Per-backend output—deterministic given a pinned backend version.** Each backend is a deterministic function of (Scene, backend-version, effect-seeds). Effect seeds for all procedural effects (NoiseTexture, CloudLayer) are derived from the scene_hash concatenated with the effect_id; no random state is consumed. The backend version must be recorded in the output’s metadata.
3. **Cross-backend pixel identity—explicitly not promised.** The SVG backend and the Raster backend, given the same Scene, are permitted and expected to produce visually different outputs. This is correct: the raster backend renders art effects that the SVG backend cannot. Two outputs from the same Scene but different backends are both correct renderings at different fidelity tiers. Cross-backend tests use per-backend golden images [10], not cross-backend pixel equality.

0.7.2 SVG Backend

Role: The clean, scalable, print and web default. SVG is the recommended first implementation target and the most widely compatible vector format. It is one beloved backend with a well-defined capability ceiling—not the universal root.

Benefits.

- **Deterministic at Tier 0 and Tier 1.** Scene geometry and Tier-0/1 effects (solid fills, linear and radial gradients, patterns, opacity) serialise to SVG with full byte-determinism.
- **Scalable.** Renders crisply at any resolution; A0 conference poster to screen thumbnail.
- **Inspectable.** XML text format; human-readable, version-controllable with meaningful diffs, and inspectable in browser developer tools.
- **Embeddable.** Inline HTML, Markdown image references (), $\text{\LaTeX}$ `\includegraphics`, or direct browser rendering.
- **Accessible.** Live `<text>` nodes support copy-paste, search, and screen readers.

Limitations and honest scope.

- SVG filters (feGaussianBlur, feDropShadow, feTurbulence) exist and can approximate glow, shadows, and noise. However, filter rendering is *not deterministic across SVG renderers*: the same filter markup produces visually different results in Inkscape, Chrome, Safari, Firefox, and resvg. For this reason, SVG filters are restricted to a “safe filter” set (Table 28) at Tier 2 with a determinism caveat; all Tier-3 effects require the Raster backend.
- Text rendering varies across SVG viewers when system fonts are used. Mitigation: embed font files or convert text to paths for archival exports.
- Art effects (glow/bloom, cloud textures, volumetric atmospheres, complex noise) are not achievable at Tier-3 fidelity in SVG. A Tier-3 theme targeting the SVG backend triggers effect fallback policies (Section 0.7.1).

- Complex diagrams with many activities produce proportionally large SVG files (no rasterisation compression).

Implementation strategy.

1. Consume the Scene element list in painting order. Serialise each primitive to its SVG equivalent: Rect→<rect>; Polygon→<polygon>; Line→<line>; Text→<text>; Path→<path>; Group→<g>; Image→<image>.
2. Apply SVG serialisation invariants:
 - All coordinates rounded to 2 decimal places (round-half-up).
 - All colour values as 6-digit hex (#RRGGBB); no named colours, no rgb() syntax.
 - All dimension values as integers for rectangles and polygons; 2-decimal for text positions.
 - Font references via <style> or @font-face in <defs>; no inline style= font declarations.
 - Pattern definitions in <defs>, referenced by stable ID (url(#tc-pattern-diagonal-hatch)).
 - Diamond vertices computed directly; no transform="rotate(...)" on individual elements.
 - Element IDs use prefix tc- followed by entity id.
 - Elements serialised in painting order: sections, gridlines, activity bars, milestones, annotations, labels.
 - Two SVG outputs from identical Scenes must be byte-identical.
3. For Tier-2 effects: emit the safe-filter SVG constructs from Table 28; record the backend version in a <desc> element and include a <!-- tc:determinism-caveat:filter --> comment in the SVG header.
4. For Tier-3 effects: apply the fallback policy from the Scene. If omit, skip the effect entirely. If embed-raster, delegate the affected layer group to the Raster backend and embed the result as <image>.

Text-as-paths option. An optional `-text-as-paths` flag converts all <text> elements to <path> elements using pre-computed glyph outlines from embedded font files. This eliminates font-dependent rendering differences at the cost of SVG searchability and file size. It is not the default.

PDF via SVG backend. For vector/consulting targets (Tier 0 and Tier 1), PDF is produced by converting the SVG backend output using a programmatic library. Recommended: `svg2pdf` (Rust, Apache 2.0) or `cairosvg` (Python, MIT). **Not** LibreOffice, headless Chrome, or Puppeteer (too heavyweight; introduces rendering non-determinism). Fonts are embedded as Type 1/CFF; PDF metadata is set deterministically from IR fields (/Title←metadata.title; /Author←metadata.author; /CreationDate←metadata.created, not the system timestamp). The PDF document ID is a SHA-256 hash of the SVG content bytes, truncated to 16 bytes.

0.7.3 Raster Backend — High-Fidelity Art Effects

Role: Full art effects—glow, bloom, soft shadows, volumetric cloud and atmosphere layers, noise textures, gradient meshes, blur, and per-pixel compositing. This is the differentiator for presentation showcase and keynote use cases.

Candidate implementations: Skia [11] (C++ with Rust bindings; used in Chrome, Flutter, and Android; permissive licence), headless HTML Canvas [14] (Node.js at a fixed, pinned version), or a WebGL-based headless pipeline [14] for GPU-accelerated art effects.

Benefits.

- Renders every Scene effect class natively: glow, bloom, drop shadows, soft shadows, cloud and atmosphere textures via noise/turbulence compositing, gradient meshes, per-pixel alpha compositing, and blur at any kernel radius.
- Output is a raster bitmap (PNG primary; also packaged as PDF for art-target print).
- Well-understood determinism control surfaces: seed pinning, fixed sampling parameters, pinned library version.

Determinism strategy for art effects. Art effects are inherently parameterised and continuous. Determinism is preserved by:

1. **Pinned renderer version.** The raster backend records its version (skia-v87, canvas-20.x, etc.) in the output metadata. A given version produces identical output for identical inputs; the version is an explicit parameter of the determinism contract.
2. **Fixed effect seeds.** All procedural effects (NoiseTexture, CloudLayer) derive seeds from `scene_hash` concatenated with the `effect_id`. No random state is consumed; seeds are deterministic functions of the Scene.
3. **Fixed sampling and rounding.** Anti-aliasing uses MSAA at 4× (fixed); sub-pixel AA is disabled (produces different results across LCD orientations). Font rendering uses pre-computed glyph outlines. PNG compression: deflate level 6 (not level 9: marginal savings, different byte sequences across zlib versions).
4. **Golden-image testing.** Reference PNGs are generated per backend version and stored as test fixtures [10]. CI compares new output to the golden image using a strict pixel-equality assertion (zero tolerance for geometry; configurable tolerance for anti-aliasing edges, default 0 pixels).

Limitations.

- Raster output is fixed resolution; upscaling produces pixelation. Default DPI: 150 for screen/web; 300 for print-quality.
- The backend introduces a runtime dependency (Skia or headless browser) that increases distribution size. Recommended: ship as an optional plugin activated for Tier-2 and Tier-3 themes only (Section 0.8).

- Cross-backend pixel identity with the SVG backend is explicitly *not* promised; see the layered determinism contract (§0.7.1.2).

PDF via Raster backend. For art-target PDF (Tier-3 themes), PDF is produced by embedding the raster output as a full-page image in a PDF/A envelope. Vector PDF (SVG backend) and art PDF (Raster backend) are distinct outputs selectable via a CLI flag (`-pdf-backend=vector|raster`).

0.7.4 PPTX — Native-Shape Backend

Format: Office Open XML Presentation (ISO/IEC 29500). **Library:** `python-pptx` (MIT licence) [34].

Architecture: native shapes with embedded raster art layers. The PPTX backend maps Scene primitives to native PowerPoint shapes wherever possible, enabling think-cell [40]-comparable editability, and falls back to embedded raster layers only for effects that have no native PowerPoint equivalent.

Native PowerPoint effects: exploiting what exists. PowerPoint’s shape model includes native effect properties that satisfy Tier-2 effect requests without embedded images. The PPTX backend exploits them as follows [6]:

- Scene Glow → PowerPoint shape Glow property (`a:glow` element), matching colour and scaled radius. Native; editable.
- Scene DropShadow → PowerPoint OuterShadow (`a:outerShdw`), matching offset, blur, and colour. Native; editable.
- Scene Bloom (heavy glow) → approximated as a larger, lower-opacity Glow; fallback policy approximate.
- Scene CloudLayer / NoiseTexture → no native PPTX equivalent; rasterised to PNG at 150 DPI and embedded as a transparent image shape above the native shapes. Policy: `embed-raster`. Native shapes beneath remain fully editable.

Implementation strategy.

1. **Slide dimensions.** Set to theme canvas width and computed height. Coordinate conversion: 1 logical pixel = 914.4 EMU (96 DPI logical); round-half-up. This conversion is deterministic.
2. **Canvas background.** Full-slide RECTANGLE in `theme.canvas.background_color`.
3. **Track headers.** MSO_AUTO_SHAPE_TYPE.RECTANGLE with TextFrame. Font from theme typography. Transparent or light fill per theme.
4. **Activity bars.** ROUNDED_RECTANGLE with corner radius from `theme.activity.bar_radius`. Fill from resolved status/category style. Diagonal-hatch: second semi-transparent overlay with MSO hatch pattern fill.
5. **Milestone diamonds.** MSO_AUTO_SHAPE_TYPE.DIAMOND shape.

6. **Labels.** `TextFrame` inside shapes; free-floating `TextBox` for labels placed outside bars.
7. **Axis.** Rectangle with tick-label `TextBox` elements; gridlines as thin `Line` shapes. Today marker as a vertical `Line` shape.
8. **Tier-2 effects.** Apply native PowerPoint effect properties as described above. Record the backend version in slide notes metadata.
9. **Tier-3 art layers.** Invoke the Raster backend for the affected Scene groups; embed results as transparent `<pic>` shapes above the native layer.
10. **Internal IDs.** Derive all `r:id` values from entity `id` fields (SHA-256 prefix), not auto-generated, for byte-stable output across runs.

Think-cell comparison. Think-cell achieves the highest-quality editable output: every element is a native PowerPoint shape at the XML level, fully fidelity in font, colour, and position [40]. Its approach is PowerPoint-locked and requires a proprietary COM add-in. Our target is to approach think-cell editability for the Tier-0/Tier-1 element vocabulary (bars, diamonds, text boxes) while retaining full pipeline determinism and a text-format IR. For Tier-2 themes (Executive, Product), the hybrid strategy produces editable bars and diamonds with native drop shadows and glow—no embedded rasters needed. For Tier-3 (Showcase), art layers are embedded rasters atop the fully editable geometric scaffold.

Limitations.

- Diagonal-hatch patterns require the `pptx.xml` XML interface, not a first-class python-pptx API.
- Gradient fades at approximate-date bar edges fall back to the image overlay strategy.
- PPTX files must be set to a fixed pixel dimension; viewers on different DPI settings scale the output proportionally.

0.7.5 HTML — Interactive Backend

Format: HTML5 with inline SVG (Tier 0/1 path) or with embedded raster frame (Tier 2/3 path).

Benefits.

- **Zero-install preview.** Any browser can display the file. Ideal for the VS Code extension live-preview pane and for sharing timelines via URL.
- **Interactive.** JavaScript hover handlers display `description` and `metadata` fields from the IR as tooltips, without altering geometry.
- **Accessible.** SVG `<text>` nodes remain live; labels are searchable and selectable. `<title>` elements on shapes are readable by assistive technology.
- **MCP-agent integration.** The MCP server (Section 0.9) can return an HTML URL as the rendering result, allowing agents to preview timelines in a browser tab without file-system access.

Two HTML rendering paths.

1. **SVG-backed HTML (Tier 0/1, default).** Take the SVG backend output as-is. Wrap in an HTML5 document shell: `<!DOCTYPE html>`, `<meta charset="utf-8">`, `<title>` from `metadata.title`, minimal CSS for page background and centre alignment. Inject the SVG inline (not as ``) to allow JavaScript access to SVG DOM nodes. Attach a lightweight JS event listener (no framework dependency) reading `data-description` and `data-id` attributes from SVG elements and displaying a tooltip `<div>` on hover. A `-no-js` flag produces geometry-identical HTML with the JavaScript block omitted.
2. **Canvas-backed HTML (Tier 2/3, art effects).** Embed a raster-backend PNG inside an HTML shell with an overlaid interactive layer. The PNG provides art-effect fidelity; the JS layer adds interactivity via a companion JSON map of entity bounding boxes (derived from the Scene) to description strings.

Limitations.

- The interactive state (hover, click, tooltip visibility) is not deterministic and is not part of the rendering contract. The underlying geometry is identical to the chosen backend output.
- Browser-to-browser rendering differences in SVG filter effects may produce minor visual variations; these are browser environment differences, not Timeline Compiler defects.
- Not suitable for archival or print without stripping the JavaScript layer.

0.7.6 Backend Capability and Fidelity-Tier Table

The following table summarises which effect classes each backend supports natively, approximates, or cannot perform. Fallback behaviour applies the policy declared in the Scene's `EffectDefinition`.

0.7.7 Output Priority Recommendation

The following priority ordering is recommended for implementation. The Scene architecture means the first investment is in the Scene model itself, which unlocks all subsequent backends.

1. **Scene / Render IR model + SVG backend** — *Build first, always.* The Scene is the correctness foundation. The SVG backend is the fastest path to a visible, inspectable output: every layout defect is immediately apparent in SVG before any rasterisation or conversion step can obscure it. It also serves as the primary debugging surface during development. Targets Tier-0 and Tier-1 themes (Minimal, Consulting, Release) with full byte-determinism.
2. **PNG via SVG backend** — *Immediate universal value.* Rasterise the SVG backend output at caller-specified DPI using a headless SVG rasteriser with embedded font support (`resvg` or `librsvg`). Unlocks the most common real-world use case:

pasting a timeline into a document, email, or Slack message. The effort is minimal once SVG is correct.

3. **PDF via SVG backend** — *Print and archival; consulting use case*. Convert the SVG backend output to PDF. The primary differentiator vs. Mermaid [20]; essential for consulting deliverables. The SVG-to-PDF conversion path is well-understood and deterministic. Priority 3 because font subsetting and PDF metadata handling are needed but engineering effort is low relative to PPTX.
4. **PPTX native-shape backend** — *Highest editability value*. Hybrid native-shapes-plus-raster-art-layer strategy. Approaches think-cell editability for Tier-0/1 vocabulary; exploits native PowerPoint effects for Tier-2. The most complex target; requires stable Scene geometry from earlier priorities.
5. **Raster backend (Skia or Canvas)** — *Art-effect differentiator*. The enabling technology for Showcase and Keynote themes. Glow, bloom, cloud textures, volumetric atmospheres. Ship as an optional plugin or separate distribution target to keep the core distribution lean.
6. **HTML / interactive** — *Developer experience and agent integration*. Nearly free once the SVG backend is correct (SVG-backed path) or the Raster backend is available (canvas-backed path). Essential for the VS Code extension live preview and the MCP agent integration path described in Section 0.9.

0.7.8 Format Comparison Summary

Determinism across backends. The Scene geometry is byte-deterministic unconditionally: the layout pipeline contract (Section 0.5.1) guarantees this regardless of which backend is invoked. Each backend is deterministic given a pinned backend version and fixed effect seeds; backend version is recorded in output metadata to make this reproducible. SVG and PDF (via SVG backend) satisfy full byte-determinism for Tier-0/1. PNG satisfies byte-determinism per pinned backend version. PPTX satisfies geometric determinism; internal `r:id` values are derived from entity `id` fields rather than auto-generated. HTML satisfies geometry determinism; its interactive JavaScript layer is explicitly excluded from the determinism contract. Cross-backend pixel identity is explicitly *not* promised and is not a correctness requirement: it is expected and correct for the SVG backend and the Raster backend to produce visually different outputs from the same Scene when the theme requests effects beyond SVG’s deterministic range. This is a feature of the architecture, not a defect.

0.8 Distribution Strategy

This section evaluates packaging and distribution models for Timeline Compiler, recommending an architecture that serves human authors, AI agents, and embedding use cases.

0.8.1 Distribution Requirements

Timeline Compiler must be accessible to multiple audiences:

CLI Users

— Developers, DevOps engineers, and technical users who invoke rendering from terminal or scripts.

Application Embedders

— Tools that integrate Timeline Compiler as a library (documentation generators, planning tools, dashboards).

AI Agents

— LLM-based agents that need to render timelines as a tool capability.

Non-Technical Users

— Product managers and executives who want live preview while authoring (via IDE integration).

CI/CD Pipelines

— Automated rendering in build processes.

Key requirements:

- Zero-friction installation for common platforms
- Embeddable as a library
- Invocable by AI agents (MCP server or similar)
- Deterministic output regardless of invocation method
- Self-contained — minimal or no external dependencies at runtime

0.8.2 Packaging Options

0.8.2.1 Core Library

Concept: A core rendering library with programmatic API, distributed as a package in the target ecosystem's package manager.

Options:

@timeline-compiler/core (npm)

Node.js/TypeScript library. Pros: large ecosystem, easy embedding in JS/TS tools, npm distribution is well-understood. Cons: Node runtime dependency, potentially slow for large renders, binary output (PDF, PNG) may require native dependencies.

Go Module

Single-binary compilation, embeddable via Go's module system. Pros: no runtime dependency, fast, easy cross-compilation. Cons: embedding in non-Go tools requires FFI or subprocess.

Rust Crate

Similar to Go — single binary, high performance. Smaller ecosystem adoption than Go for CLI tools. WASM compilation possible.

Python Package (pip)

Broad adoption in data/ML communities. Cons: runtime dependency, packaging complexity, slower execution.

Recommendation: Implementation language is not decided in this spec, but the core library should expose:

- A stable programmatic API (`render(ir, theme, options) -> output`)
- Schema validation functions
- Theme loading and composition

0.8.2.2 Command-Line Interface

Concept: A CLI tool for rendering from terminal.

```
1 # Basic rendering
2 timeline render roadmap.yaml -o roadmap.pdf
3
4 # With theme
5 timeline render roadmap.yaml --theme corporate-blue -o roadmap.svg
6
7 # Validate only
8 timeline validate roadmap.yaml
9
10 # Watch mode (optional)
11 timeline watch roadmap.yaml -o roadmap.svg
```

Listing 34: CLI usage examples

Distribution:

npm global install

`npm install -g @timeline-compiler/cli` — Easy for Node.js users, but requires Node runtime.

Standalone Binary

Distributed via GitHub Releases, Homebrew, apt, etc. No runtime dependency. Cross-platform (macOS, Linux, Windows).

Docker Image

`docker run timeline-compiler render ...` — Isolated environment, good for CI/CD.

npx

`npx @timeline-compiler/cli render ...` — Zero-install execution for one-off use.

Recommendation: Provide both:

- npm package for Node.js ecosystems
- Standalone binaries for users without Node.js

0.8.2.3 VS Code Extension

Concept: IDE integration providing:

- Syntax highlighting for `.timeline.yaml` files
- Live preview panel (re-renders on save or keystroke)

- Export commands (PDF, PNG, SVG, PPTX)
- Schema validation and error highlighting
- Theme selection

Value: Dramatically improves authoring experience for non-CLI users. Live preview is critical for iterative refinement.

Implementation: The extension embeds the core library (via WASM, bundled Node module, or subprocess call to CLI).

Distribution: VS Code Marketplace.

Recommendation: High-priority for adoption. Authors should see their timeline update as they type.

0.8.2.4 MCP Server (Model Context Protocol)

Concept: A server exposing Timeline Compiler as a tool for AI agents.

```

1 tools:
2   - name: render_timeline
3     description: "Renders a timeline IR to visual output"
4     inputSchema:
5       type: object
6       properties:
7         ir:
8           type: object
9           description: "Timeline IR conforming to schema"
10        theme:
11          type: string
12          description: "Theme name or inline theme object"
13        format:
14          type: string
15          enum: [svg, png, pdf]
16      returns:
17        type: string
18        description: "Base64-encoded output or file path"

```

Listing 35: MCP tool definition (conceptual)

Value: Agents can render timelines as a native capability. The agent generates IR, invokes the tool, receives visual output.

Deployment:

- Local MCP server (runs alongside the agent environment)
- Hosted MCP endpoint (cloud-deployed rendering service)

Recommendation: Essential for the agent use case. MCP server should be a first-class distribution target, not an afterthought.

0.8.2.5 NPM Package for Embedding

Concept: The core library packaged for use in JavaScript/TypeScript applications.


```
1 import { render, loadTheme, validate } from
  '@timeline-compiler/core';
2
3 const ir = loadYAML('roadmap.yaml');
4 const errors = validate(ir);
5 if (errors.length > 0) throw new ValidationError(errors);
6
7 const theme = await loadTheme('corporate-blue');
8 const svg = await render(ir, theme, { format: 'svg' });
9
10 res.setHeader('Content-Type', 'image/svg+xml');
11 res.send(svg);
```

Listing 36: Embedding in a Node.js application

Use Cases:

- Documentation sites that render timelines dynamically
- Internal tools with timeline visualization
- SaaS products embedding Timeline Compiler

Recommendation: Core distribution path for the JavaScript ecosystem.

0.8.2.6 Standalone Go Binary

Concept: If the implementation uses Go, the CLI is a single statically-linked binary.

Pros:

- No runtime dependencies
- Fast startup
- Easy cross-compilation (macOS, Linux, Windows, ARM)
- Simple distribution (download and run)

Cons:

- Embedding in non-Go applications requires subprocess or FFI
- Community contribution may be lower than TypeScript (smaller Go population among target users)

Recommendation: Attractive for CLI distribution. If TypeScript is the core language, a Go wrapper or WASM compilation may achieve similar benefits.

0.8.3 Implementation Language Trade-offs

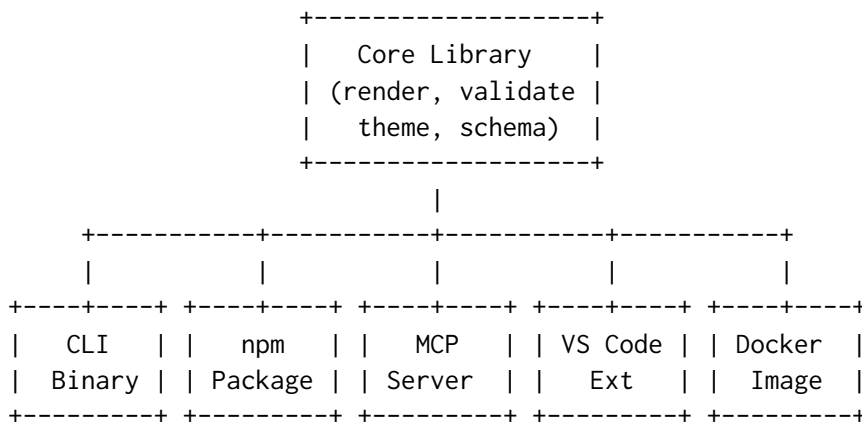
The distribution strategy depends on implementation language. Key considerations:

Recommendation: This spec does not mandate implementation language. However, the distribution architecture should support:

1. A core library embeddable in JavaScript/TypeScript contexts (npm)
2. A standalone CLI that does not require a runtime install
3. An MCP server for agent integration
4. A VS Code extension with live preview

A TypeScript core with WASM or native compilation for CLI may satisfy all requirements. Alternatively, a Go core with a thin Node wrapper for npm distribution is viable.

0.8.4 Recommended Architecture



Core Library: Single implementation of IR parsing, validation, theme loading, layout, and rendering. All distribution targets wrap this core.

CLI Binary: Wraps core library. Distributed via:

- GitHub Releases (tarballs for each platform)
- Homebrew (brew install timeline-compiler)
- npm global install (if Node.js-based)
- curl one-liner for quick install

npm Package: The core library published to npm for JavaScript/TypeScript embedding.

MCP Server: Wraps core library, exposes `render_timeline` and `validate_timeline` tools. Distributed as:

- npm package (for local MCP server)
- Docker image (for hosted deployment)

VS Code Extension: Embeds core library (via bundled WASM or Node module). Published to VS Code Marketplace.

Docker Image: Contains CLI binary. For CI/CD pipelines and isolated execution.

0.8.5 Versioning and Compatibility

Semantic Versioning: All packages follow semver. Breaking changes to IR schema or theme format require major version bump.

Schema Versioning: The IR declares its schema version (`version: "1.0"`). The compiler reports errors if the IR version is unsupported.

Theme Versioning: Themes declare compatibility with IR/compiler versions.

CLI Version Reporting: `timeline -version` reports compiler version, supported IR versions, and supported output formats.

0.8.6 Distribution Priorities

For MVP, distribution priority order:

1. **CLI Binary** — Core functionality, scriptable, CI/CD compatible
2. **npm Package** — Embedding in JS/TS tools
3. **VS Code Extension** — Authoring experience
4. **MCP Server** — Agent integration
5. **Docker Image** — Isolated execution

Post-MVP:

- Homebrew formula
- apt/yum packages
- GitHub Action for rendering in workflows
- Hosted rendering API (optional commercial offering)

0.9 Agent Integration

AI agents are first-class users of Timeline Compiler. They are not an afterthought wired to a human-facing CLI; they are a primary consumer of the IR, the primary beneficiary of a published JSON Schema, and the primary audience for the MCP server surface. This section designs the full agent integration path: from raw inputs through ingestion to a valid IR, through validation and repair, and through round-trip editing without loss of provenance.

The fundamental unit of agent work is the *ingestion task*: given a **data source** (work items, prose, structured export) and a **natural-language prompt**, produce a valid Timeline IR. The prompt is a first-class input—it determines what is selected, how it is framed, which entities are promoted to milestones, and which are dropped entirely. An agent that ignores the prompt and imports everything it can find is not doing its job.

0.9.1 The Ingestion Contract

The ingestion contract defines what the ingestion path is permitted to do with the inputs it receives. Table 33 captures the four categories of ingestion decision. Violations of this contract—particularly silent data loss or silent inference—are the most common source of incorrect or misleading timelines.

0.9.1.1 The Prompt as First-Class Input

The prompt directs every non-trivial ingestion decision. Table 34 lists what the prompt controls and what the ingestion layer must extract from it before touching the source data.

0.9.1.2 What Ingestion May Not Do

- **Must not compute dates.** If a source item has no date, it must use `tbd`, not invent one. Timeline Grammar does not compute schedules (see §0.3).
- **Must not collapse ambiguous status silently.** An ADO work item in state “On Hold” that has no clear IR equivalent must log a rejection warning, not silently map to `planned`.
- **Must not import the whole backlog.** The prompt defines the editorial scope. Importing everything that exists and hoping the renderer trims it is a contract violation.
- **Must not generate non-stable IDs.** IDs derived from sequential numbers (`act-1`, `act-2`) are permitted only as a last resort. IDs should be derived from the item’s title slug for git-friendliness (see §0.9.7).

0.9.2 Ingestion Workflows

The following subsections describe four concrete ingestion flows, each with a source excerpt, a prompt, the resulting IR YAML, and an explicit account of what the prompt selected, inferred, and rejected.

0.9.2.1 Azure DevOps Work Items

Source format. The ADO Work Items REST API [22] returns JSON objects. The fields relevant to timeline ingestion are listed in Table 35.

Source excerpt.

```

1 # ADO API response (workItems array, fields selected)
2 - id: 1024
3   fields:
4     System.Title: "API Gateway Migration"
5     System.State: "Active"
6     System.WorkItemType: "Feature"
7     System.IterationPath: "Platform\\2026\\Q2"
8     System.AreaPath: "Platform Team"
9 - id: 1025

```

```

10  fields:
11    System.Title: "Mobile App 3.0 GA"
12    System.State: "New"
13    System.WorkItemType: "Milestone"
14    System.AreaPath: "Mobile Team"
15    Microsoft.VSTS.Scheduling.TargetDate: "2026-09-15"
16  - id: 1031
17    fields:
18      System.Title: "Fix login timeout bug"
19      System.State: "Active"
20      System.WorkItemType: "Bug"
21      System.AreaPath: "Platform Team"

```

Listing 37: ADO work item JSON (abridged)

Prompt. *"Show the Q1-Q3 2026 Platform and Mobile roadmap. Include features and milestones only. Group by team. Drop bugs and tasks."*

Resulting IR.

```

1  version: "1.0"
2  metadata:
3    title: Platform & Mobile Roadmap
4    subtitle: Q1-Q3 2026
5    time_range:
6      start: 2026-Q1
7      end: 2026-Q3
8    axis_unit: quarter    # inferred: 3-quarter window -> quarter
9                           granularity
10 tracks:
11   - id: platform
12     label: Platform Team
13   - id: mobile
14     label: Mobile Team
15
16 activities:
17   - id: api-gateway-migration
18     label: API Gateway Migration
19     track: platform
20     span: 2026-Q2        # inferred from IterationPath
21                           "Platform\2026\Q2"
22     status: in-progress  # mapped from ADO state "Active"
23     category: feature
24     metadata:
25       source: ado
26       ado_id: 1024
27       ado_iteration: "Platform\\2026\\Q2"
28
29 milestones:
30   - id: mobile-app-3-ga
31     label: Mobile App 3.0 GA
32     date: 2026-09-15    # from TargetDate field
33     track: mobile
34     status: planned      # mapped from ADO state "New"
35     metadata:
36       source: ado
37       ado_id: 1025

```

Listing 38: IR produced from ADO work items
What the prompt did.

- **Selected:** Items 1024 (Feature) and 1025 (Milestone). Item 1031 (Bug) was rejected because the prompt said “features and milestones only”.
- **Framed:** `time_range` set to Q1–Q3 2026 from prompt instruction. Item 1025’s target date (2026-09-15) falls within Q3, so it is included.
- **Inferred:** Tracks derived from `AreaPath` leaf segments. `axis_unit` inferred as quarter from the three-quarter window.
- **Dropped:** Bug work item 1031 (not in filter). Had the time range excluded any item, it would be dropped with a logged rejection.

0.9.2.2 Natural Language Prose

When the source is free-form text—a planning document, a briefing note, an email thread—the agent must extract temporal structure from prose. This is the most ambiguity-prone ingestion path; the contract requires explicit handling of every point of uncertainty.

Source text.

```

1 We are planning to migrate the data warehouse in the first half of
   2026.
2 The new customer portal is targeted for Q3. Both initiatives are
   owned
3 by the Engineering org. An executive review is scheduled for 15 July
   2026.
4 The legacy decommission date is TBD, pending the warehouse cutover.
```

Listing 39: Natural language source

Prompt. *“Show this as a two-track roadmap: Infrastructure and Product. Mark the executive review as a milestone. Include the decommission even though its date is not fixed.”*

Resulting IR.

```

1 version: "1.0"
2 metadata:
3   title: 2026 Engineering Roadmap
4   time_range:
5     start: 2026-H1
6     end: 2026-Q3
7   axis_unit: quarter # inferred: H1+Q3 window, quarterly detail
8
9 tracks:
10  - id: infrastructure
11    label: Infrastructure
12  - id: product
13    label: Product
14
15 activities:
16  - id: data-warehouse-migration
```

```

17   label: Data Warehouse Migration
18   track: infrastructure
19   span: 2026-H1
20   status: planned
21
22   - id: customer-portal
23     label: New Customer Portal
24     track: product
25     span: 2026-Q3
26     status: planned
27
28   - id: legacy-decommission
29     label: Legacy Decommission
30     track: infrastructure
31     start: tbd          # explicit: date not yet determined
32     status: planned
33
34 milestones:
35   - id: executive-review
36     label: Executive Review
37     date: 2026-07-15
38     status: planned

```

Listing 40: IR produced from natural language prose

What the prompt did.

- **Structured:** Track assignments (Infrastructure vs. Product) came from the prompt. The source text only named “Engineering org”.
- **Promoted:** The executive review was elevated to a milestone (not an activity) by explicit prompt instruction.
- **Included with uncertainty:** Legacy decommission included despite no fixed date; `start: tbd` preserves the fact on the timeline without inventing a date.
- **Inferred:** `time_range` derived from the earliest (2026-H1 start) and latest (2026-07-15) concrete dates. `axis_unit` set to `quarter` as the coarsest granularity that fits the items.

Ambiguity resolution. “First half of 2026” is unambiguously 2026-H1 (ISO half, per the date model). “Q3” without a year is inferred as 2026-Q3 from context (all other dates are 2026). If the year were ambiguous, the agent must surface a warning rather than guess.

0.9.2.3 GitHub Projects

GitHub Projects (ProjectV2) [8, 9] supports custom field types that carry roadmap-relevant data. The ingestion mapping is summarised in Table 37.

The ITERATION field type is the most useful for quarterly roadmaps: iteration titles like “2026 Q2” map directly to `span: 2026-Q2`, making the ingestion path from GitHub Projects to a quarterly roadmap nearly lossless.

Gap note. GitHub Projects does not have a first-class concept analogous to “work item type” (Feature vs. Bug vs. Task). Filtering by type requires label or custom field

conventions, which vary per team. The prompt must specify the filter criterion explicitly.

0.9.2.4 Mermaid Timeline Syntax

Mermaid's timeline diagram [18, 20] is the most common structured timeline format already embedded in team documentation. Its syntax maps cleanly to IR sections and activities.

Source.

```

1 timeline
2   title Platform Evolution 2026
3   section H1 2026
4     API Gateway : Q1 : Migration complete
5     Auth Refresh : Q2 : New OAuth2 provider
6   section H2 2026
7     Mobile 3.0 GA : Q3
8     Legacy Cutover : Q4 : Final decommission

```

Listing 41: Mermaid timeline source

Resulting IR.

```

1 version: "1.0"
2 metadata:
3   title: Platform Evolution 2026
4   time_range:
5     start: 2026-Q1
6     end: 2026-Q4
7   axis_unit: quarter
8
9 tracks:
10  - id: platform
11    label: Platform          # default single-track when Mermaid has no
12    swimlanes
13
14 sections:
15  - id: h1-2026
16    label: H1 2026
17    time_range:
18      start: 2026-Q1
19      end: 2026-Q2
20
21  - id: h2-2026
22    label: H2 2026
23    time_range:
24      start: 2026-Q3
25      end: 2026-Q4
26
27 activities:
28  - id: api-gateway
29    label: API Gateway
30    track: platform
31    span: 2026-Q1
32    description: "Migration complete"
33    status: planned

```



```

34 - id: auth-refresh
35   label: Auth Refresh
36   track: platform
37   span: 2026-Q2
38   description: "New OAuth2 provider"
39   status: planned
40
41 - id: mobile-3-ga
42   label: Mobile 3.0 GA
43   track: platform
44   span: 2026-Q3
45   status: planned
46
47 - id: legacy-cutover
48   label: Legacy Cutover
49   track: platform
50   span: 2026-Q4
51   description: "Final decommission"
52   status: planned

```

Listing 42: IR produced from Mermaid timeline syntax

Mapping notes. Mermaid timeline does not have a swimlane concept; all events are rendered in a single horizontal flow grouped by section. When promoting to IR, a default single track is created unless the prompt specifies a multi-track structure. Mermaid's colon-separated description field maps to `description`. Because Mermaid `gantt` [19] uses a different syntax (task dependencies, progress), Mermaid `gantt` ingestion requires a separate mapping pass; that mapping is out of scope for the initial IR but is an extension point (see §0.3).

0.9.3 Reliable IR Generation by Agents

LLM-based agents generate IR reliably when three conditions hold: (1) the target schema is machine-readable and published, (2) the generation call is constrained to valid output shapes, and (3) the IR's structural choices minimise the surface area for generation errors.

0.9.3.1 JSON Schema Publication

The IR schema must be published as a JSON Schema document [28]. This is a binding requirement established by David's research constraints (Constraint 7 in §0.10): without a machine-readable schema, agent generation is unreliable. The schema is a first-class deliverable of this project, versioned alongside the spec.

The top-level JSON Schema structure mirrors the IR document structure directly:

```

1 # $schema: https://json-schema.org/draft/2020-12/schema
2 # $id: https://timeline-compiler.dev/schema/v1/timeline.json
3 title: Timeline IR
4 type: object
5 required: [version, metadata, tracks, activities]
6 properties:
7   version:

```

```

8     type: string
9     description: "Specification version, e.g. \"1.0\""
10
11 metadata:
12     type: object
13     required: [title, time_range]
14     properties:
15         title: { type: string }
16         subtitle: { type: string }
17         time_range:
18             type: object
19             required: [start]
20             properties:
21                 start: { "$ref": "#/$defs/Date" }
22                 end: { "$ref": "#/$defs/Date" }
23         axis_unit: { "$ref": "#/$defs/AxisUnit" }
24         theme: { type: string }
25         locale: { type: string }
26
27 tracks:
28     type: array
29     minItems: 1
30     items: { "$ref": "#/$defs/Track" }
31
32 activities:
33     type: array
34     items: { "$ref": "#/$defs/Activity" }
35
36 milestones:
37     type: array
38     items: { "$ref": "#/$defs/Milestone" }
39
40 # groups, annotations, sections, legend all optional arrays
41
42 $defs:
43     ID:
44         type: string
45         pattern: "^[a-z][a-z0-9-]*$"
46
47     Date:
48         type: string
49         description: "ISO date, quarter (2026-Q2), half (2026-H1),
50             relative (+3m),
51             symbolic (now), or uncertain token (tbd, ongoing,
52             unknown,
53             or tilde-prefix ~2026-Q2)"
54
55     AxisUnit:
56         type: string
57         enum: [day, week, month, quarter, half, year]
58
59     Status:
60         type: string
61         enum: [planned, in-progress, done, at-risk, blocked, cancelled,
62             tentative]
63
64     Track:
65         type: object

```

```

63   required: [id, label]
64   properties:
65     id:    { "$ref": "#/$defs/ID" }
66     label: { type: string }
67     color: { type: string }
68     order: { type: integer, minimum: 0 }
69
70   Activity:
71     type: object
72     required: [id, label, track]
73     oneOf:
74       - required: [start]      # start + optional end
75       - required: [span]      # span shorthand
76     properties:
77       id:    { "$ref": "#/$defs/ID" }
78       label: { type: string }
79       track: { type: string } # bare string ref to Track.id
80       start: { "$ref": "#/$defs/Date" }
81       end:   { "$ref": "#/$defs/Date" }
82       span:  { "$ref": "#/$defs/Date" }
83       status: { "$ref": "#/$defs/Status", default: planned }
84       progress: { type: number, minimum: 0, maximum: 1 }
85       category: { type: string }
86       group:   { type: string }
87       description: { type: string }
88       metadata: { type: object }
89
90   Milestone:
91     type: object
92     required: [id, label, date]
93     properties:
94       id:    { "$ref": "#/$defs/ID" }
95       label: { type: string }
96       date:  { "$ref": "#/$defs/Date" }
97       track: { type: string }
98       status: { "$ref": "#/$defs/Status", default: planned }
99       category: { type: string }
100      icon:   { type: string }
101      description: { type: string }
102      metadata: { type: object }

```

Listing 43: Abbreviated JSON Schema for the IR (YAML representation)

The schema is published at a versioned URL. Minor version bumps to the IR may add optional fields; the schema handles this via `additionalProperties: true` for metadata maps and by allowing unknown top-level fields on minor revisions.

New cite key needed. The JSON Schema specification itself does not have a cite key in `references.bib`. A key `json-schema2020` pointing to the IETF JSON Schema draft (2020-12) should be added by David.

0.9.3.2 Structured Output Constraints

Platforms that support constrained generation [28] should use the published JSON Schema as the output schema for the IR generation call. This eliminates entire classes

of generation errors:

- Required fields can never be missing when the schema enforces **required**.
- ID format is enforced by the **pattern** constraint on the ID definition, preventing spaces, uppercase, or other invalid characters.
- Status values are constrained to the enum; the model cannot hallucinate “completed” or “active”.
- References are just bare strings—the model emits **track**: “platform”, not a nested object, which is trivially correct.

When structured output is not available, the agent prompt must include the JSON Schema inline and instruct the model to validate its own output before returning it.

0.9.3.3 IR Design Choices That Help Agents

Several deliberate IR design choices reduce the error rate for LLM-generated documents:

Optional fields default to safe values.

status defaults to **planned**; **progress** defaults to **null**. An agent that omits these fields produces a valid document. Agents only need to emit fields they have evidence for.

span shorthand reduces reasoning load.

Instead of requiring an agent to compute both a **start** and **end** date for a quarter-length activity, it can emit **span**: 2026-Q2 and let the compiler expand it. This avoids off-by-one boundary errors (is Q2 end June 30 or July 1?).

References are bare strings.

track: **platform** is far easier for an LLM to emit correctly than a nested reference object. The schema context (field name) determines the target type; no wrapper object is needed.

IDs are kebab-case slugs.

Agents derive IDs from labels: “API Gateway Migration” → **api-gateway-migration**. This is a simple and reliable transformation. UUIDs would be safe but opaque; sequential numbers would collide with re-generation.

No dependency scheduling.

Since the IR is a Timeline Grammar (not a Task Scheduling Grammar), there is no dependency resolution. Each activity is independently valid. Agents do not need to compute a topological order or check resource constraints.

0.9.4 Validation Pipeline

Validation is layered. Each layer is a gate: failure at an earlier layer produces errors that block later layers. This design prevents cryptic failures (e.g., a reference-resolution error caused by a silently malformed ID).

Layer 4 dependency. Render-readiness validation—checking that the IR can produce a legible visual output—is defined and owned by Barbara in §0.5. This includes checks such as overlapping activities on the same track that cannot be visually separated, labels

that exceed track width at the chosen axis unit, and milestone dates that fall outside the document's `time_range`. The validation pipeline calls Barbara's render-readiness checks as Layer 4; this section does not duplicate them.

0.9.4.1 Errors vs. Warnings

Error

Prevents rendering. The document is invalid and the renderer must refuse to proceed. Errors come from Layers 1–3 and from Layer 4 severity “fatal”. The validator must return *all* errors in a single pass, not just the first.

Warning

The document is valid but the output may be suboptimal. Warnings come from Layer 4 (advisory) and Layer 5. Renderers proceed and may annotate the output (e.g., a clipped activity) or log the warning and continue.

0.9.5 Error Handling and Agent Repair Cycle

0.9.5.1 Error Message Format

Every validation error is **path-anchored**: it identifies the exact location in the document, the nature of the violation, and a suggested fix. This design enables an agent to mechanically apply repairs without re-reading the whole document.

The error message format is:

```

1 ERROR <path>: <description>
2     Expected: <constraint>
3     Found:    <actual value>
4     Fix:      <suggested correction>

```

Listing 44: Validation error message format

Concrete error examples:

```

1 # Missing required field
2 ERROR activities[0].id: required field 'id' is missing
3     Expected: string matching ^[a-z][a-z0-9-]*$
4     Found:    (absent)
5     Fix:      add id: api-gateway-migration
6
7 # Invalid ID format
8 ERROR activities[1].id: "API Migration" does not match
9     ^[a-z][a-z0-9-]*$
10    Expected: kebab-case slug
11    Found:    "API Migration"
12    Fix:      id: api-migration
13
14 # Unresolved track reference
15 ERROR activities[2].track: "eng" references a non-existent track
16    Expected: one of [platform, mobile, data]
17    Found:    "eng"
18    Fix:      track: platform (or add a track with id: eng)

```

```

19 # Invalid status value
20 ERROR   activities[3].status: "active" is not a valid Status
21         Expected: one of [planned, in-progress, done, at-risk,
22                        blocked, cancelled, tentative]
23         Found:      "active"
24         Fix:        status: in-progress
25
26 # Date ordering violation
27 ERROR   activities[4]: end precedes start
28         Expected: end >= start
29         Found:      start: 2026-Q3, end: 2026-Q1
30         Fix:        swap dates, or use span: 2026-Q3 if this is
31                        a single-quarter activity
32
33 # Progress out of range
34 ERROR   activities[5].progress: 1.4 is outside [0, 1]
35         Expected: number in [0, 1]
36         Found:      1.4
37         Fix:        progress: 1.0
38
39 # Duplicate ID
40 ERROR   milestones[1].id: "platform-launch" is already used by
         activities[2].id
41         Expected: globally unique identifier
42         Found:      duplicate
43         Fix:        id: platform-launch-ms (or another unique slug)
44
45 # Missing date source (neither start nor span present)
46 ERROR   activities[6]: no temporal anchor defined
47         Expected: exactly one of: (start [+ optional end]) or span
48         Found:      neither start nor span present
49         Fix:        add span: 2026-Q2 or start: 2026-04-01

```

Listing 45: Example validation errors with suggested fixes

0.9.5.2 Agent Repair Cycle

The validation-error-repair loop is the core of reliable agent IR generation. The cycle is:

1. **Generate.** Agent generates an IR document, optionally using structured output constraints (§0.9.3.2).
2. **Validate.** The IR is submitted to the `validate_timeline` MCP tool (§0.9.6.1). All layers are run; all errors collected.
3. **Report.** Errors are returned to the agent as a structured list with path, description, and suggested fix.
4. **Repair.** The agent applies targeted fixes. Because errors are path-anchored and include suggested fixes, repairs are surgical: the agent patches `activities[2].track` rather than regenerating the entire document.
5. **Re-validate.** The patched document is re-submitted. This loop repeats until validation passes with no errors (warnings may remain).
6. **Render.** Once validation passes, the agent calls `render_timeline` to produce output.

In practice, structured output constraints eliminate most Layer 2 errors on the first generation pass. Layers 3 (referential integrity, date ordering) are the most common source of residual errors for agents generating multi-track documents.

0.9.6 MCP Server

The MCP server [4] exposes Timeline Compiler as a tool-callable service for AI agents. It is a first-class distribution target, not an afterthought (see the distribution architecture in §0.8). The server hosts four tools.

0.9.6.1 Tool Contracts

validate_timeline Runs the full validation pipeline (Layers 1–5) against a provided IR document. Returns all errors and warnings in structured form.

```

1 tool: validate_timeline
2 description: >
3   Validate a Timeline IR document. Runs syntactic, schema,
4     well-formedness,
5     render-readiness, and semantic advisory checks. Returns structured
6     errors
7     and warnings with path anchors and suggested fixes.
8
9 input:
10  ir:
11    type: object | string    # IR as parsed object or raw YAML/JSON
12    string
13    required: true
14  stop_at_layer:
15    type: integer            # optional; 1-5; default: 5 (run all
16    layers)
17    required: false
18
19 output:
20  valid: boolean            # true iff zero errors (warnings do not
21  block)
22  errors:
23    - path: string          # e.g. "activities[2].track"
24      code: string          # machine-readable error code, e.g.
25        UNRESOLVED_REF
26      message: string        # human-readable description
27      fix: string            # suggested correction
28  warnings:
29    - path: string
30      code: string
31      message: string
32  layers_run: integer        # how many layers were executed before
33  stopping

```

Listing 46: validate_timeline tool contract

render_timeline Renders a valid Timeline IR to a visual output format. Returns the rendered output as a base64-encoded string or a file path (for local MCP deployments).

```

1 tool: render_timeline
2 description: >
3   Render a Timeline IR document to SVG, PNG, PDF, or PPTX. The IR
4   must
5   be valid (validate_timeline returns valid: true) before rendering.
6   Rendering is deterministic: identical IR + theme + format always
7   produces bit-identical output.
8 input:
9   ir:
10    type: object | string    # IR as parsed object or raw YAML/JSON
11    string
12    required: true
13  format:
14    type: string
15    enum: [svg, png, pdf, pptx]
16    default: svg
17    required: false
18  theme:
19    type: string              # theme identifier, e.g. "default",
20    "corporate"
21    default: "default"
22    required: false
23  width:
24    type: integer             # output width in pixels (PNG/PDF only)
25    required: false
26 output:
27   format: string             # echo of requested format
28   data: string               # base64-encoded output bytes
29   mime_type: string          # e.g. "image/svg+xml"
30   warnings:                  # render-layer warnings (does not block
31     output)
32   - message: string

```

Listing 47: render_timeline tool contract

describe_schema Returns the full JSON Schema for the IR and field-level documentation. Agents call this to bootstrap IR generation or to recover from schema errors.

```

1 tool: describe_schema
2 description: >
3   Return the JSON Schema for the Timeline IR, plus human-readable
4   documentation for each field. Use this to understand what fields
5   are
6   required, their types, allowed values, and defaults.
7 input:
8   version:
9     type: string             # IR version to describe; default: latest
10    required: false
11  entity:
12    type: string              # optional: limit to one entity, e.g.
13    "Activity"
14    required: false

```



```

15 output:
16   version:      string      # IR version described
17   schema:       object      # full JSON Schema document
18   docs:
19     - entity:    string      # entity name
20       fields:
21         - name:      string
22           type:       string
23           required:   boolean
24           default:    string | null
25           description: string
26           example:    string

```

Listing 48: describe_schema tool contract

suggest_time_range A convenience tool for agents that need to derive a sensible `time_range` and `axis_unit` from a collection of candidate activities before composing the full document. This is especially useful when ingesting data from sources that lack explicit time windows.

```

1 tool: suggest_time_range
2 description: >
3   Given a list of activity date hints (start, end, span values),
4   suggest
5   appropriate time_range and axis_unit values for the IR metadata.
6   Does not
7   require a complete IR document.
8
9 input:
10  dates:
11    type: array          # list of date strings, e.g. ["2026-Q1",
12      "2026-09-15"]
13    items: { type: string }
14    required: true
15  prefer_unit:
16    type: string          # optional hint: "quarter", "month", etc.
17    required: false
18
19 output:
20   time_range:
21     start: string
22     end: string
23   axis_unit: string      # recommended AxisUnit
24   rationale: string      # explanation of the choice

```

Listing 49: suggest_time_range tool contract

0.9.6.2 MCP Server Deployment

The MCP server is deployed in two modes:

Local server

Runs as a subprocess alongside the agent environment. Started via the CLI (`timeline mcp-server -port 3000`) or as an npm/pip package. Suitable for development, CI, and agent runtimes on the same host.

Hosted endpoint

A cloud-deployed instance of the MCP server. Suitable for cloud-native agent runtimes where a local subprocess is impractical.

Both modes expose identical tool contracts. The local server operates on the local filesystem (output file paths are local); the hosted server returns base64-encoded output in all cases.

0.9.7 Round-Trip and Iterative Editing

A Timeline IR document lives in version control alongside the source data it was derived from. Humans and agents edit it directly. Re-syncing against an updated source must not silently overwrite editorial decisions made in the IR. This section designs the mechanisms that make round-trip editing safe.

0.9.7.1 Source Provenance via Metadata

Every entity produced by an ingestion workflow includes a `metadata` block that records the source of record:

```

1 activities:
2   - id: api-gateway-migration
3     label: API Gateway Migration
4     track: platform
5     span: 2026-Q2
6     status: in-progress
7     metadata:
8       source: ado
9       ado_id: 1024
10      ado_revision: 42           # ADO ETag for change detection
11      ado_area: "Platform Team"
12      ado_iteration: "Platform\\2026\\Q2"
13      ingested_at: 2026-06-09

```

Listing 50: Provenance metadata block (ADO example)

```

1 activities:
2   - id: customer-portal-redesign
3     label: Customer Portal Redesign
4     track: product
5     span: 2026-Q3
6     status: planned
7     metadata:
8       source: github
9       github_project_id: PVT_kwDOA12345
10      github_issue: 214
11      github_url: "https://github.com/acme/portal/issues/214"
12      ingested_at: 2026-06-09

```

Listing 51: Provenance metadata block (GitHub Projects example)

The `source` key in `metadata` is a reserved provenance marker. The ingester writes it; the renderer ignores it. Re-sync logic uses it to correlate IR entities back to their source records.

0.9.7.2 Re-Sync Strategy

When source data is updated (e.g., a work item's state changes from Active to Resolved), the re-sync operation must:

1. **Match by source ID.** Use `metadata.ado_id` or `metadata.github_issue` as the stable foreign key, not the IR id. The IR id is a human-edited slug that may have been renamed.
2. **Update only source-mapped fields.** Fields that the ingestion workflow maps from the source (e.g., `status`, `span` derived from `IterationPath`) are overwritten. Fields the human may have edited in the IR (`label`, `category`, `description`, `color`, `progress`) are *not* overwritten.
3. **Preserve the IR id.** The kebab-case slug must not be regenerated on re-sync. Once set, it is stable.
4. **Log what changed.** The re-sync operation must produce a structured change log (entity id, field, old value, new value) before writing, enabling human review.
5. **Reject destructive deletions.** If a source item is deleted or removed from scope, the re-sync must flag it as a warning rather than silently removing the IR entity. A human or agent must confirm removal.

0.9.7.3 Stable IDs and Git-Friendly YAML

The combination of stable kebab-case IDs and line-oriented YAML serialisation produces IR documents that diff well under version control:

```

1  activities:
2    - id: api-gateway-migration
3      label: API Gateway Migration
4      track: platform
5      span: 2026-Q2
6    - status: in-progress
7    - progress: 0.4
8    + status: done
9    + progress: 1.0
10   metadata:
11     source: ado
12     ado_id: 1024

```

Listing 52: Git diff of an IR update (status change + progress update)

Key properties of the YAML serialisation that preserve git-friendliness:

- Fields within each entity are emitted in a **canonical order** (required fields first, optional fields in schema definition order). Reordering fields does not change semantics but does generate spurious diffs; canonical order prevents this.
- **No auto-generated IDs or timestamps** in non-metadata fields. The IR ID is derived from content and frozen; `ingested_at` in `metadata` only changes when the entity is re-ingested from source.
- **Consistent quoting conventions.** Strings that are safe as bare YAML scalars are not quoted; strings with special characters use double quotes. Serialisers must apply this consistently to avoid quote-flip diffs.

- **Lists maintain insertion order**, not alphabetical. New entities appended to the end of a list produce a single-line diff at the bottom.

0.9.7.4 Human Edits and Incremental Refinement

A human author may open the IR YAML in any editor, add annotations, adjust labels, add a milestone the ingester missed, or change a `status` value to reflect a decision made after ingestion. These edits are valid IR operations and must survive re-sync (per the strategy above).

An agent may similarly refine an IR: for example, an editorial agent might re-read the document, notice that five activities in one track are unlabelled, and add descriptions. Because the agent is editing a YAML file with stable IDs and a well-defined schema, this is a safe, auditable operation. The MCP `validate_timeline` tool can be called after any edit to confirm the document remains valid.

The editing workflow is:

```

1 1. ingest(source, prompt) -> roadmap.yaml # initial
   ingestion
2 2. validate(roadmap.yaml) -> valid, 0 errors # confirm valid
3 3. edit(roadmap.yaml) -> roadmap.yaml (modified) # human/agent
   edit
4 4. validate(roadmap.yaml) -> valid, 0 errors # re-confirm
5 5. render(roadmap.yaml, svg) -> roadmap.svg #
   deterministic output
6 6. (commit roadmap.yaml + roadmap.svg to git)
```

Listing 53: Iterative refinement loop (human or agent)

Step 6 commits both the IR source and the rendered output. Because rendering is deterministic, the SVG in the repository is always reproducible from the YAML. The YAML is the source of truth; the SVG is a derived artefact that can be regenerated on demand.

0.9.8 IR Gaps and Open Questions

During the design of this section, three gaps or ambiguities in the IR contract were identified. These are flagged here for Leslie (reviewer) and Mark to reconcile; they have *not* been silently resolved in this section’s examples.

1. **today field missing from metadata.** Leslie’s scope decisions specify that the now symbolic date must be evaluated from an explicit `today` field in the IR (not from the system clock), in order to preserve determinism. However, the `metadata` table in Mark’s IR contract does not include a `today` field. Until this is resolved, the annotation type `today-marker` with `date: now` is non-deterministic. Suggested resolution: add `today: date?` to the `metadata` schema; if absent, render-time clock is used and a warning is emitted.
2. **Track sort field: index vs. order.** Mark’s binding contract (well-formedness invariant 14) refers to a `track.index` field with the rule “track index order values unique and non-negative”. The `04-ir.tex` specification defines the same field as

order (type `int?`). The canonical field name should be resolved to one term; this section uses `order` to match the spec, but the contract should be updated to match.

3. **span vs. start+end mutual exclusion.** The IR allows either `span` (shorthand) or `start` plus optional `end`, but does not specify the behaviour when both are present (e.g., `span: 2026-Q2` and `start: 2026-05-01` coexist in the same activity). Should this be a schema error, or should one take precedence? For agent generation, structured output can prevent this from occurring, but the validation specification needs an explicit ruling. Suggested resolution: treat co-presence of `span` and `start/end` as a schema error.

0.10 Comparison Analysis

0.10.1 Framing the Comparison

Timeline Compiler is not a project-management tool, a Gantt chart engine, or an interactive web application. It is a *visual-communication compiler*: a system that accepts structured temporal data (or natural-language plans) and deterministically renders presentation-quality timeline artefacts — SVG, PNG, PDF, and PPTX — from a stable, version-controlled, LLM-friendly Intermediate Representation (IR).

The competitive landscape therefore contains two distinct clusters:

- **Diagram-as-code tools** (Mermaid, PlantUML, D2) — source- control friendly, developer-facing, but limited in presentation fidelity and output format breadth.
- **Presentation / project-management tools** (think-cell, PowerPoint, MS Project) — presentation-quality, but proprietary, manual, and hostile to automation and version control.

Timeline Compiler is designed to occupy the empty cell at the intersection: open-source, git-friendly, agent-generation-friendly, *and* presentation-quality. Each tool below is evaluated on the axes most relevant to that position.

0.10.2 Tool-by-Tool Analysis

0.10.2.1 Mermaid (timeline and gantt)

Mermaid [20] is an MIT-licensed JavaScript diagramming library with over 88,000 GitHub stars [44] and native rendering on GitHub, GitLab, Notion, Obsidian, Azure DevOps, and hundreds of other platforms [20]. Its `timeline` diagram type [18] graduated from beta in 2023. The syntax groups events under named sections on a horizontal time axis:

```

timeline
  title 2025 Platform Roadmap
  section Infrastructure
    2025 Q1 : Database migration

```

```
2025 Q2 : CDN rollout
section Product
2025 Q2 : v2 public beta
2025 Q4 : GA launch : milestone
```

Mermaid also provides a `gantt` diagram type [19] with sections, task durations, dependencies, and a today-marker.

What Mermaid does well. Git-friendly plain text; effortless integration into Markdown-driven workflows; zero-install web rendering; large community; MIT license; AI/LLM tools already generate valid Mermaid syntax.

Where Mermaid falls short for our goal.

- *Presentation quality is limited.* Output is SVG/PNG rendered by a JavaScript engine. The `timeline` type lacks per-row swimlane formatting, precise font control, brand themes, or layout tuning suitable for executive slides.
- *No PPTX or PDF output.* Export is browser-dependent; no native headless PPTX generation.
- *The gantt type defaults to project-management idioms.* It renders task bars with dependency arrows and progress indicators — appropriate for engineering sprints, not for McKinsey-style roadmaps where visual communication is the goal.
- *Non-deterministic layout at scale.* Label placement and bar sizing change with viewport width; large roadmaps (> 50 events) degrade in readability.
- *No first-class theme layer.* Theming is CSS injection, not a typed theme schema.

0.10.2.2 PlantUML

PlantUML [29] is a GPL/commercial text-to-diagram tool built on GraphViz. Its `@startgantt` [30] mode is experimental as of 2024 and materially less capable than Mermaid's `gantt`: no section grouping, no resource assignment, no today-marker, and limited coloring. Its timeline diagram type is likewise minimal.

PlantUML outputs PNG, SVG, ASCII art, and PDF, and integrates with Confluence, Jenkins, and many JetBrains plugins. The server-side rendering architecture (a Java process) is reliable for CI pipelines.

Where it falls short. Gantt/timeline support is a secondary feature. Syntax is verbose and UML-centric — not natural for non-technical stakeholders. Visual output has a distinctly technical aesthetic; unsuitable for polished executive deliverables. The GPL license creates friction for proprietary embedding.

0.10.2.3 vis-timeline

vis-timeline [41] (MIT) is an interactive JavaScript timeline widget. Items and groups are defined as JavaScript objects; the library renders them in the browser as scrollable, zoomable, drag-and-drop timelines.

Where it falls short. It is entirely browser-interactive: there is no declarative text format (source data is a JavaScript data structure), no static SVG/PDF export, and no presentation layer. It is the right tool for dashboard UIs, not for printed roadmaps or slide decks. Version-controlling the rendered output is not meaningful because the output is dynamic HTML.

0.10.2.4 Frappe Gantt

Frappe Gantt [7] (MIT) is a lightweight SVG-based Gantt library used in ERPNext. Tasks are provided as JavaScript objects with `start`, `end`, `progress`, and `dependencies` fields. It renders to SVG in the browser.

Where it falls short. Like vis-timeline, input is programmatic JavaScript, not a text-serialisable data format. There is no declarative IR, no file-format input (YAML/JSON from a file), no PPTX/PDF output, and no theme system. The presentation quality of the output is good for a project-tracking dashboard but lacks the typography and layout control needed for executive presentations.

0.10.2.5 Microsoft Project

MS Project [23] is the industry-standard project-management application. It exports to a custom XML schema (`Project.xsd`) [21] that is non-deterministic between saves: GUIDs and timestamps change on each export, making diffs meaningless without post-processing. Output formats are proprietary `.mpp` binary and the unstable XML. No git-friendly text format exists natively.

Where it falls short. Proprietary, expensive (\$~30/user/month), entirely interactive, and requires manual visual design for presentation quality. Resource allocation and critical-path scheduling are first-class concerns — exactly the project-management semantics Timeline Compiler intentionally excludes (see §0.10.4). The binary format is hostile to LLM generation.

0.10.2.6 think-cell

think-cell [40] is a PowerPoint add-in widely used by management consultancies (McKinsey, BCG, Bain). It produces presentation-quality Gantt/timeline charts [37] with quarter/month/week granularity, responsibility columns, and smart label placement. Excel data-linking enables live updates. Pricing is approximately \$27.30/user/month [40].

Where it falls short. Proprietary, expensive, and tightly coupled to PowerPoint. There is no text-format input (no YAML/JSON), no CLI, no version-control workflow, and no programmatic (agent) generation path. Every change requires a human with PowerPoint. Git-diffing a `.pptx` file is not meaningful. The tool is excellent at its intended task (consulting slide production) but solves a different problem from Timeline Compiler.

0.10.2.7 PowerPoint Timeline (native SmartArt)

PowerPoint includes SmartArt-based timeline shapes [24].

Where it falls short. Fully manual. No source format, no determinism, no agent path, no version control. Visual quality is constrained by SmartArt templates; professional roadmaps almost always require manual box placement, which is time-consuming and fragile when dates change.

0.10.3 Comparison Table

Table 39 scores each tool on seven axes relevant to Timeline Compiler’s goals. Ratings are qualitative (H = High / M = Medium / L = Low) based on the analysis above and are not precise benchmarks.

The table shows that no existing tool simultaneously achieves high scores across all seven axes. think-cell scores highest on presentation quality but is closed-source with no agent path. Mermaid scores highest on openness and SCM friendliness but is limited in presentation fidelity and output format breadth. Timeline Compiler’s design targets every axis at High — a combination that is, at the time of writing, unoccupied.

0.10.4 Where Timeline Compiler Intentionally Differs

Three design choices explicitly diverge from the dominant approaches in this landscape.

1. Visual-communication grammar, not task-scheduling grammar. Mermaid’s gantt, MS Project, and Frappe Gantt all model a project schedule: tasks have durations, dependencies, progress percentages, and critical paths. Timeline Compiler’s IR models a *narrative* about time: events, spans, milestones, and swimlane groups, with an explicit visual-communication intent. Resource allocation, dependency graphs, and critical-path analysis are intentionally out of scope. The Grammar of Graphics [45] and Vega-Lite’s declarative approach [33] inform this separation of “what to show” from “how to render it”.

2. Deterministic rendering as a first-class property. Mermaid’s layout is viewport-dependent; MS Project XML changes on every save; PowerPoint files are binary blobs. Timeline Compiler requires that identical IR input produces bit-identical output. This property is essential for CI/CD pipelines, git-based review, and reliable LLM round-trips.

3. Agent generation as a primary use case. Existing tools treat human-in-the-loop as the default workflow: a user opens an application, adjusts bars, and exports. Timeline Compiler is designed to be driven by an AI agent with no human in the loop: the agent generates a valid IR (JSON or YAML conforming to the Timeline IR schema), the compiler renders it, and the artefact lands in a pull request. This requires a small, well-typed, documented IR that LLMs can target reliably — more analogous to Vega-Lite’s JSON grammar [33] than to Mermaid’s informal text syntax or MS Project’s XML schema.

0.10.5 Real-Data Crawl: Practical Patterns Observed

During the research phase, several real-world examples of the artefacts Timeline Compiler targets were surveyed. The following recurring patterns constrain the IR and rendering model.

1. **Quarter granularity dominates executive roadmaps.** McKinsey/BCG-style roadmaps consistently use Q1–Q4 as the primary time unit, with sub-quarter resolution reserved for milestones. The IR must support a **granularity** hint (year, quarter, month, week) so the renderer scales the time axis appropriately.
2. **Swimlane + milestone is the universal visual idiom.** Horizontal swimlanes (one per product, workstream, or business unit) with diamond or dot milestones at key dates is the dominant structural pattern across consulting firms, product teams, and programme offices. The IR needs first-class lane and milestone elements.
3. **Spans and points coexist.** Real roadmaps mix spans (“Migration: Q2–Q4 2025”) and point events (“GA Launch: 15 Sep 2025”). Both must be first-class IR entities.
4. **Azure DevOps and GitHub Projects are dominant data sources.** ADO work items [22] carry iteration path (`System.IterationPath`) for sprint/quarter mapping. GitHub Projects [9] use `DATE` and `ITERATION` custom fields for roadmap bars. An ingestion layer must map these fields to IR span/milestone events.
5. **Mermaid syntax is the lowest-common-denominator input.** Mermaid is already widely embedded in documentation. An ingester that accepts Mermaid timeline and gantt blocks lowers the migration barrier significantly.
6. **Manual PowerPoint is the current state-of-the-art for presentation quality.** The gap between what Mermaid can produce and what think-cell produces is large. Meeting (or approaching) think-cell output quality is the primary rendering challenge.

0.11 MVP Design

This section defines the Minimum Viable Product for Timeline Compiler, identifying must-have features, phased delivery, risks, and complexity hotspots.

0.11.1 MVP Philosophy

The MVP must answer one question definitively: *Can Timeline Compiler produce presentation-quality timelines from structured input, with deterministic output and theme-driven styling?*

Everything else — advanced features, edge cases, integrations — can follow once this core loop is validated. The MVP is not a prototype; it is a shippable product with limited scope.

0.11.2 Feature Prioritization

0.11.2.1 Must Have (P0)

These features are required for MVP launch:

IR Parser and Validator

Parse YAML input conforming to the Timeline IR schema. Validate against JSON Schema. Report clear, actionable errors.

Core IR Elements

Support for:

- timeline root with title and range
- tracks (swimlanes)
- activities with labels, date ranges, status, and progress
- milestones with dates and labels

Deterministic Layout Engine

Compute positions for all elements deterministically. Handle overlapping activities within tracks (stacking or compression).

SVG Output

Primary output format. Clean, well-structured SVG suitable for web embedding and further processing.

PDF Output

Print-quality vector output for documents and decks.

Basic Theme Support

At least one built-in theme (default). Theme defines colors, typography, spacing, and shapes.

CLI

Command-line interface with:

- `timeline render <input> -o <output> — render to file`
- `timeline validate <input> — validate IR without rendering`
- `-theme <name> — select theme`
- `-format <svg|pdf> — output format (or infer from extension)`

Schema Documentation

Published JSON Schema for the IR, with human-readable documentation.

0.11.2.2 Should Have (P1)

These features significantly improve usability and should be included if feasible:

PNG Output

Rasterized output for contexts requiring bitmap images.

Multiple Built-in Themes

At least 3 themes: default, minimal, corporate. Demonstrates theme system versatility.

Sections

Temporal divisions (e.g., quarters, phases) that span all tracks with visual demarcation.

Status Visualization

Color or icon mapping for status values (planned, in-progress, complete, at-risk).

Progress Bars

Visual representation of progress (0.0–1.0) within activity bars.

Today Marker

Vertical line at a specified “today” date.

npm Package

Core library published to npm for embedding.

Custom Theme Loading

Load themes from user-provided YAML/JSON files.

0.11.2.3 Nice to Have (P2)

These features are desirable but not blocking for MVP:

PPTX Output

PowerPoint-compatible output. Complex due to PPTX format internals.

VS Code Extension

Live preview, syntax highlighting. High value but significant development effort.

Groups

Nested groupings within tracks for hierarchical organization.

Annotations

Callouts and emphasis markers for specific dates or activities.

Dependency Arrows

Visual arrows between activities (visual only, no scheduling).

Legend Generation

Auto-generated legends for status colors or track semantics.

Watch Mode

CLI watches input file and re-renders on change.

MCP Server

Agent integration. Important for long-term vision but not required for initial launch.

0.11.2.4 Future (P3)

These features are out of scope for MVP but part of the long-term vision:

Theme Marketplace

Community-contributed themes.

Multi-file IR Composition

\$ref or include mechanisms for large timelines.

External Image Embedding

Logos, icons from external files.

Interactive SVG

Clickable regions, tooltips (breaks determinism constraints — careful design needed).

Hosted Rendering API

Cloud endpoint for rendering without local install.

Adapter Ecosystem

Tools that convert Jira, ADO, GitHub Projects to Timeline IR.

Localization

Date formats, RTL support, translated labels.

0.11.3 MVP Scope Summary**0.11.4 Phased Roadmap****0.11.4.1 Phase 1: Core Engine (MVP Launch)**

Duration: 6–8 weeks

Deliverables:

- IR schema (JSON Schema) finalized and documented
- Parser and validator
- Layout engine (deterministic)
- SVG renderer
- PDF renderer (via SVG conversion or direct)
- CLI with `render` and `validate` commands
- Default theme
- Public repository, MIT license, README, contributing guide

Exit Criteria:

- A 3-track, 10-activity roadmap renders to SVG and PDF
- Output is byte-identical across runs
- CLI works on macOS, Linux, Windows

0.11.4.2 Phase 2: Polish and Usability

Duration: 4–6 weeks (overlapping with Phase 1 completion)

Deliverables:

- PNG output
- 2 additional themes (minimal, corporate)
- Sections support
- Status and progress visualization

- Today marker
- npm package published
- Custom theme loading from file

Exit Criteria:

- Users can author themes without modifying core code
- npm package installable and functional

0.11.4.3 Phase 3: Ecosystem

Duration: 8–12 weeks

Deliverables:

- VS Code extension (syntax highlighting, live preview, export)
- MCP server (render tool for agents)
- PPTX output
- Watch mode in CLI
- Groups and annotations in IR

Exit Criteria:

- AI agent can generate IR and invoke rendering
- VS Code extension in marketplace

0.11.4.4 Phase 4: Scale and Community

Duration: Ongoing

Deliverables:

- Theme contribution workflow
- Adapter examples (Jira, GitHub, ADO)
- Documentation site
- Advanced IR features (dependency arrows, legends)
- Performance optimization for large timelines

0.11.5 Risks and Mitigations**0.11.5.1 Technical Risks****PDF Generation Complexity**

PDF output requires either converting SVG (quality/fidelity concerns) or direct PDF writing (complex). *Mitigation:* Use proven libraries; accept minor fidelity differences in MVP; iterate.

Font Determinism

Different systems have different fonts. Varying fonts break determinism. *Mitigation:* Require explicit font specification in themes; bundle fallback fonts; document font requirements.

Layout Edge Cases

Overlapping activities, long labels, dense timelines may produce poor layouts. *Mitigation:* Start with simple layout rules; document limitations; accept that MVP handles “typical” cases, not all.

Cross-Platform Rendering Differences

Floating-point differences across platforms may affect layout. *Mitigation:* Use integer math or fixed-point where possible; test on multiple platforms early.

0.11.5.2 Adoption Risks**Learning Curve**

Users must learn the IR format. *Mitigation:* Excellent examples, quick-start guide, VS Code extension with auto-complete.

Theme Quality

If built-in themes are unappealing, users won’t adopt. *Mitigation:* Invest in theme design; consult with visual designers; study exemplary slides.

Missing Feature X

Users may expect Gantt-like features (dependencies, resources) that are out of scope. *Mitigation:* Clear documentation of scope boundaries; position as “communication tool, not PM tool.”

0.11.5.3 Complexity Hotspots**PPTX Output**

The PPTX format (Office Open XML) is complex. Producing valid, editable PPTX files is non-trivial. *Mitigation:* Defer to Phase 3; use established libraries; accept limited editability initially.

Theme System

Balancing flexibility with simplicity is hard. Too simple: users can’t customize. Too complex: themes become a programming language. *Mitigation:* Start minimal; extend based on feedback; keep theme schema documented.

Layout Engine

Deterministic layout that handles edge cases (long text, overlap, many tracks) is complex. *Mitigation:* Define clear layout rules; document limitations; iterate.

0.11.6 Smallest Viable Version

If resources are constrained, the absolute smallest viable version is:

- IR schema for tracks, activities, milestones
- YAML parser and validator

- SVG output only
- CLI with `render` command only
- Single hardcoded theme
- No npm package (CLI only)

This version proves the core thesis: structured input to presentation-quality output, deterministically. Everything else is enhancement.

0.11.7 Success Metrics

Adoption

GitHub stars, npm downloads, VS Code extension installs.

Quality

Issue count, bug severity, user-reported rendering defects.

Determinism

CI tests that verify byte-identical output across runs and platforms.

Agent Usage

Number of agents using MCP server (post-Phase 3).

Community Contribution

Theme contributions, adapter contributions, PRs.

0.12 Open-Source Strategy

0.12.1 Is This a Viable Open-Source Project?

The short answer is yes, but with conditions. The diagram-as-code category has demonstrated commercial and community viability at scale: Mermaid [20] has over 88,000 GitHub stars, raised \$7.5M in seed funding in 2024 [17], and has been natively integrated into GitHub, GitLab, Notion, Obsidian, and Azure DevOps. The lesson from Mermaid is that a plain-text format with zero-install rendering, embedded in documentation ecosystems, can achieve massive adoption.

Timeline Compiler's gap analysis (Section 0.10) identifies a cell in the tool landscape that is genuinely unoccupied: an *open-source, presentation-quality, agent-generation-friendly* timeline compiler. This gap is both an opportunity and a risk; the following sections analyse both.

0.12.2 Closest Existing Projects and the Gap They Leave

- **Mermaid** (§0.10) leaves the presentation-quality gap. Its `timeline` and `gantt` types are documentation tools, not slide-production tools. No PPTX target. No theme engine. Layout is viewport-dependent, not deterministic.

- **TimelineJS** [15] (Knight Lab, GPL) is the best-known open-source timeline tool, but it is a web-storytelling widget (iframes, Google Sheets), not a document/slide compiler. No SVG/PPTX/PDF output; no CLI; no agent path.
- **Frappe Gantt** [7] and **vis-timeline** [41] are interactive browser widgets. No declarative text format; no static artefact output.
- **think-cell** [40] produces the closest visual quality but is closed-source, expensive, and PowerPoint-locked. It is the benchmark for what Timeline Compiler output should look like, not a substitute for it.
- **Plotly.js** [31] can produce high-quality timeline charts, but requires programming and has no declarative timeline grammar or agent-generation story.

The gap is real: the market has diagram-as-code (low quality, wrong idiom) or presentation tools (high quality, closed, manual). Nothing sits in between.

0.12.3 Potential Adopters

Four adopter personas have been identified based on real-data patterns observed during the research phase:

Developer-Relations and Technical Writers. Dev-rel teams already live in Mermaid and Markdown. They need to produce roadmaps and architecture-evolution timelines for conference talks, blog posts, and product docs. The barrier to adoption is near-zero if Timeline Compiler has a CLI, a Markdown code-fence syntax, and SVG output. This group provides early adopters and community feedback.

Product Managers and Programme Leads. PMs need quarterly roadmaps and programme timelines for steering committees and executive reviews. They currently use PowerPoint or think-cell manually. A compiler that takes a YAML/CSV definition and emits a PPTX with think-cell-quality layout would replace hours of work per roadmap cycle. This group provides the strongest word-of-mouth growth vector.

Management Consultants (Freelance / Boutique). Large firms use think-cell. Freelancers and boutique consultancies cannot justify \$27.30/user/month per project. An open-source tool with a polished PPTX target has a credible value proposition here.

AI Agent Toolchains. The emergent user is not a human at all. LLM-based planning agents (GitHub Copilot, OpenAI Assistants, Anthropic Claude with tools) generate project plans, architecture proposals, and transformation roadmaps. Today there is no standard open-source *target format* for those plans. Timeline Compiler's IR, if published as a JSON Schema and exposed as an MCP tool [4], becomes the target format AI agents compile to. This persona drives long-tail adoption as agent toolchains proliferate.

0.12.4 Ecosystem Fit

0.12.4.1 Mermaid / Markdown / CI Ecosystem

Timeline Compiler should position itself as the “next step” after Mermaid: use Mermaid in your README; use Timeline Compiler when you need a slide. Concretely:

- A Mermaid-syntax ingester (accepting `timeline` and `gantt` blocks) provides a zero-friction migration path.
- A Markdown code-fence processor (similar to Mermaid’s own fence renderer) enables embedding timelines in MkDocs, VitePress, and Quarto [20].
- A GitHub Actions step for CI-driven rendering integrates into the workflow teams already use for diagram-as-code.
- SVG output embeds directly in Slidev [3] and Obsidian [27] — both are Mermaid-native environments.

0.12.4.2 MCP / Agent Ecosystem

The Model Context Protocol [4] is the emerging standard for LLM tool invocation. An MCP server that exposes Timeline Compiler as a tool would allow any MCP-compatible agent (GitHub Copilot, Claude, etc.) to:

1. Accept a natural-language plan from the user.
2. Generate a valid Timeline IR (JSON/YAML, constrained by the published JSON Schema).
3. Invoke the Timeline Compiler MCP tool.
4. Receive back an SVG, PNG, or PPTX and embed it in the agent’s response.

OpenAI’s Structured Outputs feature [28] (JSON Schema-constrained generation) enables agents to reliably produce valid IR without hallucinating schema fields. The design implication is that the Timeline IR schema *must* be published as a JSON Schema document alongside the compiler.

0.12.4.3 PPTX Ecosystem

`python-pptx` [34] is the leading open-source Python library for programmatic PPTX generation. It is MIT-licensed, well-maintained, and can produce slides that open natively in PowerPoint and Keynote. Building the PPTX output target on `python-pptx` avoids a dependency on LibreOffice or headless Chrome, making the compiler pip-installable with minimal system requirements.

0.12.5 Extension Opportunities

The compiler-IR architecture creates natural extension points:

Ingesters.

New data sources can be added without touching the renderer: CSV/Excel, Jira API, Linear API, Notion database, ADO work items [22], GitHub Projects [9], markdown plans, Mermaid blocks. Ingesters are the highest-traffic contribution path for community members.

Themes.

A typed theme schema (fonts, colour palettes, swimlane heights, milestone shapes) allows community contribution of brand themes without touching rendering code. McKinsey-style, BCG-style, GitHub-style, and corporate-brand themes are natural community contributions.

Output Targets.

Additional renderers (HTML interactivity, Keynote, Google Slides API, LaTeX beamer) can be added without changing the IR or ingesters. The Vega-Lite model [33] demonstrates that a clean IR/renderer separation enables this pattern reliably.

Grammar Extensions.

The IR can be versioned and extended (new event types, annotations, dependency arrows) without breaking existing themes or renderers, following established IR versioning practice from compiler design [1].

0.12.6 Strongest Differentiators

Three differentiators are assessed as strongest, in decreasing order of defensibility:

1. **The only open-source, agent-generation-friendly timeline compiler with PPTX output.** No existing open-source tool combines a stable text-format IR, deterministic rendering, and a PPTX output target. This is the clearest gap in the landscape.
2. **Visual-communication grammar vs. task-scheduling grammar.** Positioning the tool as a “timeline grammar for human communication” rather than a “project management tool” is a genuine conceptual distinction. It attracts users who are currently underserved by Mermaid (insufficient quality) and repelled by MS Project (wrong abstraction).
3. **AI-agent-first design.** Publishing a JSON Schema for the IR and an MCP tool interface makes Timeline Compiler the natural compiler target for any LLM that generates plans or roadmaps. As agent-driven workflows proliferate [4], this positions Timeline Compiler at the centre of a growing distribution channel with no human adoption friction.

0.12.7 Community Contribution Model

The Mermaid model [17] is instructive: open-source core (MIT), commercial SaaS layer for hosted rendering, team features, and enterprise integrations. Timeline Compiler should follow a similar open-core pattern:

- **Core compiler** (MIT): IR schema, renderer, CLI, standard output targets, standard ingesters. All of this is open-source. Community contributions to ingesters and themes are accepted here.

- **Theme marketplace** (community-driven): A GitHub-hosted registry of community themes, analogous to the npm registry for packages or the VS Code extension marketplace. Themes are independent packages; they do not require merging into the core.
- **Hosted service** (future, optional): Web UI for non-technical users, CI integration hooks, and enterprise SSO. This funds maintenance without restricting the open-source core.

The contribution surface most likely to attract community contributions (in descending order of probability) is: ingesters, then themes, then additional output targets, then IR schema extensions. Core renderer changes require higher trust and should require maintainer review.

0.12.8 Adoption Risk Assessment

No adoption analysis is honest without a candid risk section.

Risk 1: The presentation-quality bar is high. think-cell is the quality benchmark. Matching it with an open-source, text-driven renderer is a significant engineering challenge. If the initial PPTX output is visually inferior to think-cell, the “presentation quality” differentiator collapses, and Timeline Compiler is merely a less capable Mermaid. *Mitigation:* Define a strict visual quality bar in the design spec and validate against real executive roadmaps before launch.

Risk 2: The IR may be too narrow or too wide. An IR that is too narrow (only covering the simplest roadmaps) will be inadequate for real-world use. An IR that is too wide (attempting to model every possible timeline idiom) will be complex to understand, hard for agents to generate reliably, and fragile to maintain. *Mitigation:* Scope the IR to the five canonical timeline types identified in the scope section, and use real data crawls (ADO exports, GitHub Projects exports) to validate coverage before freezing the schema.

Risk 3: Mermaid is a gravity well. Mermaid has 88,000 stars, is embedded everywhere, and is improving. If Mermaid’s `timeline` type adds PPTX export and better theming, the core differentiator weakens. *Mitigation:* The visual-communication grammar thesis and the agent-generation design are harder to retrofit into Mermaid than a PPTX renderer. The IR schema and MCP interface are the durable moats.

Risk 4: Agent toolchains may standardise on Mermaid instead. LLMs already reliably generate Mermaid syntax. If the agent ecosystem converges on Mermaid + post-processing as the standard agent-to-timeline pipeline, the agent-first positioning loses force. *Mitigation:* Publish the JSON Schema early; build the MCP tool before the ecosystem settles. Structured Outputs [28] are demonstrably more reliable than free-text Mermaid generation for complex schemas.

Risk 5: PPTX format instability. The OOXML PPTX specification is controlled by Microsoft. `python-pptx` [34] may lag new PowerPoint features. *Mitigation:* Treat PPTX as an output layer, not the IR. SVG and PDF are canonical; PPTX is a convenience export. Degrade gracefully when PPTX features are unavailable.

0.13 Timeline Grammar vs. Task Scheduling Grammar

This section presents the central thesis of Timeline Compiler: the correct abstraction for this problem is a **Timeline Grammar** (optimized for visual communication) rather than a **Task Scheduling Grammar** (optimized for project management). This distinction is not merely terminological — it fundamentally shapes the IR design, scope boundaries, and long-term extensibility.

0.13.1 The Thesis

Gantt-thinking is the wrong default. Timeline Compiler should not model tasks, dependencies, and schedules. It should model visual narratives: tracks, phases, milestones, emphasis, and annotations. The IR is a grammar for visual communication, not a grammar for work decomposition.

0.13.2 Two Grammars Compared

0.13.2.1 Task Scheduling Grammar

A Task Scheduling Grammar models *work to be done*. Its primitives:

Tasks

Discrete units of work with effort estimates, assignees, and dependencies.

Dependencies

Relationships between tasks (finish-to-start, start-to-start, etc.) that constrain scheduling.

Resources

People, teams, or assets assigned to tasks, subject to capacity constraints.

Dates

Computed from dependencies and resource availability, not directly authored.

Critical Path

The longest chain of dependent tasks; determines project duration.

Milestones

Markers for task completion or deliverable readiness, often derived from task dependencies.

This grammar powers MS Project, Primavera, Jira Advanced Roadmaps, and enterprise scheduling tools. Its purpose is *operational*: compute feasible schedules, optimize resource allocation, identify bottlenecks.

A Gantt chart is the canonical visualization of a task scheduling grammar. It shows tasks as bars, dependencies as arrows, resources as assignments, and the critical path highlighted. The visual is a *byproduct* of the scheduling model.

0.13.2.2 Timeline Grammar

A Timeline Grammar models *communication about time*. Its primitives:

Tracks

Visual lanes that organize content by theme, workstream, or category — chosen for *narrative clarity*, not operational structure.

Activities

Time-bounded visual elements representing initiatives, phases, or efforts. Dates are directly authored, not computed. Activities have no inherent dependencies.

Milestones

Emphasized points in time representing decisions, deliverables, or checkpoints. Their meaning is communicative, not operational.

Sections

Temporal divisions (quarters, phases, years) that provide visual structure and context.

Annotations

Callouts, emphasis, and labels that direct attention.

Status and Progress

Visual indicators that communicate state to an audience, not operational data for a scheduler.

This grammar powers McKinsey roadmaps, BCG transformation timelines, board-deck quarterly plans, and executive communication artifacts. Its purpose is *rhetorical*: convey a plan, tell a story about time, align stakeholders.

0.13.3 Why Gantt-Thinking Is Wrong Here

If Timeline Compiler adopted a Task Scheduling Grammar, it would inherit these problems:

0.13.3.1 Computational Complexity

Task scheduling is NP-hard in the general case (resource-constrained project scheduling). A scheduling grammar implies:

- Dependency resolution algorithms
- Constraint satisfaction
- Resource leveling
- What-if scenario computation

None of this is relevant to producing a PDF for a board presentation. It is massive accidental complexity.

0.13.3.2 Semantic Overload

A scheduling grammar interprets user input operationally. If users declare:

```
1 activities:
2   - id: design
3     depends_on: [requirements]
```

A scheduling grammar must:

- Validate that `requirements` exists
- Compute `design`'s start date from `requirements`' end date
- Propagate changes transitively
- Detect cycles

A timeline grammar simply draws an arrow from `requirements` to `design`. The dependency is a *visual annotation*, not a scheduling constraint.

0.13.3.3 Schema Bloat

Scheduling grammars require:

- Resource definitions (people, teams, capacities)
- Effort estimates (person-days, story points)
- Calendars (working days, holidays)
- Dependency types (FS, SS, FF, SF with lags)
- Constraint types (must-start-on, no-earlier-than)

None of this serves the communication use case. It bloats the IR, confuses authors, and distracts from the actual goal.

0.13.3.4 Agent Hostility

LLMs can easily generate timeline grammars. “Show Platform work from January to March, with a milestone in February” maps directly to:

```
1 tracks:
2   - label: "Platform"
3     activities:
4       - label: "Core Development"
5         range: { start: 2026-01, end: 2026-03 }
6 milestones:
7   - label: "Alpha Release"
8     date: 2026-02-15
```

Agents struggle with scheduling grammars because:

- Dependencies must be globally consistent (cycles are invalid)
- Dates must satisfy constraint propagation

- Resource assignments must respect capacities

These invariants are hard to maintain without iterative refinement. Timeline grammars are *flat*: each element is independently valid.

0.13.3.5 The Gantt Trap

Gantt charts are so ubiquitous that “timeline” often defaults to “Gantt chart.” But Gantt charts are poor communication artifacts:

- Dense with dependency arrows that obscure the narrative
- Emphasize task-level detail over strategic phases
- Require operational context (resources, constraints) that audiences lack
- Visually noisy — optimized for schedulers, not executives

Timeline Compiler explicitly rejects the Gantt default. It produces *timeline visualizations*, not Gantt charts. Gantt-style output may be achievable via themes, but it is not the design center.

0.13.4 Timeline Grammar: Design Implications

Adopting a Timeline Grammar shapes the architecture:

0.13.4.1 IR Simplicity

The IR is a flat description of visual elements:

```
1 timeline:
2   title: "Product Roadmap 2026"
3   range: { start: 2026-01, end: 2026-12 }
4   tracks:
5     - id: platform
6       label: "Core Platform"
7       activities:
8         - label: "API v2"
9           range: { start: 2026-01, end: 2026-04 }
10        - label: "Scale Testing"
11          range: { start: 2026-05, end: 2026-07 }
12   milestones:
13     - label: "GA Launch"
14       date: 2026-08-01
```

No dependencies to resolve. No resources to level. No constraints to satisfy. The IR *is* the plan, directly.

0.13.4.2 No Computation

The compiler performs no computation on plan semantics:

- Dates are taken as given
- Progress is taken as given
- Status is taken as given

The compiler’s job is *layout and rendering*, not interpretation.

0.13.4.3 Visual-First Extensions

Future extensions follow the timeline grammar:

- **Annotations** — callouts, emphasis, arrows (visual, not operational)
- **Sections** — phase labels, quarter markers (narrative structure)
- **Legends** — explaining visual encodings (communication aid)
- **Dependency arrows** — optional visual hints, not scheduling constraints

Extensions that imply computation (resource leveling, critical path) are rejected by the grammar’s philosophy.

0.13.4.4 Determinism by Default

Timeline grammars are inherently deterministic. There is no computation that could introduce variability. The IR fully specifies the content; the theme fully specifies the style; the renderer maps one to the other.

Scheduling grammars resist determinism because scheduling is sensitive to algorithm implementation, floating-point accumulation, and tie-breaking heuristics.

0.13.5 Addressing the Counter-Arguments

0.13.5.1 “But I need dependency arrows”

Response: Timeline Compiler can render dependency arrows as visual annotations. The IR may include:

```

1 activities:
2   - id: alpha
3     label: "Alpha Phase"
4     range: { start: 2026-01, end: 2026-03 }
5   - id: beta
6     label: "Beta Phase"
7     range: { start: 2026-04, end: 2026-06 }
8     depends_on: [alpha] # Visual arrow, not scheduling constraint

```

The `depends_on` field means “draw an arrow from alpha to beta.” It does not mean “compute beta’s start from alpha’s end.” The dates are author-provided. The arrow is visual.

0.13.5.2 “What if my dates are inconsistent?”

Response: The validator may warn if beta starts before alpha ends (given a declared dependency). But this is a *warning*, not an error. The compiler renders what the author specified. Fixing inconsistencies is the author’s job, or an upstream tool’s job.

This matches the communication use case. A roadmap slide may intentionally show overlapping phases. The compiler should not second-guess the author.

0.13.5.3 “Project management tools already have timeline views”

Response: True, and those views are outputs of a scheduling grammar. They:

- Look like their source tool (Jira exports look like Jira)
- Include scheduling clutter (dependencies, resources)
- Require the source tool to generate

Timeline Compiler is a *standalone* rendering target. It accepts a simple IR that any tool (or agent) can generate. It produces presentation-quality output that looks like a designed slide, not a tool export.

0.13.5.4 “Won’t users expect scheduling features?”

Response: Clear positioning is essential. Timeline Compiler is a *communication compiler*, not a scheduling engine. Users who need scheduling should use MS Project, Jira, or Linear — and then export to Timeline IR for rendering.

This is the L^AT_EX model: L^AT_EX is a typesetting system, not a word processor. It does one thing excellently. Timeline Compiler is a timeline renderer, not a project manager.

0.13.6 Optimization Goals Alignment

The brief specifies five optimization goals. Timeline Grammar aligns with all five:

Presentation Quality

Timeline Grammar focuses on visual communication, not operational detail. Outputs are clean, narrative-driven, and boardroom-ready.

Agent Generation

Timeline Grammar is flat and simple. Agents generate valid IR without constraint satisfaction. The schema fits in a context window.

Deterministic Rendering

No scheduling computation means no computational variability. Same IR, same output, always.

Long-Term Extensibility

The grammar is a foundation for visual extensions (annotations, animations, interactive elements) without importing scheduling complexity.

Simplicity

The IR is small. The compiler is focused. There is no hidden complexity in dependency resolution or resource leveling.

0.13.7 Conclusion

Timeline Compiler adopts a **Timeline Grammar**: a model for visual communication about time, not operational scheduling. This is the correct abstraction because:

1. It produces presentation-quality visuals without scheduling complexity.
2. It is agent-friendly: flat, simple, constraint-free.
3. It is deterministic by construction.
4. It is extensible along visual dimensions without inheriting scheduling scope creep.
5. It is simple: the IR describes what to render, not what to compute.

The Gantt chart is not the design center. The McKinsey slide is. Timeline Compiler is a compiler from structured communication intent to visual artifact — and that is all it should be.

IR field	iCal schema.org	/ OWL- Time	Assessment
start, end	DTSTART/DTEND; startDate/endDate	hasBeginning/ hasEnd	Aligned. Standard names; shorter preferred for authoring.
label	SUMMARY; name	—	Aligned. Shorter form preferred for YAML.
description	DESCRIPTION	—	Aligned.
url	URL; url	—	Aligned.
tags	CATEGORIES (multi)	—	Aligned. Correct plural form.
category	CATEGORIES (single)	—	Aligned.
status:	STATUS:TENTATIVE	—	Aligned. Verbatim iCalendar value.
tentative			
status:	STATUS:CANCELLED	—	Aligned. Lower-case per IR convention.
cancelled			
status:	—	—	Original. No standard analog; justified by executive-presentation idiom.
at-risk			
status: blocked	—	—	Original. No standard analog; justified by visual-communication need.
end: ongoing	—	omit hasEnd	Semantically aligned. Explicit encoding is clearer than omission.
span	—	—	Original. Convenient shorthand; no standard equivalent.
track	CALENDAR (weak)	—	Original. Markwhen uses <code>group</code> ; <code>track</code> is clearer.
progress	—	—	Original. No standard analog; no change needed.
Activity (type)	VEVENT (duration > 0)	Interval	Aligned. Correct modelling of time-spanning entities.
Milestone (type)	VEVENT (zero dur.)	Instant	Aligned. Correct modelling of point-in-time markers.

Table 19: Alignment of Mark’s IR field names and types with prior-art standards.

Symbol	Definition
W	<code>theme.canvas.width</code> — total canvas width in logical pixels
m_L, m_R, m_T, m_B	Left, right, top, bottom margins
H_{hdr}	<code>theme.track.header_width</code> — left column for track labels
H_{axis}	<code>theme.axis.height</code> — horizontal time-axis strip
W_{draw}	$W - m_L - m_R - H_{\text{hdr}}$ — drawable timeline width
H_{draw}	$\sum_i h_i + \sum_{i>0} g$ — total height of all track rows plus gaps
H	Computed canvas height: $m_T + H_{\text{axis}} + H_{\text{draw}} + m_B$

Table 20: Coordinate system symbols

time_range span (days, inclusive)	Inferred axis_unit
≤ 14	day
≤ 91	week
≤ 548	month
≤ 1461	quarter
≤ 2922	half
> 2922	year

Table 21: Axis unit inference thresholds (first match wins)

Date form	As left edge (start)	As right edge (end)
2026-06-09 (day)	2026-06-09	2026-06-09
2026-06 (month)	2026-06-01	2026-06-30
2026 (year)	2026-01-01	2026-12-31
2026-Q2 (quarter)	2026-04-01	2026-06-30
2026-H1 (half)	2026-01-01	2026-06-30
FY26-Q2 (fiscal)	First day of fiscal Q2	Last day of fiscal Q2

Table 22: Date coercion to day boundaries by edge role

axis_unit	Primary label content	Secondary label (every N ticks)
day	Day-of-month number	Month name (every 7)
week	Week-start day	Month name (every 4)
month	Month abbreviation	Year number (every 12)
quarter	Q1 / Q2 / Q3 / Q4	Year number (every 4)
half	H1 / H2	Year number (every 2)
year	Year number	None

Table 23: Tick label content rules per axis unit

Status	Fill role	Opacity	Pattern	Visual intent
planned	Accent-light	1.0	solid	Default; not yet started
in-progress	Accent-dark	1.0	solid	Active; full saturation
done	Grey-mid	0.85	solid	Muted; completed
at-risk	Amber	1.0	solid	Warning; attention needed
blocked	Red	1.0	solid	Danger; cannot proceed
cancelled	Grey-light	0.60	diagonal-hatch	De-emphasized; struck through
tentative	Grey-light	0.70	dashed-border	Uncertain; may not happen

Table 25: Status-to-style map structure (colour roles; each theme supplies hex values)

Property	Consulting		Executive		Product		Release		Minimal		Showcase	
Fidelity tier	Tier	1	Tier 2	Polished	Tier 2	Polished	Tier	1	Tier	0	Tier	3
Background	Crisp		White		White		Crisp		Minimal		Showcase	
	White		White		White		White		White		Deep	
Axis labels	Bold Q	la-	Small-		Week		Week		Year only		Large Q	
	labels		caps		numbers		dates				labels	
Gridlines	None		Light		Dashed		Solid		None		Soft glow	
			quarterly		monthly		weekly				lines	
Track header	180	px	150	px	140	px	120	px	140	px	180	px
	bold		serif		colored		mono		serif		frosted	
Bar corners	Square		Rounded		Rounded		Square		Square		Rounded	
			4 px		3 px						8 px	
Bar effects	None		None		None		None		None		Drop	
											shadow	
Status signal	Colour		Colour	+	Icon	+	Colour		Pattern		Colour	+
	only		icon		colour		only		only		glow	
Milestones	Filled		Navy	+	Small,		Triangle		Open dia-		Large	+
	black		dot		low				mond		glow ring	
Today marker	Off		Red		Red solid		Bold red		None		Bloom	
			dashed								halo	
Cloud layer	None		None		None		None		None		10% opac-	
											ity	
Legend	None		Right, full		Bottom,		Top-right		None		Bottom,	
					full						white	

Table 26: Visual comparison: same IR rendered under six themes; Showcase requires Raster backend for full fidelity

Tier	Name	Effect classes	Min. backend	Themes
0	Minimal	No effects; solid fills, patterns, and opacity only	SVG backend	Minimal
1	Crisp	Linear/radial gradients, diagonal hatch, dashed borders, clip paths	SVG backend	Consulting, Release
2	Polished	Drop shadows, soft glow; native PPTX effects; SVG safe-filter set (determinism caveat)	SVG (caveat) or Raster	Executive, Product
3	Showcase	Bloom, cloud/atmosphere layers, noise textures, gradient meshes, volumetric compositing	Raster backend required	Showcase

Table 27: Fidelity tier definitions, effect classes, minimum required backend, and theme assignments

Effect	SVG construct	Determinism
Linear gradient	<linearGradient> in <defs>	Fully deterministic
Radial gradient	<radialGradient> in <defs>	Fully deterministic
Pattern (hatch)	<pattern> in <defs>	Fully deterministic
Opacity	opacity attribute	Fully deterministic
Drop shadow	feDropShadow composite	Non-deterministic (caveat)
Soft glow	feGaussianBlur + feComposite	Non-deterministic (caveat)
Noise / texture	feTurbulence	Non-deterministic (caveat)

Table 28: SVG effect constructs and determinism status; “Non-deterministic (caveat)” means the construct exists but renders inconsistently across SVG renderers and browsers

Strategy	Fidelity	Editability	Implementation
Image embedding	Perfect fidelity to raster output	None: opaque image element	Trivial (one API call per slide)
Native shapes	Geometry faithful; effects approximated or omitted	Full: bars, diamonds, and labels are movable, recolourable MSO shapes	High: EMU conversion, MSO shape API, hatch XML
Hybrid (recommended)	High for Tier-0/1 geometry; approximated for Tier-2 effects; rasterised for Tier-3 art layers	Partial: native shapes beneath opaque art-layer overlays	Medium

Table 29: PPTX output strategies and trade-offs

Effect Class	SVG Backend	Raster Backend	PPTX Backend
Solid fills	Native	Native	Native
Linear/radial gradient	Native	Native	Native
Gradient mesh	Approximated	Native	Approximated
Diagonal hatch	Native (pattern)	Native	Native (hatch fill)
Opacity/transparency	Native	Native	Native
Clip paths	Native	Native	Native (limited)
Drop shadow	Non-det. filter	Native	Native shape effect
Soft glow	Non-det. filter	Native	Native shape effect
Bloom / light scatter	Not available	Native	Approximated
Diffuse soft shadow	Non-det. filter	Native	Approximated
Blur	Non-det. filter	Native	Not available
Noise / turbulence	Non-det. filter	Native	Raster embed
Cloud / atmosphere	Not available	Native	Raster embed
Per-pixel compositing	Limited (opacity)	Native	Limited
<i>Max fidelity tier</i>	Tier 1 (det.); Tier 2 (caveat)	Tier 3	Tier 2 (native) + Tier 3 (hybrid)

Table 30: Backend capability profile by effect class; “Non-det. filter” means the SVG construct exists but is not deterministic across renderers/browsers

Property	SVG	PNG	PDF	PPTX	HTML
Scalable	Yes	No	Yes	Yes	Yes (SVG path)
Byte-deterministic	Tier 0/1	Per backend	Per backend	Geometry only	Geometry only
Art effects	Tier 1 only	Tier 3 (raster)	Both paths	Hybrid	Both paths
Editable shapes	Via Inkscape	No	No	Yes (native)	No
Text searchable	Yes	No	Yes	Yes	Yes
Print-ready	Via viewer	At 300 DPI	Yes	Via PPT	No
Universal compat.	High	Very high	Very high	Office-only	Browser-only
Interactive	No	No	No	No	Yes
Impl. complexity	Foundation	Low	Low-medium	High	Low
Build priority	1	2	3	4	6

Table 31: Output format properties and recommended build priority

Table 32: Language Trade-offs for Distribution

Factor	TypeScript/Node	Go
CLI standalone binary	Requires pkg/nexe or similar	Native
npm embedding	Native	Requires WASM or subprocess
WASM (browser/edge)	Via esbuild or similar	Native compilation
MCP server	Native (Node ecosystem)	Native
VS Code extension	Native embedding	Subprocess or WASM
Community contributions	Larger pool (frontend devs)	Smaller but growing
PDF generation	Needs native dep (puppeteer, etc.)	Native libraries available
Startup time	Slower (V8 init)	Fast
Binary size	Larger (bundled runtime)	Smaller

Category	Definition	Examples
Assumed	Facts taken from the prompt or schema without source evidence	Title derived from prompt instruction; version "1.0"
Inferred	Derived from source data by a deterministic rule	axis_unit from date span; time_range from min/max activity dates; track from AreaPath
Defaulted	Omitted from IR, relying on documented schema default	status: planned omitted when all items are future; locale omitted
Rejected	Source data discarded, reason logged	Bugs dropped when prompt says "features and milestones only"; items outside prompted time window; duplicate or zero-duration events with no label

Table 33: Ingestion decision categories

Prompt Signal	Ingestion Effect
Time horizon (“ <i>Q1–Q3 2026</i> ”)	Sets or constrains <code>time_range</code> ; items outside window are rejected
Entity filter (“ <i>features and milestones only</i> ”)	Restricts work item types included; bugs, tasks dropped
Track grouping (“ <i>group by team</i> ”)	Maps source field (e.g., <code>AreaPath</code>) to tracks
Emphasis (“ <i>highlight at-risk items</i> ”)	Promotes <code>status</code> field; may add annotation
Framing (“ <i>executive summary</i> ”)	Reduces label verbosity; promotes high-level items
Title / subtitle	Populates <code>metadata.title</code> / <code>metadata.subtitle</code>
Milestone promotion (“ <i>mark GA as a milestone</i> ”)	Converts matching activity to milestone entity

Table 34: Prompt signals and their ingestion effects

ADO Field	IR Target	Notes
<code>System.Title</code>	<code>activity.label</code> / <code>milestone.label</code>	Verbatim or prompt-shortened
<code>System.State</code>	<code>activity.status</code>	Via Table 36
<code>System.WorkItemType</code>	<code>activity.category</code> / entity type	Feature → activity; Milestone →
<code>System.IterationPath</code>	<code>span</code> or <code>start/end</code>	E.g. <code>Platform\2026\Q2</code> → <code>span</code>
<code>System.AreaPath</code>	<code>track</code>	Leaf segment becomes track label
<code>Microsoft.VSTS.Scheduling.TargetDate</code>	<code>milestone.date</code>	Used when promoting to milestone
<code>System.Tags</code>	<code>activity.tags</code>	Passed through
<code>System.Id</code>	<code>metadata.ado_id</code>	Provenance (not rendered)

Table 35: ADO work item field mapping

ADO State	IR Status
New	planned
Active	in-progress
Resolved	done
Closed	done
Removed	cancelled
On Hold	blocked
(<i>other</i>)	planned (warn: unmapped state)

Table 36: ADO state to IR status mapping

GitHub ProjectV2 Field	IR Target	Notes
<code>title</code> (built-in)	<code>activity.label</code> / <code>milestone.label</code>	Verbatim or shortened
<code>status</code> (select field)	<code>activity.status</code>	Label mapped to IR enum
<code>DATE</code> custom field	<code>start</code> / <code>milestone.date</code>	Named field determines role
<code>ITERATION</code> custom field	<code>span</code>	Iteration title maps to quarter
<code>labels</code>	<code>activity.category</code> (first) / <code>activity.tags</code>	First label to category
<code>milestone</code> (linked)	<code>milestones</code> entry	Promoted to IR milestone
<code>repository</code>	<code>track</code>	When prompt says “group by repo”
<code>assignees</code>	<code>metadata.assignees</code>	Not rendered; preserved for provenance
<code>number</code> (issue #)	<code>metadata.github_issue</code>	Provenance link

Table 37: GitHub ProjectV2 field mapping

Layer	Name	What it checks	Failure mode
1	Syntactic parse	Valid YAML or JSON; no parse errors	Hard error: stop
2	Schema conformance	Required fields present; types match; enum values valid; ID regex; oneOf for start/span	Hard error: list all violations
3	Well-formedness	Referential integrity; ID uniqueness; date ordering; progress bounds; group acyclicity	Hard error: list all violations
4	Render-readiness	See §0.5 (Barbara)	Error or warning depending on severity
5	Semantic advisory	Degenerate documents; empty activities and milestones; items outside time_range	Warning only

Table 38: Validation pipeline layers

Table 39: Comparison of timeline and roadmap tools on axes relevant to Timeline Compiler. H = High, M = Medium, L = Low. “Agent-gen” = suitability for automated generation by an LLM/agent. “Determinism” = stable, reproducible output from the same input. “SCM” = source-control management friendliness.

Tool	<i>Pres. quality</i>	<i>Determinism</i>	<i>SCM</i>	<i>Agent-gen</i>	<i>Open source</i>	<i>PPTX out</i>	<i>SVG/PDF out</i>
Mermaid timeline [18]	M	M	H	H	H	L	M
Mermaid gantt [19]	L	M	H	H	H	L	M
PlantUML [29]	L	H	H	M	H	L	H
vis-timeline [41]	M	L	L	L	H	L	L
Frappe Gantt [7]	M	L	L	L	H	L	L
MS Project [23]	M	L	L	L	L	L	M
think-cell [40]	H	M	L	L	L	H	L
PowerPoint [24]	M	L	L	L	L	H	L
Timeline Compiler (target)	H	H	H	H	H	H	H

Table 40: MVP Feature Summary

Priority	Label	Features
P0	Must Have	IR parser, validation, tracks, activities, milestones, layout engine, SVG, PDF, CLI, 1 theme, schema docs
P1	Should Have	PNG, 3 themes, sections, status colors, progress bars, to-day marker, npm package, custom themes
P2	Nice to Have	PPTX, VS Code extension, groups, annotations, dependency arrows, legends, watch mode, MCP server
P3	Future	Theme marketplace, multi-file IR, images, interactive SVG, hosted API, adapters, localization

Bibliography

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, 2 edition, 2006. ISBN 978-0-321-48681-3. The “Dragon Book”. Chapter 5 covers intermediate representations. Informs the argument that a stable, typed IR is preferable to direct rendering from raw input.
- [2] James F. Allen. Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26(11):832–843, 1983. doi: 10.1145/182.358434. Establishes the 13 binary interval relations (before, meets, overlaps, starts, during, finishes, equals, and converses) that underpin OWL-Time and qualitative temporal reasoning. Validates open/ongoing interval semantics: an activity with no end date “starts” the document time range without requiring a concrete bound.
- [3] Anthony Fu and contributors. Slidev: Presentation slides for developers. <https://sli.dev/>, 2024. MIT-licensed Markdown-first slide tool with Mermaid integration. Relevant as a downstream consumer of Timeline Compiler SVG output.
- [4] Anthropic. Model context protocol — specification. <https://spec.modelcontextprotocol.io/>, 2024. Open protocol for LLM tool/resource access. Defines how agents invoke external tools and consume structured outputs. Timeline Compiler can expose MCP tools for plan-to-timeline generation.
- [5] Ed. Desruisseaux, B. RFC 5545: Internet calendaring and scheduling core object specification (iCalendar). <https://datatracker.ietf.org/doc/html/rfc5545>, 2009. IETF Standards Track. Defines VEVENT with fields DTSTART, DTEND, SUMMARY, DESCRIPTION, CATEGORIES, STATUS (TENTATIVE, CONFIRMED, CANCELLED), and URL. The field naming is the dominant vocabulary donor for Timeline IR. The ICS wire format is not git-friendly and has no swimlane or visual-status concept.
- [6] ECMA International. ECMA-376: Office open XML file formats. <https://ecma-international.org/publications-and-standards/standards/ecma-376/>, 2026. Standard for Office Open XML, including DrawingML shapes and effects (a:glow, a:outerShdw, etc.) embedded in PPTX. Informs Timeline Compiler’s PPTX shape generation and styling fidelity.
- [7] Frappe Technologies. Frappe gantt: A modern, configurable gantt library for the web. <https://github.com/frappe/gantt>, 2024. MIT license. SVG-based interactive Gantt; used in ERPNext. No declarative text-format input; requires JavaScript task objects. No static file export built-in.
- [8] GitHub, Inc. ProjectV2 — GitHub GraphQL API reference. <https://docs.github.com/en/graphql/reference/objects#projectv2>, 2024.
- [9] GitHub, Inc. About GitHub projects — GitHub documentation. <https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>, 2024. ProjectV2 GraphQL API fields include DATE, ITERATION, SINGLE_SELECT custom fields for roadmap bars. Roadmap view became GA in March 2023.

- [10] Google Flutter Team. Writing a golden file test for package flutter. <https://github.com/flutter/flutter/wiki/Writing-a-golden-file-test-for-package%3Aflutter>, 2026. Reference-image / golden-image testing pattern for deterministic visual regression in timeline rendering. Essential for ensuring output consistency and catching unintended visual changes across builds.
- [11] Google, Inc. Skia: An open source 2d graphics library. <https://skia.org>, 2026. Open source 2D graphics engine. Powers Chrome, ChromeOS, Android, Flutter, and many other products. Candidate rendering backend for deterministic, high-quality SVG and raster timeline output.
- [12] ISO/TC 154. ISO 8601-1:2019 — date and time — representations for information interchange. <https://www.iso.org/standard/70907.html>, 2019. The dominant standard for date/time strings in structured data. Timeline IR should support ISO-8601 dates natively.
- [13] John MacFarlane. Pandoc: A universal document converter. <https://pandoc.org/>, 2024. MIT-licensed document converter. Pandoc filters could be used to embed Timeline Compiler output in converted documents.
- [14] Khronos Group. WebGL: 3d graphics for the web. <https://www.khronos.org/webgl/>, 2026. Low-level 3D graphics API based on OpenGL ES, implemented in all major browsers. Enables GPU-accelerated rendering of complex timeline visualizations in web contexts.
- [15] Knight Lab, Northwestern University. TimelineJS: Beautifully crafted timelines that are easy, and intuitive to use. <https://timeline.knightlab.com/>, 2024. Open source (GPL). Input via Google Sheets or JSON. Interactive web output; no native SVG/PDF/PPTX export. Storytelling-oriented, not presentation-oriented.
- [16] mark-when Contributors. Markwhen: An interactive text-to-timeline tool. <https://github.com/mark-when/markwhen>, 2024. MIT license (parser, view container, VS-Code extension). Web editor (markwhen.com) is proprietary. Markdown-inspired text format: `date: label` events grouped into `group:/section: blocks`. Swim-lanes supported; no PPTX/PDF export; no stable machine-verifiable schema; no theme/render separation.
- [17] Mermaid Chart, Inc. Mermaid chart: AI-powered diagramming for teams. <https://www.mermaidchart.com/>, 2024. Commercial SaaS layer on top of open-source Mermaid. \$7.5M seed funding (March 2024). Demonstrates the open-core commercial model for diagram tooling.
- [18] Mermaid Contributors. Timeline diagram — mermaid documentation. <https://mermaid.js.org/syntax/timeline.html>, 2024. Official documentation for the timeline diagram type, which graduated from beta in 2023. Sections group events; time axis accepts year, quarter, or ISO-8601 strings.
- [19] Mermaid Contributors. Gantt diagrams — mermaid documentation. <https://mermaid.js.org/syntax/gantt.html>, 2024. Documents the gantt diagram type with `dateFormat`, `section`, and `task` syntax. Export quality is SVG/PNG only; no native PPTX or PDF.
- [20] Mermaid-js Contributors. Mermaid: Generation of diagrams like flowcharts or sequence diagrams from text in a similar manner as markdown. <https://github.com/mermaid-js/mermaid>, 2023. MIT license. Over 88,000 GitHub stars as of 2026. Raised \$7.5M seed round (Sequoia/M12) in March 2024.

- [21] Microsoft Corporation. Microsoft project XML data interchange file format. <https://learn.microsoft.com/en-us/office-project/xml-reference/introduction-to-the-microsoft-project-xml-data-interchange-file-format>, 2023. Official schema reference for Project XML export format.
- [22] Microsoft Corporation. Work items — Get — Azure DevOps REST API reference. <https://learn.microsoft.com/en-us/rest/api/azure/devops/wit/work-items/get?view=azure-devops-rest-7.1>, 2024. Work items export fields include System.IterationPath, System.State, System.Title, Microsoft.VSTS.Scheduling.Effort, etc. Quarter/sprint granularity captured via IterationPath.
- [23] Microsoft Corporation. Microsoft project: Project management software. <https://www.microsoft.com/en-us/microsoft-365/project/project-management-software>, 2024. Proprietary. Exports to a custom XML schema (Project.xsd) that is non-deterministic between saves (GUIDs, timestamps change). Not git-diffable without post-processing.
- [24] Microsoft Corporation. Create a timeline — Microsoft PowerPoint support. <https://support.microsoft.com/en-us/office/create-a-timeline-ec28c7c7-7b0a-40da-9b74-9a85f8ab4b97>, 2024. Manual SmartArt-based timeline creation in PowerPoint. Not source-controlled; not deterministic; not agent-friendly.
- [25] Microsoft Research. Timeline storyteller: An expressive visual storytelling tool for presenting timelines. <https://github.com/Microsoft/timelinestoryteller>, 2019. Open-sourced 2019; MIT license. Research prototype accepting CSV or JSON array (start, end, label). No swimlanes, no status, no PPTX output, no active maintenance. Not adoptable.
- [26] Observable, Inc. Observable plot: The javascript library for exploratory data visualization. <https://github.com/observablehq/plot>, 2024. ISC license. Concise JavaScript charting API inspired by Vega-Lite grammar. No native swimlane roadmap type; requires imperative mark composition. Not suitable for adoption or declarative text-format authoring.
- [27] Obsidian. Obsidian: A second brain, for you, forever. <https://obsidian.md/>, 2024. Popular Markdown knowledge-management tool with native Mermaid rendering. Potential embedding target for Timeline Compiler output.
- [28] OpenAI. Structured outputs — OpenAI API documentation. <https://platform.openai.com/docs/guides/structured-outputs>, 2024. JSON Schema-constrained generation from LLMs. A well-specified Timeline IR with JSON Schema enables agents to generate valid IR directly.
- [29] PlantUML Contributors. PlantUML: Open-source tool to draw UML diagrams, using a simple and human readable text description. <https://plantuml.com/>, 2024. Outputs PNG, SVG, ASCII art, LaTeX, PDF via GraphViz/Ditaa. Gantt support (@startgantt) is experimental as of 2024. GPL/LGPL/Commercial licenses.
- [30] PlantUML Contributors. Gantt diagram — PlantUML documentation. <https://plantuml.com/gantt-diagram>, 2024.

- [31] Plotly Technologies Inc. Plotly.js: Open source graphing library. <https://github.com/plotly/plotly.js>, 2024. MIT license. Supports Gantt/timeline charts via horizontal bar charts. High-quality SVG export. Requires programming; no declarative text format.
- [32] Prabhu Murthy and contributors. react-chrono: A modern timeline component for React. <https://github.com/prabhuignoto/react-chrono>, 2024. MIT license. JavaScript items array: `title`, `cardTitle`, `cardDetailedText`. Date fields are plain strings with no semantic granularity. Browser-only; no swimlanes; no static SVG/PDF/PPTX export. Not adoptable.
- [33] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. Vega-Lite: A grammar of interactive graphics. In *IEEE Transactions on Visualization and Computer Graphics (Proc. InfoVis)*, volume 23, pages 341–350, 2017. doi: 10.1109/TVCG.2016.2599030. High-level declarative JSON grammar for interactive statistical graphics; BSD-3-Clause open source. Demonstrates how declarative IRs enable deterministic, composable rendering.
- [34] Scanny and contributors. python-pptx: Create and update PowerPoint files. <https://github.com/scanny/python-pptx>, 2024. MIT license. Python library for programmatic PPTX generation. Candidate for the PPTX output target of Timeline Compiler.
- [35] Schema.org Community Group. Event — schema.org. <https://schema.org/Event>, 2024. Schema.org type for events, defining properties `name`, `startDate`, `endDate`, `description`, `url`, `organizer`, and `eventStatus`. Open vocabulary maintained by W3C community. Not a visual-communication format, but confirms canonical web naming for timeline entity fields.
- [36] Simon Brown. Structurizr lite: C4 model architecture tooling. <https://github.com/structurizr/structurizr-lite>, 2024. Apache 2.0. Domain-specific to software architecture (C4 model); not a general-purpose timeline tool.
- [37] SlideScience. Ultimate guide to Gantt charts (timelines) in think-cell. <https://slidescience.co/gantt-charts-timelines-think-cell/>, 2024.
- [38] TaskJuggler Contributors. TaskJuggler: A free and open source project management tool. <https://taskjuggler.org/>, 2024. GPL v2. Plain-text `.tjp` DSL with `project`, `task`, `resource`, `depends` keywords. Full scheduling grammar: dependency resolution, resource levelling, critical path. Out of scope for visual-communication timelines; no vocabulary worth borrowing beyond what iCalendar and schema.org already provide.
- [39] Terrastruct, Inc. D2: A modern diagram scripting language that turns text to diagrams. <https://d2lang.com/>, 2024. MPL-2.0 license. Minimalist syntax, auto-layout. No native timeline/gantt type.
- [40] think-cell Software GmbH. think-cell: Intelligent charting and slide automation software for PowerPoint. <https://www.think-cell.com/>, 2024. Proprietary PowerPoint add-in. Approx. \$27.30/user/month (2026 pricing). Industry standard for McKinsey/BCG-style Gantt and waterfall charts. Not open source; not source-control friendly.
- [41] visjs Contributors. vis-timeline: Create fully customizable, interactive timelines and 2d graphs with items and groups. <https://github.com/visjs/vis-timeline>,

2024. MIT license. Browser-only interactive timeline; no native static SVG/PDF export. Successor to the original vis.js timeline component.
- [42] W3C OWL Time Ontology Working Group. OWL-Time: Time ontology in OWL — W3C recommendation. <https://www.w3.org/TR/owl-time/>, 2022. W3C Recommendation (updated 2022). Defines TemporalEntity, Instant, Interval, and properties hasBeginning, hasEnd, hasTemporalDuration. Omitting hasEnd denotes an open/ongoing interval, validating the IR’s end: ongoing semantics. Implements Allen’s 13 interval relations.
- [43] Hadley Wickham. A layered grammar of graphics. *Journal of Computational and Graphical Statistics*, 19(1):3–28, 2010. doi: 10.1198/jcgs.2009.07098. Formalizes the Grammar of Graphics as a layered system (data → aesthetics → geom → stat → coord → facet). Powers ggplot2.
- [44] Wikipedia contributors. Mermaid (software) — Wikipedia, the free encyclopedia. [https://en.wikipedia.org/wiki/Mermaid_\(software\)](https://en.wikipedia.org/wiki/Mermaid_(software)), 2024.
- [45] Leland Wilkinson. *The Grammar of Graphics*. Springer, New York, 2 edition, 2005. ISBN 978-0-387-24544-7. Foundational theory decomposing statistical graphics into data, aesthetics, geometry, statistics, scales, coordinates, and facets. Basis for ggplot2 and Vega-Lite.